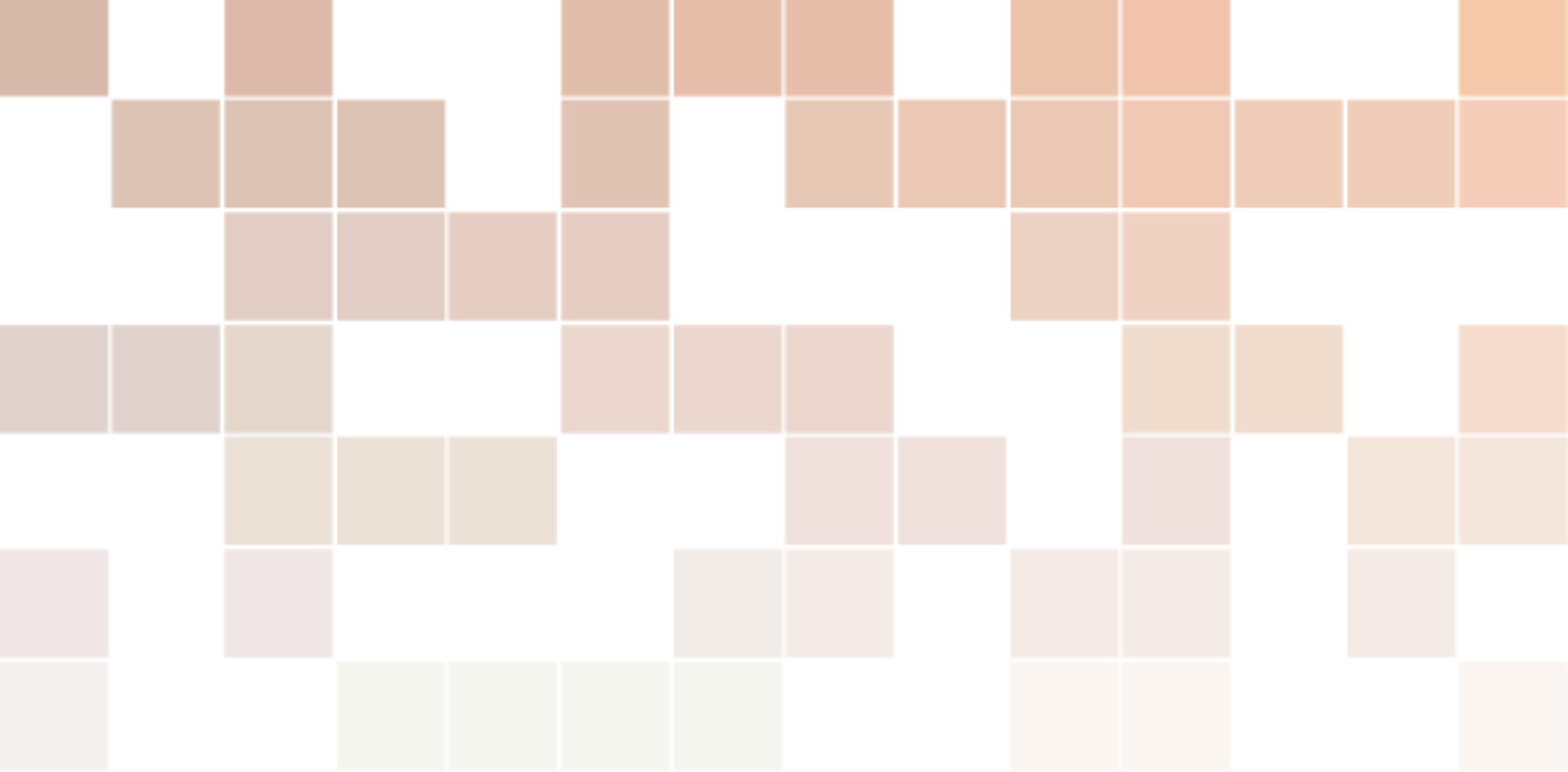
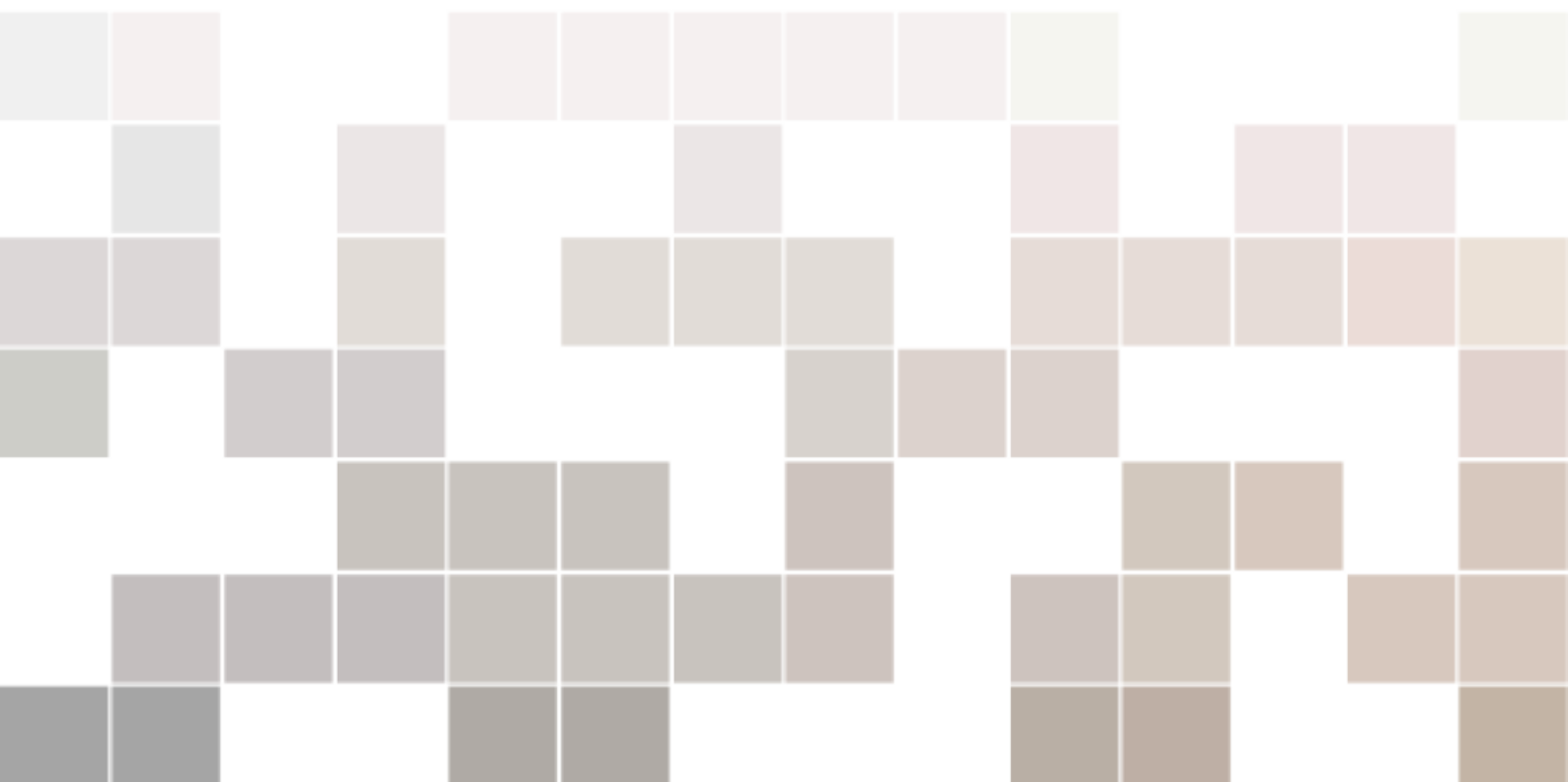




强化学习和多模态教程

理论与实践

左元



目录

III

多模态

1	Vision Transformer	19
1.1	导入库和模块	20
1.2	补丁嵌入 (Patch Embedding)	20
1.3	类别对应的 token 和位置编码	22
1.4	注意力头	24
1.5	多头注意力	26
1.6	Transformer 编码器	27
1.7	Vision Transformer	28
1.8	训练参数	30
1.9	加载 MNIST 数据集	30
1.10	训练循环	31
1.11	评估模型	31
2	CLIP	33
2.1	导入库和模块	33
2.2	图像和文本编码器	34
2.2.1	位置嵌入	34
2.2.2	注意力头	34
2.2.3	多头注意力	35
2.2.4	Transformer 编码器	35
2.2.5	文本的分词器	36
2.2.6	文本编码器	37
2.2.7	图像编码器	39
2.3	CLIP 模型	40
2.3.1	数据	43
2.3.2	训练参数	45

2.3.3	加载数据集	46
2.3.4	训练模型	46
2.3.5	评估模型	47
2.3.6	零样本分类	48
3	ClipCap (图生文)	51
3.1	ClipCap 的工作原理	51
3.2	关于训练	52
3.3	推理	52
3.4	ClipCap 的代码实现	53
3.4.1	项目配置	53
3.4.2	处理数据	53
3.4.3	模型结构	55
3.4.4	准备训练数据集	57
3.4.5	训练	58
3.4.6	推理	60
3.5	应用：图生文	63
3.6	拓展：Qwen3-VL	64
4	扩散模型	65
4.1	通俗讲解	65
4.1.1	扩散	65
4.1.2	扩散模型的架构	66
4.1.3	模型设计	68
4.2	扩散模型理论简介	70
4.2.1	概述	70
4.2.2	正向扩散过程	71
4.2.3	反向扩散过程	74
4.2.4	U-Net 模型	74
4.3	代码实现	79
4.3.1	扩散过程相关代码	79
4.3.2	U-Net 神经网络	81
4.4	完整代码	89
5	扩散模型背后的数学理论	97
5.1	正向扩散过程	98
5.2	逆向扩散过程	101
6	条件扩散模型	109
6.1	向扩散模型添加条件	109
6.2	条件扩散模型的实现	111
6.3	得分函数	119
6.3.1	什么是得分函数	119
6.3.2	式 (6.5) 的证明	120
6.4	分类器指引	121
6.4.1	什么是分类器	121

6.4.2	分类器指引的推导	121
6.5	无分类器指引	123
6.5.1	无分类器指引的理论知识	123
6.5.2	无分类器指引的实现	124
6.6	Stable Diffusion	131
7	DALL-E2 (文生图)	133
7.1	CLIP	134
7.2	先验模型 (Prior Model)	134
7.2.1	先验模型的训练	135
7.2.2	先验模型的推理	136
7.2.3	关键代码解释	136
7.3	解码器	148
7.3.1	概述	148
7.3.2	残差块	149
7.3.3	注意力块	152
7.3.4	下采样	155
7.3.5	上采样	155
7.3.6	模型的训练	156
7.3.7	损失的计算	165
7.4	训练配置	166
7.5	训练	168

List of Figures

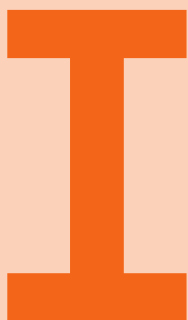
1.1 ViT 架构图	19
1.2 补丁的例子	20
1.3 将图像转换为补丁：第一步	21
1.4 将图像转换为补丁：第二步	22
1.5 将图像转换为补丁：第三步	22
1.6 位置编码的作用	23
1.7 左图：使用 Conv2d 运算分成 16 个 8x8 块的 32x32 MNIST 图像。右图：添加位置编码和类别 token 后的 16 个图像补丁，使用随机数据初始化。	24
1.8 注意力机制	25
2.1 clip 原理，来自原始论文	33
2.2 注意力分数的掩码	35
2.3 分词过程	37
2.4 余弦相似度的计算	42
2.5 复制掩码	44
2.6 因果注意力分数的掩码	45
3.1 图片标题模型的简单示例	51
3.2 将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间	52
3.3 ClipCap 的推理过程	53
3.4 clipcap 的模型设计要点	55
3.5 clipcap 的训练目标	58
3.6 clipcap 只根据图片的嵌入预测文本的 token，也就是看图说话，不再需要文本了！	60
3.7 A800 训练 10 个小时的效果	63
3.8 Qwen2.5-VL 框架展示了视觉编码器与语言模型解码器的集成，用以处理包括图像和视频在内的多模态输入。视觉编码器被设计为处理原生分辨率的输入并支持动态帧率采样。不同尺寸的图像和具有不同时帧率的视频帧被动态映射为长度各异的 token 序列。值得注意的是，MRoPE 在时间维度上将时间 ID 与绝对时间对齐，使模型能够更好地理解时间动态，例如事件的节奏和精确的时刻定位。处理后的视觉数据随后被输入到 Qwen2.5 LM 解码器。我们对 ViT 架构进行了重构，加入了诸如带 SwiGLU 激活的前馈网络（FFN）、用于归一化的 RMSNorm 以及基于窗口的注意力机制等先进组件，以提升性能和效率。	64
4.1 水滴的扩散过程	65
4.2 高清图像的扩散过程	66

4.3	从高斯分布中采样一个随机值	67
4.4	预测图像与实际图像之间的差异使用损失函数计算	67
4.5	将噪声作为预测目标	68
4.6	U-net 架构 (以最低分辨率为 32×32 像素为例)。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 x-y 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。	69
4.7	共享模型	70
4.8	正向扩散和逆向扩散	70
4.9	正向扩散公式图解	71
4.10	以噪声为预测目标	74
4.11	训练 unet	75
4.12	U-net 训练算法	75
4.13	扩散模型训练步骤	76
4.14	采样算法 (去噪算法, 反向扩散算法)	77
4.15	扩散模型的采样过程	78
4.16	以噪声为预测目标	80
4.17	采样算法 (去噪算法, 反向扩散算法)	81
4.18	U-Net 执行语义分割	81
4.19	本章要实现的 U-Net	82
4.20	ConvBlock 类执行的处理	83
4.21	我们实现的 U-Net	86
4.22	加入正弦位置编码的 U-Net	87
4.23	新的 ConvBlock 类	88
5.1	郎之万动力学	97
5.2	正向扩散和逆向扩散	98
5.3	正向扩散公式图解	98
5.4	重参数技巧的使用	100
5.5	去噪的解析解无法计算	102
5.6	通过贝叶斯定理可以对反向分布 $q(x_{t-1} x_t)$ 进行推断。右图上的红色曲线显示了用 3 个高斯分布混合表示的边缘分布 $q(x_{t-1})$, 而左图显示了以 x_{t-1} 为中心的高斯前向噪声过程 $q(x_t x_{t-1})$ 作为 x_t 的分布。通过将它们相乘并进行归一化, 我们得到了蓝色曲线所示的对于特定选择的 x_t 的分布 $q(x_{t-1} x_t)$ 。因为左图的分布相对较宽, 对应着较大的方差 β_t , 所以分布 $q(x_{t-1} x_t)$ 具有复杂的多峰结构。	102
5.7	左图中的高斯分布 $q(x_t x_{t-1})$ 的方差 β_t 要小的多。我们可以看到右图中相应的分布 $q(x_t x_{t-1})$ (蓝色) 接近高斯分布, 具有与 $q(x_t x_{t-1})$ 类似的方差	103
5.8	高斯混合模型 (GMM)	104
6.1	本章将要实现的条件扩散模型	109
6.2	预测时刻 $t-1$ 的图像 (正态分布的均值向量) 的横型 (上) 和预测添加到 x_t 中的噪声的模型 (下)	110
6.3	条件扩散模型的神经网络	111
6.4	在扩散模型中添加嵌入层	111
6.5	扩散模型中使用的推断噪声 ϵ 的神经网络	119
6.6	用于推断得分的神经网络	119
6.7	使用预训练神经网络进行分类的示例 (参数为 ϕ)	121
6.8	通过反向传播可以求出 $\nabla_{x_t} \log p(y x_t)$	122
6.9	使用了推断得分的神经网络的分类器指引	123
6.10	上面式子的可视化	124
6.11	Stable Diffusion 生成的高清图像	131
7.1	DALL-E2 模型整体架构	134
7.2	先验模型架构图	135
7.3	线性调度方差和余弦调度方差的对比	136

7.4	解码器架构	148
7.5	残差块的架构	149
7.6	注意力块	152
7.7	解码器训练流程	156



List of Tables



强化学习

II 基于人类反馈的强化学习



多模态

1	Vision Transformer	19
1.1	导入库和模块	20
1.2	补丁嵌入 (Patch Embedding)	20
1.3	类别对应的 token 和位置编码	22
1.4	注意力头	24
1.5	多头注意力	26
1.6	Transformer 编码器	27
1.7	Vision Transformer	28
1.8	训练参数	30
1.9	加载 MNIST 数据集	30
1.10	训练循环	31
1.11	评估模型	31
2	CLIP	33
2.1	导入库和模块	33
2.2	图像和文本编码器	34
2.3	CLIP 模型	40
3	ClipCap (图生文)	51
3.1	ClipCap 的工作原理	51
3.2	关于训练	52
3.3	推理	52
3.4	ClipCap 的代码实现	53
3.5	应用: 图生文	63
3.6	拓展: Qwen3-VL	64
4	扩散模型	65
4.1	通俗讲解	65
4.2	扩散模型理论简介	70
4.3	代码实现	79
4.4	完整代码	89
5	扩散模型背后的数学理论	97
5.1	正向扩散过程	98
5.2	逆向扩散过程	101
6	条件扩散模型	109
6.1	向扩散模型添加条件	109
6.2	条件扩散模型的实现	111
6.3	得分函数	119
6.4	分类器指引	121
6.5	无分类器指引	123
6.6	Stable Diffusion	131
7	DALL-E2 (文生图)	133
7.1	CLIP	134
7.2	先验模型 (Prior Model)	134
7.3	解码器	148
7.4	训练配置	166
7.5	训练	168

1. Vision Transformer

🔥 提示

ViT: Vision Transformer

基于自注意力的 Transformer 模型由 Vaswani 等人在 2017 年的论文《Attention Is All You Need》中首次提出，并已广泛应用于自然语言处理中。Transformer 模型是 OpenAI 用来创建 ChatGPT 的模型。Transformer 不仅适用于文本，还适用于图像，基本上可以处理任何序列数据。2021 年，Dosovitsky 等人在他们的论文《An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale》中引入了将 Transformer 用于计算机视觉任务（例如图像分类）的想法。在他们的论文中，与卷积网络相比，他们的 Vision Transformer 模型能够取得出色的结果，并且需要更少的资源来训练。

在本教程中，我们将从头开始构建一个 Vision Transformer 模型，并在 MNIST 数据集上进行测试。

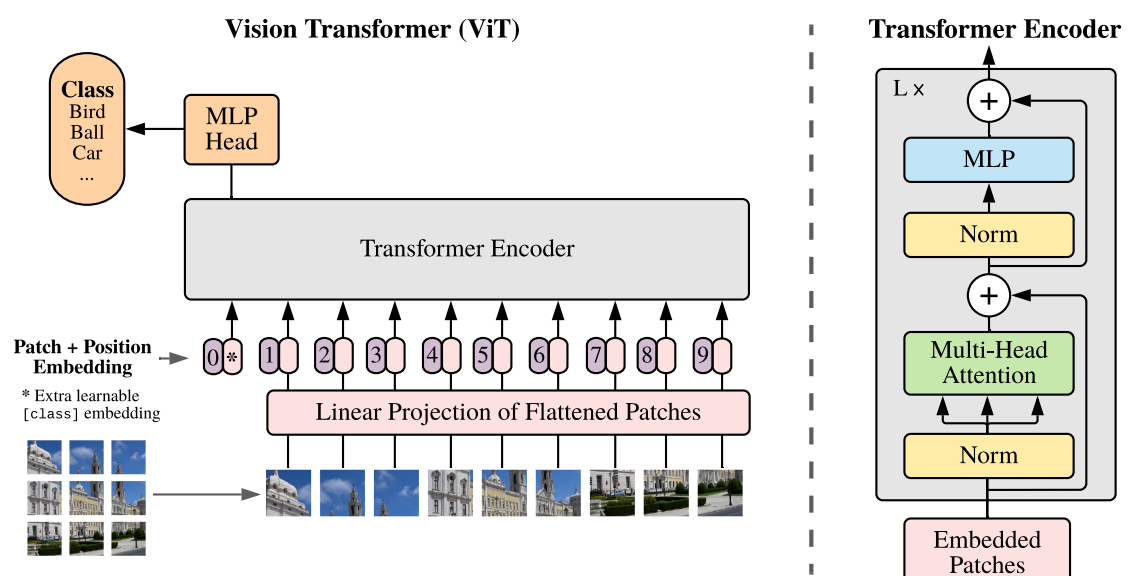


图 1.1 Vit 架构图

1.1 导入库和模块

本章所需依赖

Python

```

1 import torch
2 import torch.nn as nn
3 import torchvision.transforms as T
4 from torch.optim import Adam
5 from torchvision.datasets.mnist import MNIST
6 from torch.utils.data import DataLoader
7 import numpy as np

```

1.2 补丁嵌入 (Patch Embedding)

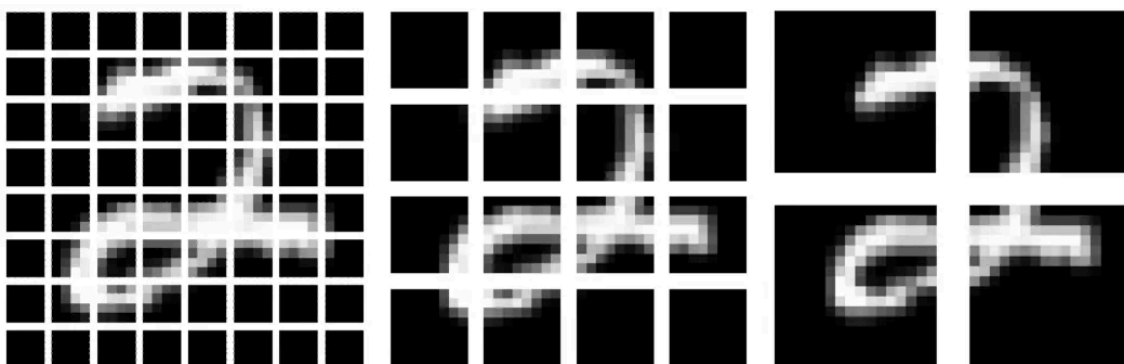


图 1.2 补丁的例子

补丁嵌入类

Python

```

1 class PatchEmbedding(nn.Module):
2     def __init__(
3         self,
4         d_model,    # 模型的维度
5         img_size,   # 图片大小
6         patch_size, # 补丁大小
7         n_channels  # 通道数量
8     ):
9         super().__init__()
10
11         self.d_model = d_model
12         self.img_size = img_size
13         self.patch_size = patch_size
14         self.n_channels = n_channels
15
16         self.linear_project = nn.Conv2d(
17             self.n_channels, # in_channels
18             self.d_model, # out_channels
19             kernel_size=self.patch_size, # kernel_size
20             stride=self.patch_size # stride
21         )

```

```

22
23     # B: 批次大小
24     # C: 通道数量
25     # H: 图像高度
26     # W: 图像宽度
27     # P_col: 补丁的列
28     # P_row: 补丁的行
29     def forward(self, x):
30         x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)
31         x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)
32         x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
33         return x

```

创建 Vision Transformer 的第一步是将输入图像拆分为补丁，并创建这些补丁的线性嵌入序列。我们能够通过使用 PyTorch 的 Conv2d 方法来实现这一点。

Conv2d 方法获取输入图像，将它们拆分为补丁，并提供大小等于 `d_model` 的线性投影。通过将 `kernel_size` 和步幅设置为补丁大小，我们确保补丁大小正确且没有重叠。

```

1 self.linear_project = nn.Conv2d(
2     self.n_channels,
3     self.d_model,
4     kernel_size=self.patch_size,
5     stride=self.patch_size
6 )

```

[Python](#)

在 `forward` 方法中，我们通过 `linear_project/Conv2d` 方法传递具有形状 (B, C, H, W) 的输入，并输出形状 $(B, d_model, P_col, P_row)$ 的张量。

```

1 def forward(self, x):
2     x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)

```

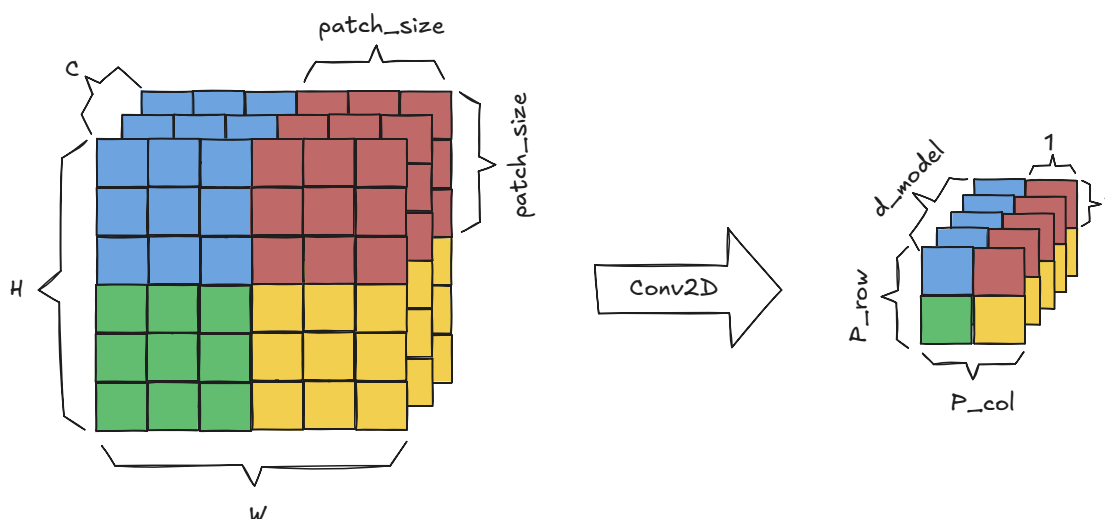
[Python](#)


图 1.3 将图像转换为补丁：第一步

我们使用展平方法将补丁列和补丁行维度组合成一个补丁维度，从而得到 (B, d_model, P) 的形状

```

1 x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)

```

[Python](#)

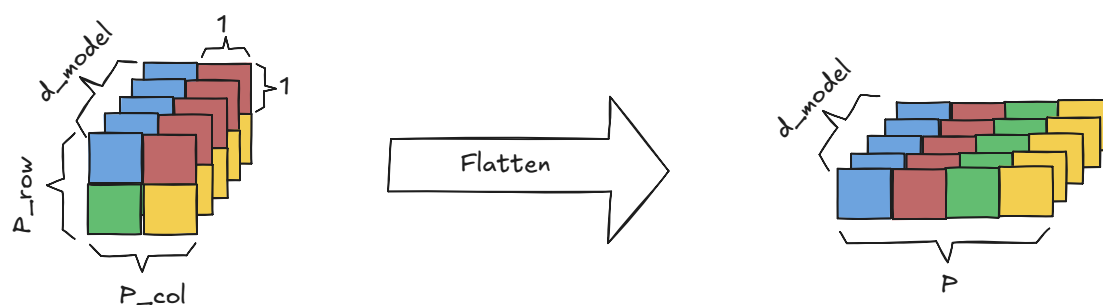


图 1.4 将图像转换为补丁：第二步

最后，我们使用转置方法切换 d_model 和补丁维度，得到 (B, P, d_model) 的形状。

```
1 x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
```

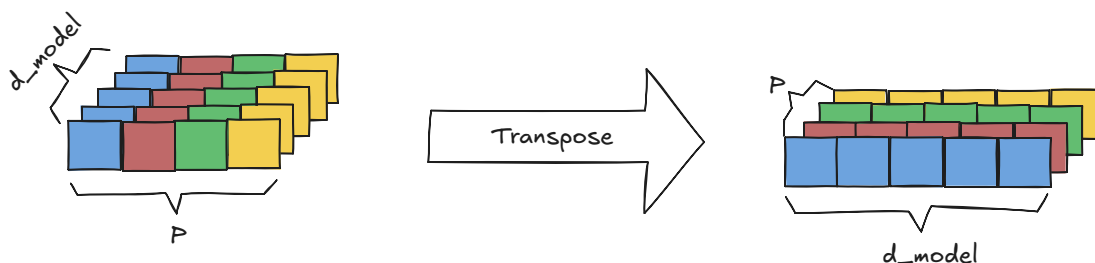
[Python](#)


图 1.5 将图像转换为补丁：第三步

1.3 类别对应的 token 和位置编码

位置编码类

[Python](#)

```
1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_seq_length):
3         super().__init__()
4         # 类别token
5         self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
6         # 创建位置编码
7         pe = torch.zeros(max_seq_length, d_model)
8
9         for pos in range(max_seq_length):
10             for i in range(d_model):
11                 if i % 2 == 0:
12                     pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
13                 else:
14                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
15         # 将位置编码固定
16         self.register_buffer("pe", pe.unsqueeze(0))
17
18     def forward(self, x):
19         # 为批次中的每张图片分配一个类别token
```

```

20 tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
21 # 将类别token添加到每个图像的补丁嵌入数组的开头
22 x = torch.cat((tokens_batch,x), dim=1)
23 # 将位置编码添加到嵌入中
24 x = x + self.pe
25 return x

```

ViT 模型使用向补丁嵌入添加可学习的类别 token 的标准方法来执行分类。

```

1 # class token: 类别或者分类对应的token
2 self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))

```

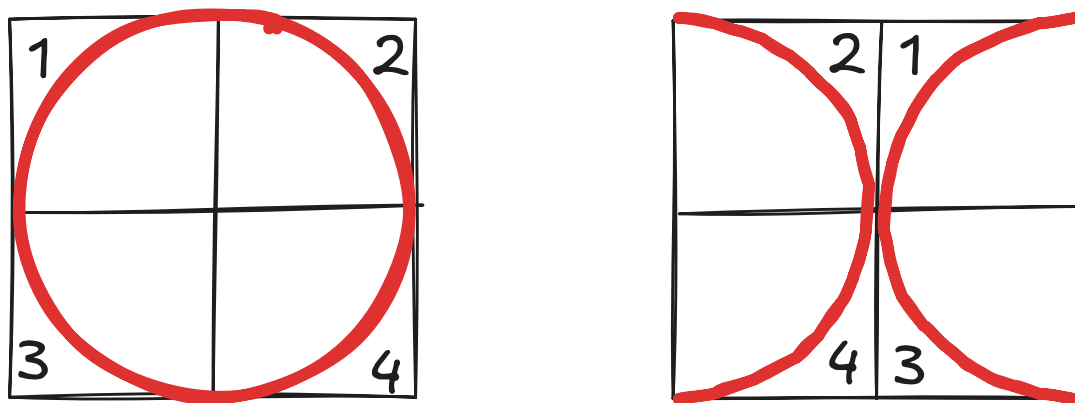
 Python


图 1.6 位置编码的作用

与 LSTM 等按顺序接受嵌入的模型不同，Transformer 并行的接受嵌入。虽然这提高了速度，但 Transformer 不知道序列的顺序是什么。这是一个很大的问题，因为改变序列的顺序很可能会改变其含义。上图就是一个例子，它显示更改图像的补丁顺序可以将图像从 0 更改为更接近 X 的图像。为了解决这个问题，需要将位置编码添加到嵌入中。每个位置编码对于它所表示的位置都是唯一的，这允许模型识别每个补丁嵌入应该在的位置。为了将位置编码添加到补丁嵌入中，它们必须具有相同的维度， d_{model} 。我们使用下面的公式获得位置编码。

定义 1.1 — 位置编码.

$$\begin{aligned}
 \text{PE}_{(\text{pos}, 2i)} &= \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \\
 \text{PE}_{(\text{pos}, 2i+1)} &= \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)
 \end{aligned} \tag{1.1}$$

```

1 # max_seq_length: 补丁嵌入的数量, d_model: 嵌入维度
2 pe = torch.zeros(max_seq_length, d_model)
3
4 for pos in range(max_seq_length):
5     for i in range(d_model):
6         if i % 2 == 0:
7             pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
8         else:
9             pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))

```

 Python

```
10 # 禁止pe更新, pe保存在了显存中, 不会被反向传播更新
11 self.register_buffer('pe', pe.unsqueeze(0))
```

在 `forward` 方法中, 输入是多个图像的一批补丁嵌入。例如, 32×32 的图像可以分解为 16 个 8×8 大小的补丁。在此 `max_seq_length` 中, 需要为 $16+1=17$ 才能创建足够的位置嵌入, 每个补丁一个, 分类对应的 `token` 一个。

因此, 我们需要使用 `expand` 函数才能使用 `self.cls_token` 为批处理中的每个图像创建分类对应的 `token`。注意力机制会将有关整个序列的信息编码到序列中的每个 `token` 中。由于每个 `token` 都受到其自身信息的偏见, 因此分类对应的 `token` 会创建序列中所有 `token` 的独立的摘要信息。

```
1 def forward(self, x):
2     tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
```

Python

然后, 使用 `torch.cat` 方法将这些分类 `token` 添加到每个补丁嵌入的开头。

```
1 x = torch.cat((tokens_batch, x), dim=1)
```

Python

位置编码在输出之前添加。

```
1 x = x + self.pe
2 return x
```

Python

这是添加分类 `token` 之前和之后的数据的样子:

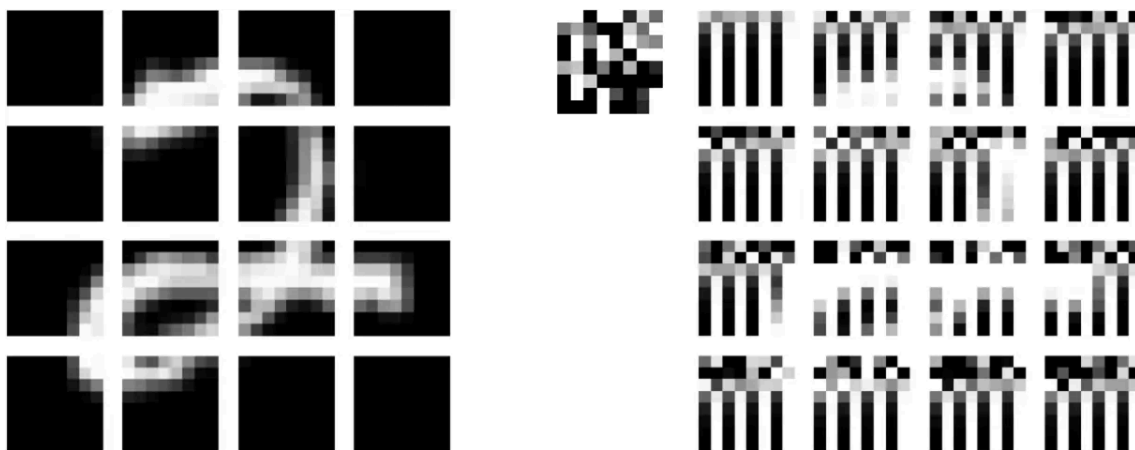


图 1.7 左图: 使用 `Conv2d` 运算分成 16 个 8×8 块的 32×32 MNIST 图像。右图: 添加位置编码和类别 `token` 后的 16 个图像补丁, 使用随机数据初始化。

请注意, 我们已经用 64 个卷积核初始化了 `Conv2d` 运算, 每个卷积核中的每个补丁只占用一个像素, 以免扭曲图像。

1.4 注意力头

注意力头

Python

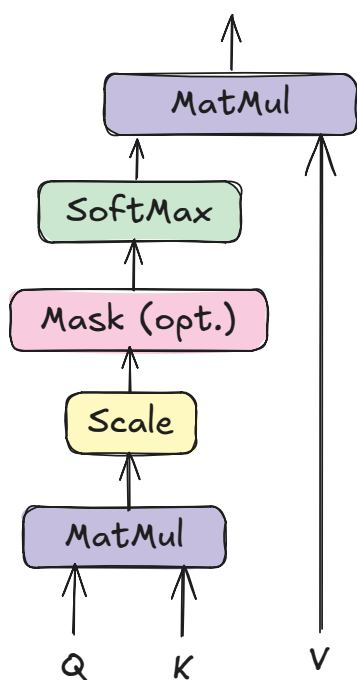
```
1 class AttentionHead(nn.Module):
2     def __init__(self, d_model, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(d_model, head_size)
```

```

7     self.key = nn.Linear(d_model, head_size)
8     self.value = nn.Linear(d_model, head_size)
9
10    def forward(self, x):
11        # 计算Q, K, V
12        Q = self.query(x)
13        K = self.key(x)
14        V = self.value(x)
15
16        #  $QK^T$ 
17        attention = Q @ K.transpose(-2, -1)
18
19        #  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}}\right)V$ 
20        attention = attention / (self.head_size ** 0.5)
21        attention = torch.softmax(attention, dim=-1)
22        attention = attention @ V
23
24    return attention

```

Scaled Dot-Product Attention



Multi-Head Attention

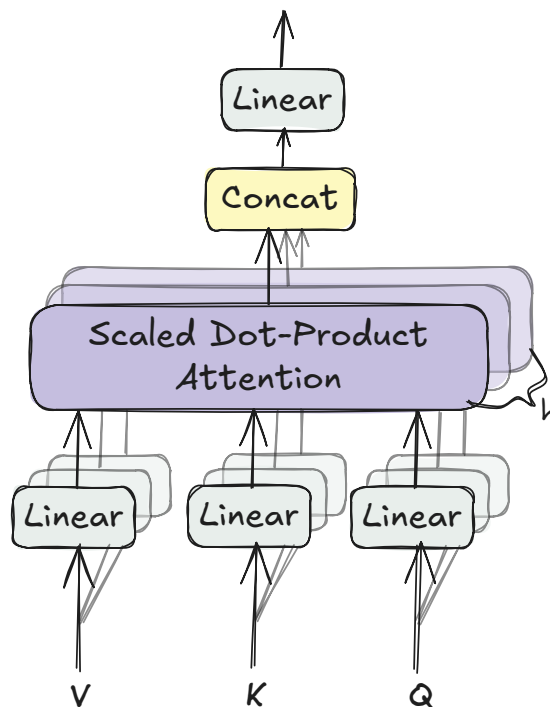


图 1.8 注意力机制

ViT 使用注意力机制，这是一种通信机制，允许模型专注于图像的重要部分。可以使用下面的公式计算注意力分数。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.2)$$

计算注意力的第一步是获取 token 的 Q、K 和 V。token 的 Q 是 token 要查找的内容，K 是 token 包含的内容，V 是 token 之间的互信息。Q、K 和 V 可以通过线性层传递 token 来计算。

```
1 def forward(self, x):
2     # 计算Q, K, V
3     Q = self.query(x)
4     K = self.key(x)
5     V = self.value(x)
```

 Python

我们能够通过计算 Q 和 K 的点积来获取序列中 token 之间的关系。

```
1 attention = Q @ K.transpose(-2, -1)
```

 Python

我们需要缩放这些值以控制初始化时的方差,以便 token 能够聚合来自多个其他 token 的信息。通过将点积除以注意力头大小的平方根来应用缩放。

```
1 attention = attention / (self.head_size ** 0.5)
```

 Python

然后,我们需要对缩放的点积应用 softmax。

```
1 attention = torch.softmax(attention, dim=-1)
```

 Python

最后,我们需要得到 softmax 和 V 矩阵之间的点积。这本质上是在相应的 token 之间传递信息。

```
1 attention = attention @ V
2 return attention
```

 Python

1.5 多头注意力

多头注意力

 Python

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         self.head_size = d_model // n_heads
5         self.W_o = nn.Linear(d_model, d_model)
6         self.heads = nn.ModuleList([
7             AttentionHead(d_model, self.head_size) for _ in range(n_heads)
8         ])
9
10    def forward(self, x):
11        # 拼接多个注意力头
12        out = torch.cat([head(x) for head in self.heads], dim=-1)
13        out = self.W_o(out)
14        return out
```

多头注意力只是并行运行多个头的自注意力并将它们组合起来。我们可以通过将注意力头添加到模块列表中来做到这一点,

```
1 self.heads = nn.ModuleList([
2     AttentionHead(d_model, self.head_size)
3     for _ in range(n_heads)
4 ])
```

 Python

传递输入然后拼接。

```
1 def forward(self, x):
```

 Python


```

2     # 拼接多个注意力头
3     out = torch.cat([head(x) for head in self.heads], dim=-1)

```

然后，我们需要将输出传递给另一个线性层。

```

1 out = self.W_o(out)
2 return out

```

 Python

1.6 Transformer 编码器

Transformer编码器

 Python

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, d_model, n_heads, r_mlp=4):
3         super().__init__()
4         self.d_model = d_model
5         self.n_heads = n_heads
6
7         # 层归一化
8         self.ln1 = nn.LayerNorm(d_model)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(d_model, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(d_model)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(d_model, d_model*r_mlp),
19            nn.GELU(),
20            nn.Linear(d_model*r_mlp, d_model)
21        )
22
23    def forward(self, x):
24        out = x + self.mha(self.ln1(x)) # 跳跃连接
25        out = out + self.mlp(self.ln2(out)) # 跳跃连接
26        return out

```

Transformer 编码器由两个子层组成：第一个子层执行多头注意力，第二个子层包含 MLP。多头注意力子层执行 token 之间的通信，而 MLP 子层允许 token 单独“思考”与它们通信的内容。

层归一化是一种优化技术，可跨其特征独立归一化批处理中的每个输入。对于我们的模型，我们将在每个子层的开头通过层归一化模块传递我们的输入。

```

1 self.ln1 = nn.LayerNorm(d_model)
2 self.ln2 = nn.LayerNorm(d_model)

```

 Python

MLP 将由两个线性层组成，中间有一个 GELU 层。使用 GELU 代替 RELU，因为它没有 RELU 在零点不可导的限制。

```

1 # 编码器的MLP
2 self.mlp = nn.Sequential(
3     nn.Linear(width, width*r_mlp),

```

 Python

```

4     nn.GELU(),
5     nn.Linear(width*r_mlp, width)
6 )

```

在编码器的 `forward` 方法中，输入在执行多头注意力之前通过第一层归一化模块。通过执行多头注意力将原始输入添加到输出中，以创建残差连接。

然后，在输入 MLP 之前，它会通过另一个层归一化模块。通过将 MLP 的输出添加到第一个残差连接的输出，创建另一个残差连接。

残差连接用于通过创建一条路径来帮助防止梯度消失问题，以便梯度不受阻碍地反向传播回原始输入。

```

1 def forward(self, x):
2     out = x + self.mha(self.ln1(x))
3     out = out + self.mlp(self.ln2(out))
4     return out

```

 Python

1.7 Vision Transformer

```

ViT类
1 class VisionTransformer(nn.Module):
2     def __init__(
3         self,
4         d_model,
5         n_classes,
6         img_size,
7         patch_size,
8         n_channels,
9         n_heads,
10        n_layers
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size 必须能被 patch_size 整除"
17        assert d_model % n_heads == 0, \
18            "d_model 必须能被 n_heads 整除"
19
20        self.d_model = d_model # 模型维度，嵌入的维度（宽度）
21        self.n_classes = n_classes # 类别的数量
22        self.img_size = img_size # 图片大小
23        self.patch_size = patch_size # 补丁大小
24        self.n_channels = n_channels # 通道数
25        self.n_heads = n_heads # 注意力头的数量
26        # 补丁的数量 =  $\frac{32 \times 32}{4 \times 4}$ 
27        self.n_patches = (self.img_size[0] * self.img_size[1]) \
28            // (self.patch_size[0] * self.patch_size[1])
29        # 序列的长度 = 1（分类token） + 补丁的数量
30        self.max_seq_length = self.n_patches + 1
31        # 补丁嵌入

```

 Python

```

32         self.patch_embedding = PatchEmbedding(
33             self.d_model,
34             self.img_size,
35             self.patch_size,
36             self.n_channels
37         )
38         # 位置编码
39         self.positional_encoding = PositionalEncoding(
40             self.d_model,
41             self.max_seq_length
42         )
43         self.transformer_encoder = nn.Sequential(*[
44             TransformerEncoder(self.d_model, self.n_heads)
45             for _ in range(n_layers)
46         ])
47
48         # 用于分类的MLP
49         self.classifier = nn.Sequential(
50             nn.Linear(self.d_model, self.n_classes),
51             nn.Softmax(dim=-1)
52         )
53
54     def forward(self, images):
55         # 将图片转换成补丁的嵌入 (embedding)
56         x = self.patch_embedding(images)
57         # 添加位置编码
58         x = self.positional_encoding(x)
59         # 编码
60         x = self.transformer_encoder(x)
61         # 分类的线性层
62         x = self.classifier(x[:,0])
63         return x

```

在创建我们的 `VisionTransformer` 类时，我们首先需要确保输入图像可以均匀地分成大小为 `patch_size` 的补丁，并且模型的维数可以被注意力头的数量整除。

```

1  assert img_size[0] % patch_size[0] == 0 \
2      and img_size[1] % patch_size[1] == 0, \
3      "img_size 必须能被 patch_size 整除"
4  assert d_model % n_heads == 0, \
5      "d_model 必须能被 n_heads 整除"

```

 Python

我们还需要计算位置编码的最大序列长度，该长度等于补丁的数量加 1。补丁的数量可以通过将输入图像的高和宽的乘积除以补丁大小的高和宽的乘积来计算。

```

1  self.n_patches = (self.img_size[0] * self.img_size[1]) \
2      // (self.patch_size[0] * self.patch_size[1])
3  self.max_seq_length = self.n_patches + 1

```

 Python

`VisionTransformer` 还需要能够拥有多个编码器模块。这可以通过将编码器层列表放入顺序包装器中来实现。

```
1 self.encoder = nn.Sequential(*[
2     TransformerEncoder(self.d_model, self.n_heads) for _ in range(n_layers)
3 ])
```

VisionTransformer 模型的最后一部分是 MLP 分类头。它由一个线性层和一个 softmax 层组成。

```
1 self.classifier = nn.Sequential(
2     nn.Linear(self.d_model, self.n_classes),
3     nn.Softmax(dim=-1)
4 )
```

在 forward 方法中，输入图像首先经过补丁嵌入层，将图像分割成补丁，并获取这些补丁的线性嵌入序列。然后，它们经过位置编码层，添加分类 token 和位置编码，最后经过编码器模块。之后，分类 token 再经过分类 MLP，以确定图像的分类。

```
1 def forward(self, images):
2     x = self.patch_embedding(images)
3     x = self.position_embedding(x)
4     x = self.encoder(x)
5     x = self.classifier(x[:,0])
6     return x
```

我们已经完成了模型的构建。现在我们需要对其进行训练和测试。

1.8 训练参数

```
1 d_model = 9 # 嵌入的维度9
2 n_classes = 10 # 类别数量为10
3 img_size = (32,32) # 图片大小为32x32
4 patch_size = (16,16) # 补丁的大小是16x16
5 n_channels = 1 # 灰度图片通道数量为1
6 n_heads = 3 # 3个注意力头
7 n_layers = 3 # 3层编码器
8 batch_size = 128 # 每个批次128张图片
9 epochs = 5 # 训练5个epoch
10 alpha = 0.005 # 学习率5e-3
```

1.9 加载 MNIST 数据集

```
1 transform = T.Compose([
2     T.Resize(img_size), # 28x28 → 32x32
3     T.ToTensor() # 转换成torch.tensor
4 ])
5
6 train_set = MNIST(
7     root="datasets", train=True, download=True, transform=transform
8 )
9 test_set = MNIST(
10    root="datasets", train=False, download=True, transform=transform
11 )
12
```

```
13 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
14 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

1.10 训练循环

下面的代码使用 MNIST 训练集训练我们的 VisionTransformer 类，并展示了整个 epoch 的平均损失。

```
1  device = torch.device("cuda")
2
3  ViT = VisionTransformer(
4      d_model,
5      n_classes,
6      img_size,
7      patch_size,
8      n_channels,
9      n_heads,
10     n_layers
11 ).to(device)
12
13 optimizer = Adam(ViT.parameters(), lr=alpha)
14 criterion = nn.CrossEntropyLoss()
15
16 for epoch in range(epochs):
17     training_loss = 0.0
18     for i, data in enumerate(train_loader, 0):
19         # 取出图像和对应的标签
20         inputs, labels = data
21         inputs, labels = inputs.to(device), labels.to(device)
22
23         optimizer.zero_grad()
24         outputs = ViT(inputs)
25         loss = criterion(outputs, labels)
26         loss.backward()
27         optimizer.step()
28
29         training_loss += loss.item()
30
31     print(f"Epoch {epoch + 1}/{epochs} loss: \
32           {training_loss / len(train_loader) :.3f}")
```

1.11 评估模型

```
1  correct = 0
2  total = 0
3
4  with torch.no_grad():
5      for data in test_loader:
6          images, labels = data
7          images, labels = images.to(device), labels.to(device)
```

```
8     outputs = ViT(images)
9     _, predicted = torch.max(outputs.data, 1)
10    total += labels.size(0)
11    correct += (predicted == labels).sum().item()
12    print(f"\n预测准确率: {100 * correct // total} %")
```

使用此模型，我们仅用 5 个 `epoch` 就能够在 MNIST 数据集上实现约 92% 的准确率。此示例展示了自注意力机制可以替代深度卷积网络。

2. CLIP

计算机视觉系统历来仅限于一组固定的类别，CLIP 是一场革命，它允许通过“预测哪些图像和文本配对在一起”来识别开放世界中的对象。CLIP 能够通过学习批量训练数据的图像和文本特征之间的余弦相似度来预测这一点。这在下图的对比预训练部分显示，其中图像之间的点积特征 $\{I_1, I_2, \dots, I_N\}$ 和文本特征 $\{T_1, T_2, \dots, T_N\}$ 被占用。

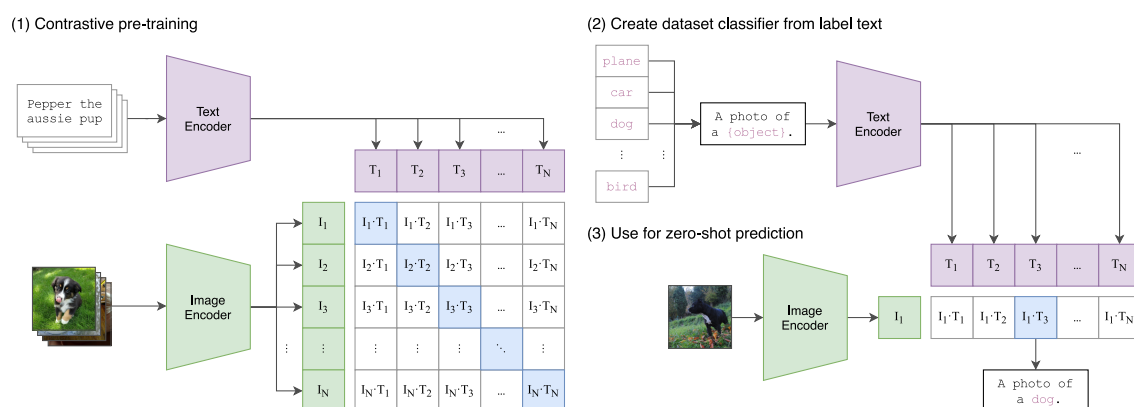


图 2.1 clip 原理，来自原始论文

在本章中，我们将从头开始构建 CLIP 并在 MNIST 数据集上对其进行测试。

2.1 导入库和模块

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision.transforms as T
5 from torch.utils.data import Dataset, DataLoader
6 from datasets import load_dataset
7 import matplotlib.pyplot as plt
8 import numpy as np
```

Python

2.2 图像和文本编码器

我们将首先构建图像和文本编码器。两者分别将图像和文本嵌入到单个 token 中, 然后可用于对比损失计算。

2.2.1 位置嵌入

```
1 class PositionalEmbedding(nn.Module):
2     def __init__(self, width, max_seq_length):
3         super().__init__()
4         # 创建一个 (token的数量, 嵌入的维度) 形状的全0张量
5         # width 就是 d_model
6         pe = torch.zeros(max_seq_length, width)
7         # 将位置编码信息填充到pe中
8         for pos in range(max_seq_length):
9             for i in range(width):
10                 if i % 2 == 0:
11                     pe[pos][i] = np.sin(pos/(10000 ** (i/width)))
12                 else:
13                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/width)))
14         # 位置编码信息进行冻结, 不参与反向传播
15         self.register_buffer('pe', pe.unsqueeze(0))
16
17     def forward(self, x):
18         # 直接将位置编码和token的嵌入进行相加
19         x = x + self.pe
20         return x
```

2.2.2 注意力头

```
1 class AttentionHead(nn.Module):
2     def __init__(self, width, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(width, head_size)
7         self.key = nn.Linear(width, head_size)
8         self.value = nn.Linear(width, head_size)
9
10    def forward(self, x, mask=None):
11        Q = self.query(x)
12        K = self.key(x)
13        V = self.value(x)
14
15        attention = Q @ K.transpose(-2, -1)
16        attention = attention / (self.head_size ** 0.5)
17        # 掩码
18        if mask is not None:
19            attention = attention.masked_fill(mask == 0, float("-inf"))
20        attention = torch.softmax(attention, dim=-1)
```



```

21     attention = attention @ V
22     return attention

```

Transformer 编码器和解码器之间的主要区别在于解码器使用注意力掩码，而编码器则不使用。虽然 CLIP 是仅编码器模型 (Encoder-Only)，但由于在分词时，对输入文本添加了 pad (填充符)，所以仍然需要与文本编码器一起使用掩码。请注意，掩码是可选的，因此这个注意力头可用于文本和视觉编码器。

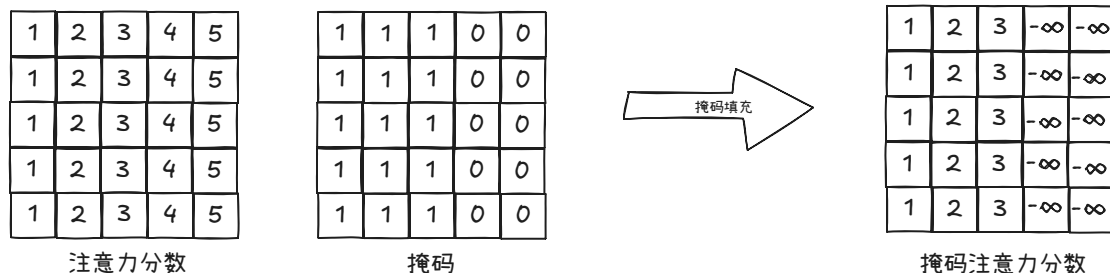


图 2.2 注意力分数的掩码

```

1 # 使用注意力掩码
2 if mask is not None:
3     attention = attention.masked_fill(mask == 0, float("-inf"))

```

Python

2.2.3 多头注意力

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, width, n_heads):
3         super().__init__()
4         self.head_size = width // n_heads
5         self.W_o = nn.Linear(width, width)
6         self.heads = nn.ModuleList([
7             AttentionHead(width, self.head_size)
8             for _ in range(n_heads)
9         ])
10
11     def forward(self, x, mask=None):
12         # 拼接多个注意力头
13         out = torch.cat([head(x, mask=mask) for head in self.heads], dim=-1)
14         out = self.W_o(out)
15         return out

```

Python

2.2.4 Transformer 编码器

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, width, n_heads, r_mlp=4):
3         super().__init__()
4         self.width = width # 嵌入的维度 d_model
5         self.n_heads = n_heads
6
7         # 层归一化

```

Python

```

8         self.ln1 = nn.LayerNorm(width)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(width, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(width)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(self.width, self.width*r_mlp),
19            nn.GELU(),
20            nn.Linear(self.width*r_mlp, self.width)
21        )
22
23        def forward(self, x, mask=None):
24            x = x + self.mha(self.ln1(x), mask=mask)
25            x = x + self.mlp(self.ln2(x))
26            return x

```

2.2.5 文本的分词器

我们的词汇表是 `ascii` 码，所以 `vocab_size = 256`。

```

1  def tokenizer(text, encode=True, mask=None, max_seq_length=32):
2      if encode:
3          out = chr(2) + text + chr(3) # 添加 SOT token 和 EOT token
4          out = out + "".join([
5              chr(0) for _ in range(max_seq_length-len(out))
6          ]) # 添加Padding
7          out = torch.IntTensor(list(out.encode("utf-8"))) # 对文本进行编码
8          mask = torch.ones(len(out.nonzero()))
9          mask = torch.cat((
10              mask,
11              torch.zeros(max_seq_length - len(mask))
12          )).type(torch.IntTensor)
13      else:
14          # 将input_ids解码为text文本
15          out = [chr(x) for x in text[1:len(mask.nonzero())-1]]
16          out = "".join(out)
17          mask = None
18
19      return out, mask

```

Transformer 无法处理原始文本，因此我们需要做的第一件事是在将输入字符串通过文本编码器之前对其进行分词。

在本教程中，我们将进行一个简单版本的分词，其中我们只使用 **UTF-8** 编码。为什么使用 **UTF-8** 编码进行分词？因为我们只会在示例中使用简单的 `ascii` 码文本。对于更复杂的示例，可能需要使用 **BPE** 分词器。这是因为使用 **UTF-8** 编码时，词表大小 (`vocab_size`) 为 **256**，这意味着对于更复杂的示例，可能会有更长的输入序列，由于上下文长度有限，这在计算注意力时效率低下。

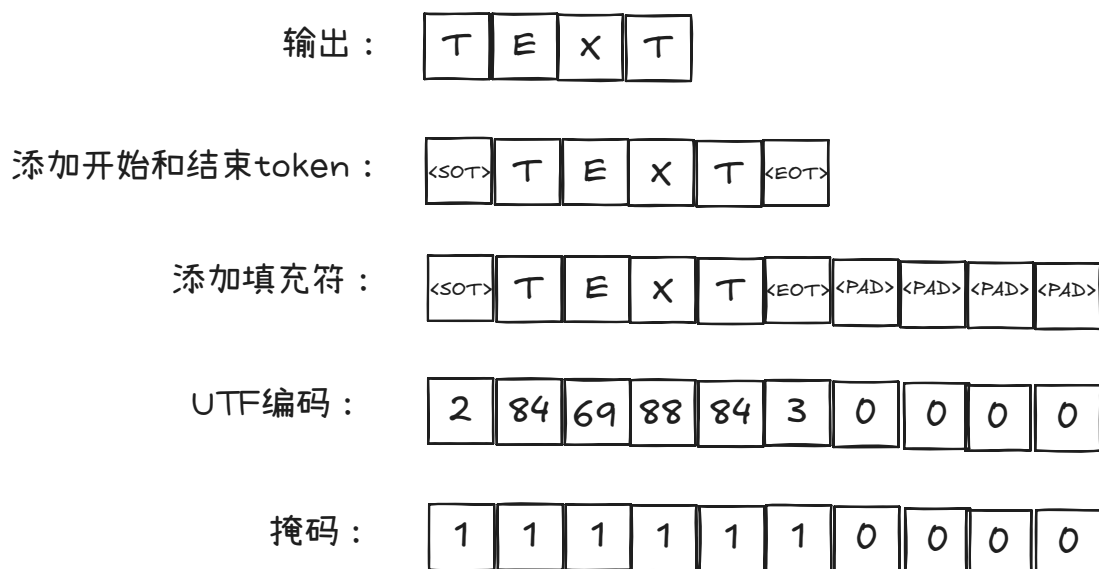


图 2.3 分词过程

分词器的第一步是将文本的开头和文本的结尾 token 添加到输入字符串中。

```
1 text = chr(2) + text + chr(3)
```

[Python](#)

添加文本的开头和文本的结尾 token 后，我们需要将序列的长度填充到最大序列长度。

```
1 text = text + "".join([chr(0) for _ in range(10-len(text))])
```

[Python](#)

我们通过将文本序列编码为 UTF-8 并将输出转换为 IntTensor 来完成分词。

```
1 text = torch.IntTensor(list(text.encode("utf-8")))
```

[Python](#)

对文本进行分词后，我们需要为文本创建掩码。虽然 Transformer 中通常使用的掩码用于确保 token 不与未来的 token 通信，但我们在这里应用的掩码只是使 pad token 被忽略。因此，掩码将只是一个大小等于最大序列长度的张量，其中元素在有填充的情况下为 0，否则为 1。

```
1 mask = torch.ones(len(text.nonzero()))
```

[Python](#)

```
2 mask = torch.cat((mask, torch.zeros(10-len(mask)))).type(torch.IntTensor)
```

2.2.6 文本编码器

```
1 class TextEncoder(nn.Module):
```

[Python](#)

```
2     def __init__(
```

```
3         self,
```

```
4         vocab_size, # 词汇表大小=256
```

```
5         width, # 宽度d_model
```

```
6         max_seq_length, # 文本最大长度
```

```
7         n_heads,
```

```
8         n_layers,
```

```
9         emb_dim # 嵌入维度
```

```
10    ):
```

```
11        super().__init__()
```

```
12        self.max_seq_length = max_seq_length
```

```

13     self.encoder_embedding = nn.Embedding(vocab_size, width)
14     self.positional_embedding = PositionalEmbedding(
15         width,
16         max_seq_length
17     )
18     self.encoder = nn.ModuleList([
19         TransformerEncoder(width, n_heads)
20         for _ in range(n_layers)
21     ])
22     # 可学习投影 (projection)
23     #  $W_{width \times emb\_dim}$ 
24     self.projection = nn.Parameter(torch.randn(width, emb_dim))
25
26     def forward(self, text, mask=None):
27         # 文本嵌入
28         x = self.encoder_embedding(text)
29         # 位置嵌入
30         x = self.positional_embedding(x)
31         # Transformer编码器
32         for encoder_layer in self.encoder:
33             x = encoder_layer(x, mask=mask)
34         # 从EOT的嵌入抽取特征
35         x = x[
36             torch.arange(text.shape[0]), # 批次中数据的索引
37             # 取出掩码mask矩阵的第0行, 加和再减1, 就得到了EOT的索引
38             torch.sub(torch.sum(mask[:, 0], dim=1), 1)
39         ]
40         if self.projection is not None:
41             x = x @ self.projection
42         # 向量x转换为模长为1的向量
43         x = x / torch.norm(x, dim=-1, keepdim=True)
44         return x

```

(1) 将文本嵌入映射到CLIP嵌入空间

对于文本编码器，我们将使用常规 Transformer 模型。创建文本编码器的第一步是创建大小为 (vocab_size, width) 的嵌入表。此嵌入表包含一个向量表示，其大小等于词汇表中每个 token 的 Transformer 模型的 width。

```
1 self.encoder_embedding = nn.Embedding(vocab_size, width)
```

 Python

在输出 Transformer 的结果之前，我们需要将特征嵌入到联合嵌入空间中。我们将通过获取文本特征的点积以及使用 nn.Parameter 创建的可学习的投影来实现这一点。

```
1 self.projection = nn.Parameter(torch.randn(width, emb_dim))
```

 Python

在 forward 方法中，我们要做的第一件事是通过嵌入表传递文本的 token。

```
1 x = self.encoder_embedding(text)
```

 Python

然后，我们需要将位置编码添加到嵌入表的输出中。

```
1 x = self.positional_embedding(x)
```

 Python

添加位置编码后，我们现在可以将其与掩码一起通过编码器层。

```
1 for encoder_layer in self.encoder:
2     x = encoder_layer(x, mask=mask)
```

Python

编码器层的输出是文本的特征。我们将使用从 EOT 的嵌入中抽取的特征。

```
1 # 从EOT的嵌入抽取特征
2 x = x[torch.arange(
3     text.shape[0]),
4     torch.sub(torch.sum(mask[:,0],dim=1),1)
5 ]
```

Python

最后，我们通过计算特征和投影之间的点积，将文本嵌入映射到 CLIP 嵌入空间中，并通过除以归一化的点积对其进行归一化。

🔥 为什么要做映射？

主要是为了在 CLIP 嵌入空间中，文本嵌入向量的维度和图像嵌入向量的维度一致。向量除以向量的模长，就是模长为 1 的向量。这样文本嵌入向量和图像嵌入向量都变成了模长为 1 的向量。两个向量的点积，就是两个向量的余弦相似度！

```
1 if self.projection is not None:
2     x = x @ self.projection
3 x = x / torch.norm(x, dim=-1, keepdim=True)
4 return x
```

Python

2.2.7 图像编码器

```
1 class ImageEncoder(nn.Module):
2     def __init__(
3         self,
4         width, # 补丁嵌入的维度d_model
5         img_size,
6         patch_size,
7         n_channels,
8         n_layers,
9         n_heads,
10        emb_dim
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size必须能被patch_size整除"
17        assert width % n_heads == 0, \
18            "width必须能被n_heads整除"
19
20        self.n_patches = (img_size[0] * img_size[1]) \
21            // (patch_size[0] * patch_size[1])
22        self.max_seq_length = self.n_patches + 1
23        self.linear_project = nn.Conv2d(
24            n_channels,
```

Python

```

25         width,
26         kernel_size=patch_size,
27         stride=patch_size
28     )
29     self.cls_token = nn.Parameter(torch.randn(1, 1, width))
30     self.positional_embedding = PositionalEmbedding(
31         width,
32         self.max_seq_length
33     )
34     self.encoder = nn.ModuleList([
35         TransformerEncoder(width, n_heads)
36         for _ in range(n_layers)
37     ])
38
39     #  $W_{width \times emb\_dim}$ 
40     self.projection = nn.Parameter(torch.randn(width, emb_dim))
41
42     def forward(self, x):
43         # 补丁嵌入
44         x = self.linear_project(x)
45         x = x.flatten(2).transpose(1, 2)
46
47         # 位置嵌入
48         x = torch.cat((self.cls_token.expand(x.size()[0], -1, -1), x), dim=1)
49         x = self.positional_embedding(x)
50
51         # Transformer编码器
52         for encoder_layer in self.encoder:
53             x = encoder_layer(x)
54
55         # 获取类别token
56         x = x[:, 0, :]
57
58         if self.projection is not None:
59             x = x @ self.projection
60
61         # 向量x转换为模长为1的向量
62         x = x / torch.norm(x, dim=-1, keepdim=True)
63         return x

```

(1) 将图像嵌入映射到CLIP嵌入空间

2.3 CLIP 模型

```

1  class CLIP(nn.Module):
2      def __init__(
3          self,
4          emb_dim, # 经过可学习投影后的嵌入维度
5          vit_width, # 图像编码器的宽度(d_model)
6          img_size,
7          patch_size,
8          n_channels,

```

Python

```

9         vit_layers,
10        vit_heads,
11        vocab_size,
12        text_width, # 文本编码器的宽度 (d_model)
13        max_seq_length,
14        text_heads,
15        text_layers
16    ):
17        super().__init__()
18        self.image_encoder = ImageEncoder(
19            vit_width,
20            img_size,
21            patch_size,
22            n_channels,
23            vit_layers,
24            vit_heads,
25            emb_dim
26        )
27        self.text_encoder = TextEncoder(
28            vocab_size,
29            text_width,
30            max_seq_length,
31            text_heads,
32            text_layers,
33            emb_dim
34        )
35        # 可学习温度
36        self.temperature = nn.Parameter(torch.ones([1]) * np.log(1 / 0.07))
37        self.device = torch.device("cuda")
38
39
40    def forward(self, image, text, mask=None):
41        #  $I_e$  是图像嵌入, 形状 [B, D=emb_dim]
42        I_e = self.image_encoder(image)
43        #  $T_e$  是文本嵌入, 形状 [B, D=emb_dim]
44        T_e = self.text_encoder(text, mask=mask)
45
46        # 缩放逐点余弦相似度 [n, n]
47        # 形状  $I_e \cdot T_e : [B, D] @ [D, B] \rightarrow [B, B]$ 
48        logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)
49
50        # 对称损失函数: labels 形状为 [B], 值为 [0, 1, 2, ..., B-1]
51        labels = torch.arange(logits.shape[0]).to(self.device)
52        # 从文本  $\rightarrow$  图像方向, 以文本嵌入  $T_3$  为例子,
53        # 交叉熵损失的目标是让  $T_3 \cdot I_3$  越大越好
54        loss_i = nn.functional.cross_entropy(
55            logits.transpose(-2, -1),
56            labels
57        )

```

```

58     # 从图像 → 文本方向，以图像嵌入  $I_3$  为例子，
59     # 交叉熵损失的目标是让  $I_3 \cdot T_3$  越大越好
60     loss_t = nn.functional.cross_entropy(
61         logits,
62         labels
63     )
64     # 两个方向的损失求平均值
65     loss = (loss_i + loss_t) / 2
66
67     return loss

```

当给定一批图像和标题时，CLIP 应该告诉我们哪些标题与哪些图像搭配。它通过一起训练文本编码器和图像编码器来最大化应该在一起的对的成对余弦相似度分数，并最小化不应该在一起的对来做到这一点。

为此，我们首先需要从图像和文本编码器中获取特征的嵌入。

```

1 def forward(self, image, text, mask=None):
2     I_e = self.image_encoder(image)
3     T_e = self.text_encoder(text, mask=mask)

```

Python

使用嵌入的特征，我们可以通过使用嵌入的图像特征和嵌入文本特征的转置版本之间的点积来计算缩放的成对余弦相似度。余弦相似度应沿图中正确的图像和文本配对在一起的对角线最大化。

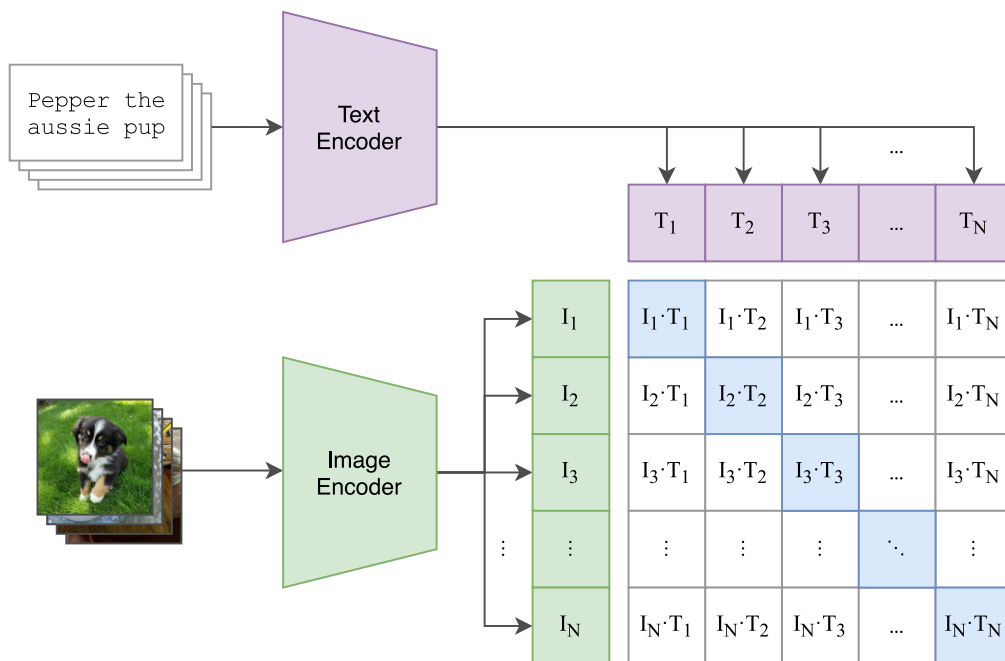


图 2.4 余弦相似度的计算

```

1 logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)

```

Python

这在 I_e 和 T_e 分别包含 N 个批次时工作，从而产生上图所示的矩阵。为了最大化相关图像之间的余弦相似度，CLIP 使用对称/对比损失。我们可以通过首先创建与批次中的每条数据相对应的标签来计算此损失。

```

1 # 对称损失函数
2 labels = torch.arange(logits.shape[0]).to(self.device)

```

Python

然后，我们计算沿 **logit** 行的交叉熵损失，以获得图像的损失。

```
1 loss_i = nn.functional.cross_entropy(logits.transpose(-2,-1), labels)
```

 Python

文本的损失是通过计算沿列的交叉熵损失来计算的。

```
1 loss_t = nn.functional.cross_entropy(logits, labels)
```

 Python

我们通过计算图像损失和文本损失之间的平均值来获得最终损失。

```
1 loss = (loss_i + loss_t) / 2
2 return loss
```

 Python

2.3.1 数据

```
1 class MNIST(Dataset):
2     def __init__(self, train=True):
3         self.dataset = load_dataset("../datasets/clip-mnist/")
4         self.transform = T.ToTensor()
5         if train:
6             self.split = "train"
7         else:
8             self.split = "test"
9
10        self.captions = {
11            0: "An image of Zero",
12            1: "An image of One",
13            2: "An image of Two",
14            3: "An image of Three",
15            4: "An image of Four",
16            5: "An image of Five",
17            6: "An image of Six",
18            7: "An image of Seven",
19            8: "An image of Eight",
20            9: "An image of Nine"
21        }
22
23    def __len__(self):
24        return self.dataset.num_rows[self.split]
25
26    def __getitem__(self, i):
27        # 取出第i张图片
28        img = self.dataset[self.split][i]["image"]
29        # 转换成张量
30        img = self.transform(img)
31        # 图片对应的文本，以及掩码
32        cap, mask = tokenizer(
33            self.captions[self.dataset[self.split][i]["label"]]
34        )
35        # 为什么要repeat?
36        mask = mask.repeat(len(mask), 1)
37        return {"image": img, "caption": cap, "mask": mask}
```

 Python

在本教程中，我们将使用 MNIST 数据集。我们选择这个数据集是因为它相当小并且保持训练时间合理。

```
1 self.dataset = load_dataset("clip-mnist")
```

[Python](#)

对于数据集中的每个条目，我们将需要 3 样东西：图像、文本和文本掩码。

对于图像，我们唯一需要进行的更改是将图像转换为张量。

```
1 img = self.dataset[self.split][i]["image"]
2 img = self.transform(img)
```

[Python](#)

对于标题，我们需要将其传递给我们创建的分词器，以获取 token 表示以及 token 的掩码。

```
1 cap, mask = tokenizer(self.captions[self.dataset[self.split][i]["label"]])
```

[Python](#)

我们从分词器获得的掩码是大小为 `max_seq_length` 的 1 维张量。在文本编码器中，掩码将应用于形状为 `(max_seq_length, max_seq_length)` 的注意力分数。因此，我们需要扩展掩码，以便将其应用于注意力分数的每一行。

原始掩码：

1	1	1	0	0
---	---	---	---	---

新掩码：

1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0

图 2.5 复制掩码

```
1 mask = mask.repeat(len(mask), 1)
```

[Python](#)

图像、标题和掩码作为字典保存在数据集中。

```
1 return {"image": img, "caption": cap, "mask": mask}
```

[Python](#)

✗ 不要和因果注意力混淆!

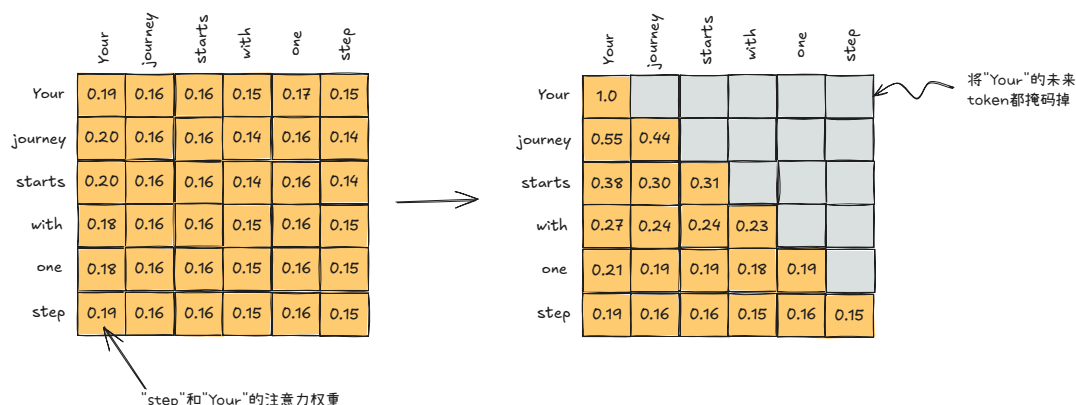


图 2.6 因果注意力分数的掩码

创建 [context_length, context_length] 的上三角矩阵。

```
1 self.register_buffer(
2     'mask',
3     torch.triu(torch.ones(
4         context_length,
5         context_length
6     ), diagonal = 1))
```

将注意力分数的上三角置为 `-torch.inf`，下三角保留。得到注意力分数的下三角矩阵。

```
1 attention_score.masked_fill_(
2     self.mask[:seq_len, :seq_len].bool(),
3     -torch.inf
4 )
```

2.3.2 训练参数

```
1 emb_dim = 32 # 文本编码器和图像编码器输出的张量的维度
2 vit_width = 9 # 图像编码器的嵌入的宽度
3 img_size = (28,28)
4 patch_size = (14,14)
5 n_channels = 1
6 vit_layers = 3 # 图像编码器中编码器的层的数量
7 vit_heads = 3 # 图像编码器中注意力头的数量
8 vocab_size = 256 # 词汇表大小
9 text_width = 32 # 文本编码器的嵌入的宽度
10 max_seq_length = 32 # 最大序列长度
11 text_heads = 8 # 文本编码器中注意力头的数量
12 text_layers = 4 # 文本编码器中编码器的层数
13 lr = 1e-3 # 学习率
14 epochs = 10
15 batch_size = 128
```

2.3.3 加载数据集

```
1 train_set = MNIST(train = True)
2 test_set = MNIST(train = False)
3
4 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
5 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

 Python

2.3.4 训练模型

```
1 device = torch.device("cuda")
2
3 model = CLIP(
4     emb_dim,
5     vit_width,
6     img_size,
7     patch_size,
8     n_channels,
9     vit_layers,
10    vit_heads,
11    vocab_size,
12    text_width,
13    max_seq_length,
14    text_heads,
15    text_layers
16 ).to(device)
17
18 optimizer = optim.Adam(model.parameters(), lr=lr)
19
20 best_loss = np.inf
21 for epoch in range(epochs):
22     for i, data in enumerate(train_loader, 0):
23         img = data["image"].to(device)
24         cap = data["caption"].to(device)
25         mask = data["mask"].to(device)
26         loss = model(img, cap, mask)
27         optimizer.zero_grad()
28         loss.backward()
29         optimizer.step()
30
31     print(f"Epoch [{epoch+1}/{epochs}], Batch Loss: {loss.item():.3f}")
32
33     # 保存模型
34     if loss.item() <= best_loss:
35         best_loss = loss.item()
36         torch.save(model.state_dict(), "./clip.pt")
37     print("模型已经保存.")
```

 Python

2.3.5 评估模型

```

1  # 加载最好的模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 获取数据集的标签和图片进行对比
19 text = torch.stack(
20     [tokenizer(x)[0] for x in test_set.captions.values()]
21 ).to(device)
22 mask = torch.stack(
23     [tokenizer(x)[1] for x in test_set.captions.values()]
24 )
25 mask = mask.repeat(
26     1,
27     len(mask[0])
28 ).reshape(
29     len(mask),
30     len(mask[0])
31 ), len(mask[0])).to(device)
32
33 correct, total = 0, 0
34 with torch.no_grad():
35     for data in test_loader:
36         # 图像
37         images = data["image"].to(device)
38         # 文本
39         labels = data["caption"].to(device)
40         # 使用clip模型中的图像编码器对图像抽取特征
41         image_features = model.image_encoder(images)
42         # 使用clip模型中的文本编码器对文本抽取特征
43         text_features = model.text_encoder(text, mask=mask)
44         # 转换为模长为1的向量
45         image_features /= image_features.norm(dim=-1, keepdim=True)
46         text_features /= text_features.norm(dim=-1, keepdim=True)
47         #  $I_e @ T_e^T: I_e \cdot T_e$ 
48         similarity = (

```

```

49         100.0 * image_features @ text_features.T
50     ).softmax(dim=-1)
51     _, indices = torch.max(similarity, 1)
52     # 预测结果
53     pred = torch.stack([
54         tokenizer(test_set.captions[int(i))][0]
55         for i in indices
56     ]).to(device)
57     # 预测正确的样本数量
58     correct += int(sum(torch.sum((pred==labels),dim=1)//len(pred[0])))
59     total += len(labels)
60
61 print(f'\n预测准确率: {100 * correct // total} %')

```

我们通过获取模型训练的标题并将其与实际标题进行比较来测试模型。在训练时，我们使用了相同的标题模板（"A image of a(n) {class}"），因此这个测试阶段与任何其他图像分类器几乎相同。我们实现了大约 85% 的模型准确率。

2.3.6 零样本分类

```

1  # 加载模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 标题
19 class_names = [
20     "a photo of 0",
21     "an image of one",
22     "2",
23     "3",
24     "4",
25     "5",
26     "6",
27     "7",
28     "8",
29     "9",
30     "Trump",

```

 Python

```

31     "Musk"
32 ]
33
34 text = torch.stack(
35     [tokenizer(x)[0] for x in class_names]
36 ).to(device)
37 mask = torch.stack(
38     [tokenizer(x)[1] for x in class_names]
39 )
40 mask = mask.repeat(
41     1,
42     len(mask[0])
43 ).reshape(len(mask), len(mask[0]), len(mask[0])).to(device)
44
45 idx = 1000
46 # 去测试数据集中的第1000张图片
47 img = test_set[idx]["image"][None,:]
48 plt.imshow(img[0].permute(1, 2, 0), cmap="gray")
49 # 将图片的标题文本展示, 例如"An Image Of Nine"
50 plt.title(tokenizer(
51     test_set[idx]["caption"],
52     encode=False,
53     mask=test_set[idx]["mask"][0]
54 )[0])
55 plt.show()
56 img = img.to(device)
57 with torch.no_grad():
58     # 抽取图片的特征
59     image_features = model.image_encoder(img)
60     # 抽取文本的特征
61     text_features = model.text_encoder(text, mask=mask)
62
63 image_features /= image_features.norm(dim=-1, keepdim=True)
64 text_features /= text_features.norm(dim=-1, keepdim=True)
65 # 计算第1000张图片和所有文本的相似度
66 similarity = (
67     100.0 * image_features @ text_features.T
68 ).softmax(dim=-1)
69 # 返回所有文本中和图片特征最相似的5个文本
70 values, indices = similarity[0].topk(5)
71
72 # 打印结果
73 print("\n预测结果:\n")
74 for value, index in zip(values, indices):
75     print(f"{class_names[int(index)]:>16s}: {100 * value.item():.2f}%")

```

对于 **zero-shot** 分类, 我们将图像与类别的名称进行比较。我们输入标签以与图像进行比较, 它将返回前 5 个预测以及预测的可能性。这不是 CLIP 执行 **zero-shot** 分类的最佳示例。使用 MNIST 数据集使模型易于训练, 但标题不是很丰富。要真正理解 CLIP 的 **zero-shot** 功能, 包含多个名称的训练集会更适合。真正的 **zero-shot** 检测将允许检测以前未见过的排列。

🔥 应用举例：文搜图

具体步骤如下：

1. 将数据库中的所有图片使用 `clip` 的图像编码器进行抽取特征
2. 将抽取的图片特征存入向量数据库
3. 将输入的搜索文本通过 `clip` 的文本编码器抽取文本特征
4. 使用余弦相似度找出向量数据库中和文本特征最接近的几张图片

3. ClipCap (图生文)

🔥 Image Captioning Model

图片标题模型(image captioning model)是一种将图片作为输入并生成图片描述模型。

下面是一个简单的示例，展示了一个图片标题模型：

Image Captioning Model

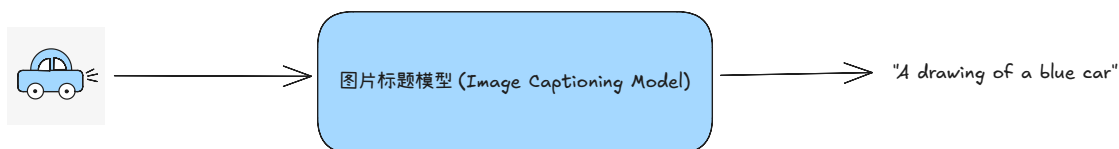


图 3.1 图片标题模型的简单示例

3.1 ClipCap 的工作原理

ClipCap 是一种结合了 CLIP 和 GPT-2 的图片标题架构。

CLIP 是我们将来用来创建输入图像嵌入的模型。

GPT-2 是一种基于解码器的模型，用于生成文本。

ClipCap 的基本工作原理如下：

输入图像首先通过 CLIP 模型转换为嵌入，目的是利用这种嵌入(捕捉图像意义)来引导 GPT-2 生成文本。

但有一个问题：CLIP 和 GPT-2 的嵌入空间不同。所以我们不能直接把这个嵌入输入到 GPT-2 中。

为了解决这个问题，我们使用一个映射网络将 CLIP 嵌入映射到 GPT-2 的嵌入空间。

这些映射的图像嵌入称为前缀(prefixes)，因为它们是 GPT-2 生成图片说明所需的上下文。

我们将使用这些嵌入作为“前缀”(prefix)来使得 GPT2 生成标题

但问题是 CLIP 和 GPT2 有着不同的嵌入空间，所以必须将 CLIP 嵌入映射到 GPT2 的嵌入空间，才能使用这些嵌入

将图片的嵌入从 CLIP 的嵌入空间映射到 GPT2 的嵌入空间

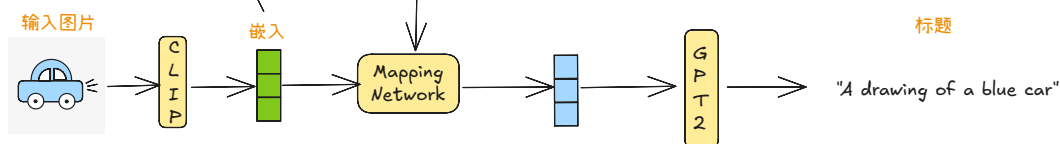


图 3.2 将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间

3.2 关于训练

CLIP 生成的图像嵌入开箱即用已经足够好——所以我们不训练 CLIP 模型。

根据 GPT-2 是否经过微调，ClipCap 有两种变体：

- 如果我们对 GPT-2 进行微调，那么我们就用 MLP 作为映射网络。GPT-2 和 MLP 都经过训练。
- 如果我们不对 GPT-2 进行微调，那么我们就用 Transformer 架构（例如 Bert）作为映射网络，只有 Transformer 本身被训练。

就我而言，我选择对 GPT-2 模型进行微调，并使用 MLP 作为映射网络。

3.3 推理

在我们这里，这意味着为没见过的图片生成说明文字。

让我们用一辆蓝色汽车的图片作为推理过程的例子：

1. 输入图像被转换为 CLIP 图像嵌入。
2. CLIP 图像嵌入通过映射网络传递，生成前缀嵌入。
3. 在时间步 $t=1$ 时，我们将这些前缀嵌入传递到 GPT2。模型预测下一个 token——假设是“a”。我们将此附加到序列中。
4. 在 $t=2$ 时，我们将更新后的输入序列（前缀+“a”）传递给 GPT2。它预测下一个 token——这次可能是“蓝色”。
5. 这一过程持续进行，模型一次预测一个 token，直到它：
 - 生成 <EOS>（序列结束）token，或
 - 生成的标题长度达到最大。

如下图所示

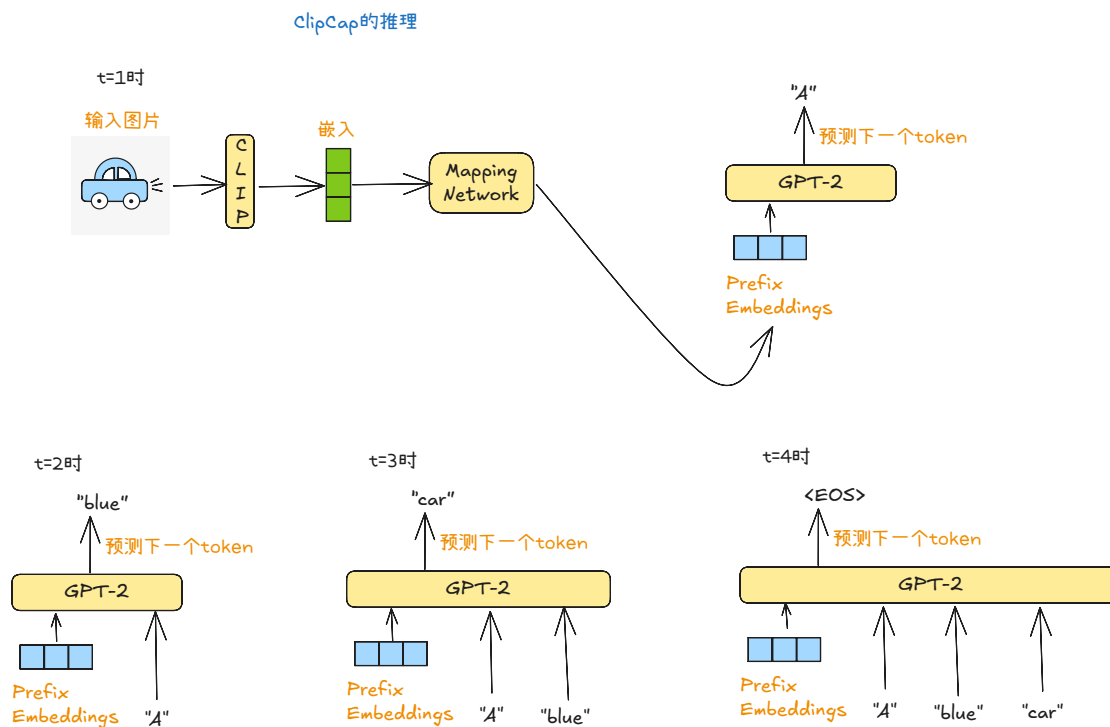


图 3.3 ClipCap 的推理过程

3.4 ClipCap 的代码实现

3.4.1 项目配置

```

config.py
Python
1 import torch
2
3 CLIP_MODEL_PATH = "./chinese-clip-vit-base-patch16"
4 # 一张图片的嵌入经过投影转换成10个tokens的embedding, 每个embedding的dim是768
5 IMAGE_TOKEN_LENGTH = 10 # 图片的tokens的数量
6 MAX_LENGTH = 100 # 最大token数量
7 # clip对接的大语言模型
8 LLM_PATH = "./gpt2-chinese-cluecorpussmall"
9 LLM_WORD_EMBD_DIM = 768 # gpt2的词嵌入维度
10 IMAGE_EMBD_DIM = 512 # clip输出的图像嵌入的维度
11 device = torch.device("cuda")

```

3.4.2 处理数据

```

process_data.py
Python
1 from PIL import Image
2 import pickle
3 from transformers import ChineseCLIPProcessor, ChineseCLIPModel
4 from config import CLIP_MODEL_PATH
5
6

```

```
7 def main():
8     # 加载clip模型
9     # clip模型只用来生成图片的嵌入，不进行微调。
10    clip_model = ChineseCLIPModel.from_pretrained(CLIP_MODEL_PATH)
11    # 加载clip处理器
12    processor = ChineseCLIPProcessor.from_pretrained(CLIP_MODEL_PATH)
13    # 将2张图片进行处理，处理完之后交给clip抽取特征
14    inputs_1 = processor(images=Image.open("1.jpg"), return_tensors="pt")
15    inputs_2 = processor(images=Image.open("2.jpg"), return_tensors="pt")
16    # 获取第一张图片的嵌入 (dim: 512)
17    image_1_features = clip_model.get_image_features(**inputs_1)
18    image_2_features = clip_model.get_image_features(**inputs_2)
19    # 除以模长，归一化
20    image_1_features = image_1_features / \
21        image_1_features.norm(p=2, dim=-1, keepdim=True) # normalize
22    image_2_features = image_2_features / \
23        image_2_features.norm(p=2, dim=-1, keepdim=True) # normalize
24    # key: 图片id
25    # value: 图片的嵌入
26    # 下面的字典也可以放在chroma这样的向量数据库
27    image_id2embed = {
28        1: image_1_features,
29        2: image_2_features,
30    }
31    # 图片的id和图片标题对
32    caption_list = [
33        (1, "两只狗在雪地里嬉闹"),
34        (2, "一件好看的立领的很特别的紫色风衣"),
35    ]
36
37    with open("caption_image.pkl", 'wb') as f:
38        pickle.dump([caption_list, image_id2embed], f)
39
40    print(f'图像嵌入的数量:{len(image_id2embed)}')
41    print(f'图像文本的数量:{len(caption_list)}')
42
43
44 if __name__ == '__main__':
45     main()
```

3.4.3 模型结构

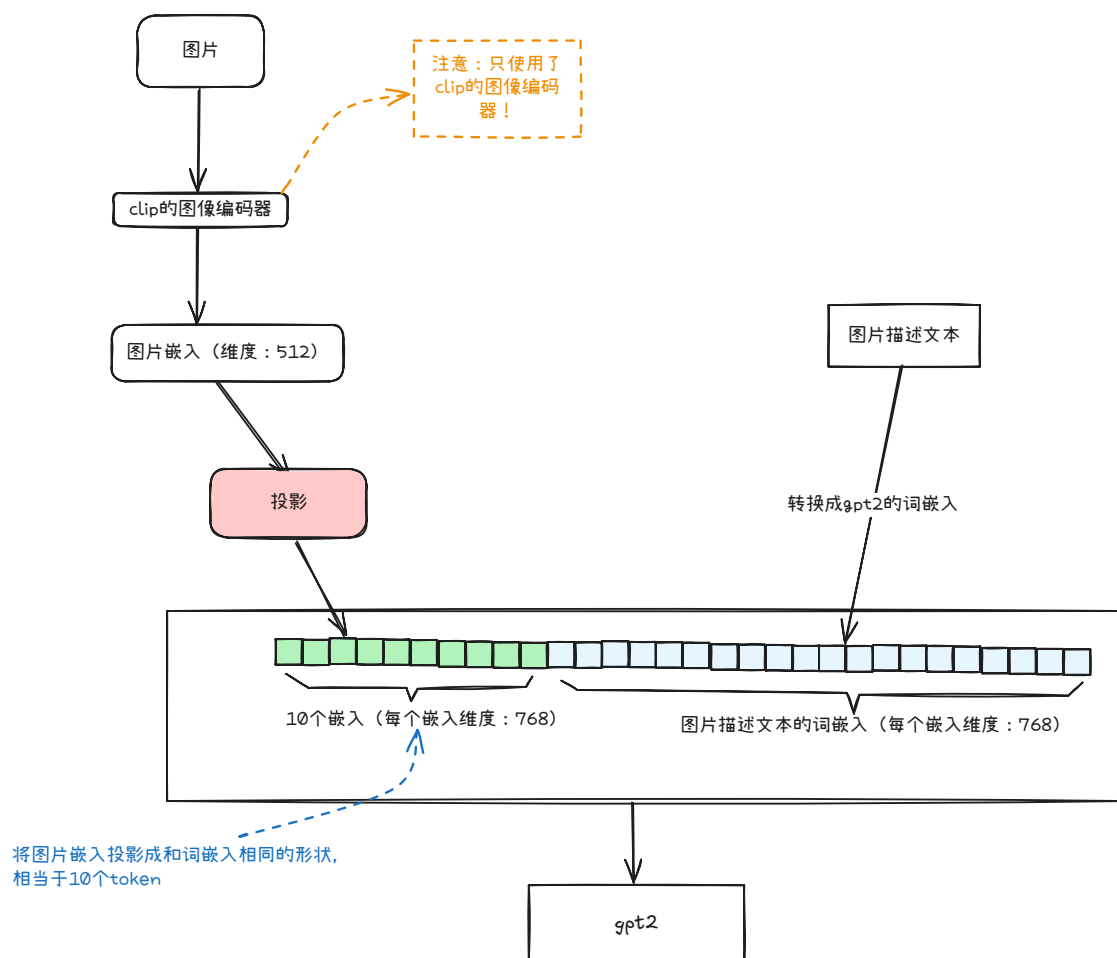


图 3.4 clipcap 的模型设计要点

```

model.py
Python
1 import torch
2 import torch.nn as nn
3 from transformers import GPT2LMHeadModel
4
5 from typing import Sequence
6 from config import LLM_PATH, IMAGE_TOKEN_LENGTH, IMAGE_EMBD_DIM
7
8
9 class MLP(nn.Module):
10     """投影层"""
11     def __init__(self, sizes: Sequence[int]):
12         super().__init__()
13
14         in_dim, h1, out_dim = sizes
15         self.l1 = nn.Linear(in_dim, h1)
16         self.act1 = nn.Tanh()
17         self.l2 = nn.Linear(h1, out_dim)

```

```

18
19     def forward(self, x: torch.Tensor) → torch.Tensor:
20         x = x.float()
21         x = self.l1(x)
22         x = self.act1(x)
23         x = self.l2(x)
24         return x
25
26
27 class ClipCaptionModel(nn.Module):
28     def __init__(self):
29         super(ClipCaptionModel, self).__init__()
30         # 大语言模型: 用来生成图片的文本描述
31         self.gpt2 = GPT2LMHeadModel.from_pretrained(LLM_PATH)
32         # gpt2的词嵌入维度是768
33         self.word_embd_dim = self.gpt2.config.n_embd
34         # 投影层定义
35         self.projection = MLP((
36             # 输入维度是512,也就是clip的图像编码器输出的嵌入的维度
37             IMAGE_EMBD_DIM,
38             # (768 * 10) // 2
39             # 10是图片嵌入转换成的token数量
40             (self.word_embd_dim * IMAGE_TOKEN_LENGTH) // 2,
41             # 768 x 10, 图片占10个token, 每个token的嵌入和词嵌入相同是768维度
42             self.word_embd_dim * IMAGE_TOKEN_LENGTH
43         ))
44
45     def forward(self, image_embeds, caption_ids, mask):
46         # 张量形状: [B, 文本长度, gpt2的词嵌入的维度]
47         # 标题caption的每个token的词嵌入
48         caption_embeds = self.gpt2.transformer.wte(caption_ids)
49         # 将图片的嵌入转换为像词嵌入那样的维度:
50         # [B, 图片token的长度为10, gpt2的词嵌入的维度]
51         image_as_word_embeds = self.projection(
52             image_embeds
53         ).view(-1, IMAGE_TOKEN_LENGTH, self.word_embd_dim)
54         # 10个图片的token + 文本的token
55         # 张量形状: [B, 10+文本长度, 词嵌入维度]
56         embedding_cat = torch.cat((
57             image_as_word_embeds, # 图像的token
58             caption_embeds # 文本的token
59         ), dim=1)
60         out = self.gpt2(inputs_embeds=embedding_cat, attention_mask=mask)
61         # 张量形状: [B, 10+文本长度, 词嵌入维度]
62         logits = out.logits
63         return logits

```

3.4.4 准备训练数据集

```

clipcap_dataset.py Python
1  import torch
2  from torch.utils.data import Dataset
3  import pickle
4  from typing import Tuple
5  from config import IMAGE_TOKEN_LENGTH, MAX_LENGTH
6
7
8  class ClipCapDataset(Dataset):
9      def __init__(self, tokenizer):
10         # 填充符
11         pad_id = tokenizer.pad_token_id
12         # 取出图片的文本和图片的嵌入
13         with open("caption_image.pkl", 'rb') as f:
14             caption_list, image_id2embed = pickle.load(f)
15             print('图片嵌入的总数:{}'.format(len(image_id2embed)))
16             print('图片描述的总数:{}'.format(len(caption_list)))
17
18             image_embed_list = []
19             caption_ids_list = []
20             mask_list = []
21             for image_id, caption in caption_list:
22                 # 使用图像id获取图像的特征 (clip.image_encoder输出的)
23                 image_embed = image_id2embed[image_id]
24                 # 只对文本进行分词, 不添加任何特殊token
25                 caption_ids = tokenizer.encode(
26                     caption,
27                     add_special_tokens=False
28                 )
29                 # 在文本的token列表后面添加一个分隔符token
30                 caption_ids.append(tokenizer.sep_token_id)
31
32                 # 截断
33                 # 只能留下前90个token, 因为图像对应的token需要占用10个token的位置
34                 # 最终的数据是: 图像的token列表 + 文本的token列表
35                 caption_ids = caption_ids[:MAX_LENGTH - IMAGE_TOKEN_LENGTH]
36                 # 图像部分和文本部分的token都要掩码
37                 mask = [1] * (IMAGE_TOKEN_LENGTH + len(caption_ids))
38
39                 # 填充pad
40                 padding_len = MAX_LENGTH \
41                     - IMAGE_TOKEN_LENGTH \
42                     - len(caption_ids)
43                 caption_ids += [pad_id] * padding_len
44                 # 将填充符掩码为0
45                 mask += [0] * padding_len
46
47                 caption_ids = torch.tensor(caption_ids).long()

```

```

48     mask = torch.tensor(mask).long()
49
50     image_embed_list.append(image_embed)
51     caption_ids_list.append(caption_ids)
52     mask_list.append(mask)
53     # 保存训练数据
54     with open("train_data.pkl", 'wb') as f:
55         pickle.dump([
56             image_embed_list, # clip输出的图片特征的列表
57             caption_ids_list, # 图片文本的input_ids的列表
58             mask_list # 掩码的列表
59         ], f)
60     self.image_embed_list = image_embed_list
61     self.caption_ids_list = caption_ids_list
62     self.mask_list = mask_list
63     print(f'训练数据总数: {len(self.image_embed_list)}')
64
65     def __len__(self) -> int:
66         return len(self.caption_ids_list)
67
68     def __getitem__(self, index: int) -> Tuple[torch.Tensor, ...]:
69         image_embed = self.image_embed_list[index]
70         caption_ids = self.caption_ids_list[index]
71         mask = self.mask_list[index]
72         return image_embed, caption_ids, mask

```

3.4.5 训练

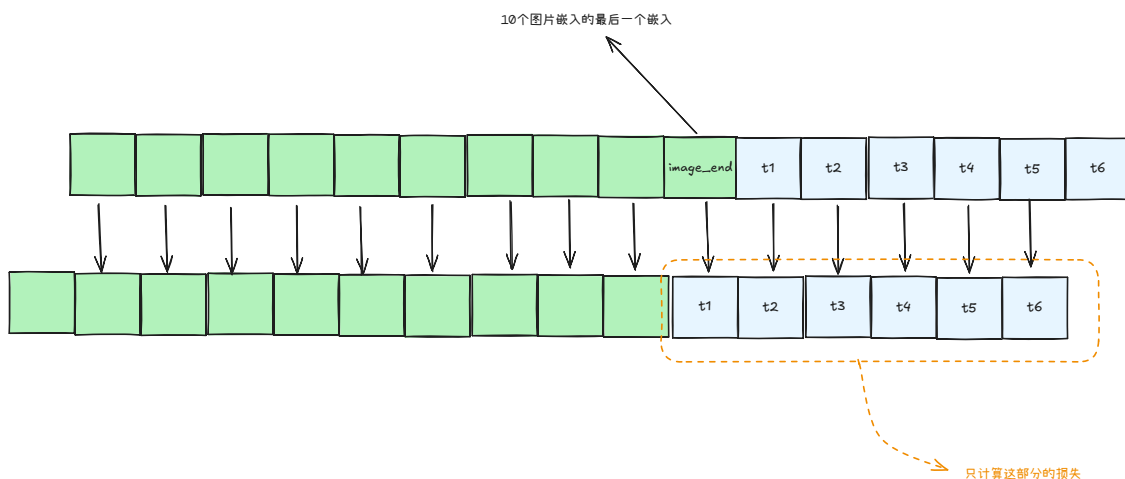


图 3.5 clipcap 的训练目标

train.py

Python

```

1 import torch
2 from torch.utils.data import DataLoader
3 from transformers import BertTokenizer
4 from tqdm import tqdm

```



```

5  from clipcap_dataset import ClipCapDataset
6  from model import ClipCaptionModel
7  import torch.nn.functional as F
8  from config import LLM_PATH, IMAGE_TOKEN_LENGTH, device
9
10
11 def train(model, train_loader, optimizer):
12     model.train()
13     for _ in range(20):
14         for _, data in enumerate(tqdm(train_loader)):
15             image_embed, caption_ids, mask = data
16             image_embed = image_embed.to(device)
17             caption_ids = caption_ids.to(device)
18             mask = mask.to(device)
19             # 输出的logits
20             logits = model(image_embed, caption_ids, mask)
21
22             # 计算loss
23             # [图片的最后一个token], [两], [只], [狗]
24             #           ↓           ↓   ↓
25             #           [两]       [只] [狗]
26             shift_logits = logits[
27                 :,
28                 # 截取范围[图片的最后一个token~倒数第二个token]
29                 IMAGE_TOKEN_LENGTH - 1: -1, # 去掉最后一个token
30                 :
31             ].contiguous().view(-1, logits.size(-1))
32             # 预测目标
33             shift_labels = caption_ids.view(-1)
34             loss = F.cross_entropy(shift_logits, shift_labels)
35             # logits.size(-1): 取 logits 的最后一维大小。一般最后一维是词表大小 vocab_size。
36             # 原 logits 形状通常是 [B, L, V] (批大小、序列长度、词表大小)。
37             # 经过切片后, shift_logits 的形状是 [B, L-1, V]。
38             # 再 `.contiguous().view(-1, logits.size(-1))`
39             # 就变成 [B*(L-1), V], 把前两维展平, 便于和标签做交叉熵。
40             # caption_ids 形状通常是 [B, L-1] (与 shift_logits 的时间步对齐)。
41             # caption_ids.view(-1) 把它展平成 [B*(L-1)], 与 shift_logits 的第一维对齐,
42             # 用于计算 CrossEntropyLoss(shift_logits, shift_labels)。
43
44             loss.backward()
45             optimizer.step()
46             optimizer.zero_grad()
47
48     torch.save(model.state_dict(), f'model.pt')
49
50
51 def main():
52     # 分词器

```

```

53 tokenizer = BertTokenizer.from_pretrained(LLM_PATH)
54 # 加载模型
55 model = ClipCaptionModel().to(device)
56
57 dataset = ClipCapDataset(tokenizer)
58 train_dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
59 optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
60
61 train(model, train_dataloader, optimizer)
62
63
64 if __name__ == '__main__':
65     main()

```

3.4.6 推理

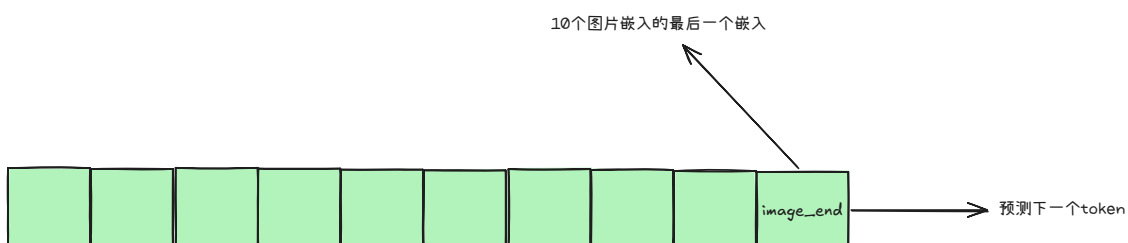


图 3.6 clipcap 只根据图片的嵌入预测文本的 token，也就是看图说话，不再需要文本了！

```

infer.py
Python
1 from PIL import Image
2 import torch
3 from transformers import BertTokenizer, ChineseCLIPModel, ChineseCLIPProcessor
4 from model import ClipCaptionModel
5 import torch.nn.functional as F
6 from config import LLM_PATH, CLIP_MODEL_PATH, IMAGE_TOKEN_LENGTH,
  LLM_WORD_EMBD_DIM, device, MAX_LENGTH
7
8
9 def generate(model, image_embeds, tokenizer):
10     """
11     :param image_embeds: [B, IMAGE_EMBD_DIM=512]
12     """
13
14     b_size = image_embeds.size(0)
15     pad_id = tokenizer.pad_token_id
16     sep_id = tokenizer.sep_token_id
17     unk_id = tokenizer.unk_token_id
18     temperature = 0.7
19
20     cur_len = 0
21     caption_ids = [] # 存储生成的caption
22

```

```

23     # gpt2模型的输入: inputs_embeds:[B, 图片token的数量为10, gpt2的词嵌入维度768]
24     # 先将图片特征投影为10个图片token
25     inputs_embeds = model.projection(
26         image_embeds
27     ).view(-1, IMAGE_TOKEN_LENGTH, LLM_WORD_EMBD_DIM)
28     finish_flag = [False] * b_size # 第i个输入是否完成生成的标志
29
30     while True:
31         out = model.gpt2(inputs_embeds=inputs_embeds)
32         logits = out.logits # [B, len, vocab_size]
33         # 采样下一个token
34         next_token_logits = logits[:, -1, :] # 取最后一个单词的预测分布
35         next_token_logits = next_token_logits / temperature
36         next_token_logits[:, unk_id] = -float('Inf') # 将unk设为无穷小
37
38         # 采样下一个token, 多项分布
39         next_token_ids = torch.multinomial(
40             F.softmax(next_token_logits, dim=-1),
41             num_samples=1
42         ).squeeze(1).tolist()
43
44         # 分别判断生成图片是否已生成完毕
45         # index表示第index张正在生成文本的图片
46         for index in range(len(next_token_ids)):
47             token_id = next_token_ids[index]
48             # 如果第i个句子已经生成结束
49             if finish_flag[index]:
50                 next_token_ids[index] = pad_id
51             # 如果第i个句子生成结束, 预测到了分隔符
52             elif token_id == sep_id:
53                 finish_flag[index] = True
54             # 生成刚开始
55             elif cur_len == 0:
56                 caption_ids.append([token_id])
57             else:
58                 caption_ids[index].append(token_id)
59             next_token_ids = torch.tensor(next_token_ids).to(device)
60             next_token_embeds = model.gpt2.transformer.wte(
61                 next_token_ids).to(device).unsqueeze(1)
62             # 将生成的next token拼接至上文的后面, 继续生成
63             inputs_embeds = torch.cat((inputs_embeds, next_token_embeds), dim=1)
64
65             cur_len += 1 # 生成长度+1
66             # 如果生成长度大于最大长度, 或者所有图片的生成文本都结束了, 退出生成过程。
67             if cur_len > MAX_LENGTH or False not in finish_flag:
68                 break
69
70     # 对token_id进行解码
71     captions = []

```

```

72     for caption_id in caption_ids:
73         caption = tokenizer.convert_ids_to_tokens(caption_id)
74         caption = ''.join(caption)
75         captions.append(caption)
76
77     return captions
78
79
80 def main():
81     # 分词器
82     tokenizer = BertTokenizer.from_pretrained(LLM_PATH)
83     # 初始化模型
84     model = ClipCaptionModel().to(device)
85     # 加载权重
86     model.load_state_dict(torch.load(
87         "model.pt",
88         map_location=device
89     ), False)
90     model.eval()
91
92     # 加载clip模型
93     clip_model = ChineseCLIPModel.from_pretrained(CLIP_MODEL_PATH)
94     processor = ChineseCLIPProcessor.from_pretrained(CLIP_MODEL_PATH)
95     inputs_1 = processor(images=Image.open("1.jpg"), return_tensors="pt")
96     inputs_2 = processor(images=Image.open("2.jpg"), return_tensors="pt")
97     image_1_features = clip_model.get_image_features(**inputs_1)
98     image_2_features = clip_model.get_image_features(**inputs_2)
99     image_1_features = image_1_features / \
100         image_1_features.norm(p=2, dim=-1, keepdim=True) # normalize
101     image_2_features = image_2_features / \
102         image_2_features.norm(p=2, dim=-1, keepdim=True) # normalize
103     # 将两张图片的特征打包成一个批次数据
104     data = torch.stack([
105         image_1_features,
106         image_2_features
107     ], dim=0).to(device)
108     captions = generate(model, data, tokenizer)
109     print(captions)
110     captions = generate(model, data, tokenizer)
111     print(captions)
112     captions = generate(model, data, tokenizer)
113     print(captions)
114     captions = generate(model, data, tokenizer)
115     print(captions)
116     captions = generate(model, data, tokenizer)
117     print(captions)
118
119
120 if __name__ == '__main__':
121     main()

```

3.5 应用：图生文

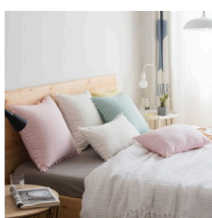
当前电商平台上的商品描述主要依赖商家手工编写，质量参差不齐，存在描述不准确、信息不完整、语言不标准等问题。这不仅影响用户的购买决策，还增加了平台内容审核与优化的成本。如何利用智能化手段快速生成高质量的商品描述，成为亟待解决的核心问题。



时尚阔腿裤套装，优雅
干练显气质



：黑色的短袖，带有一
点点复古的气息，上身
穿着很有学院风的感
觉，领口的撞色设计，
增加了整体的层次感，
搭配一条牛仔裤和一
双帆布鞋，简约清新。
：白色的短袖，带有一
点点复古的气息，上身
穿着很有学院风的味
道



抱枕，给你一个温暖的
拥抱



哑铃弯举，让你做个健
身房里的杠铃男神



简约大方的圆领卫衣，
宽松的版型，穿着舒
适自在。胸前的撞色
印花点缀，丰富整体
的视觉感。搭配高腰
的牛仔裤和帆布鞋，
穿出学院风。



花园摆件，给家增添一
份生机



衬衫穿搭，时髦精的专
属



夏季雪纺衫，清新又甜
美



条纹男装，穿出你的时
尚范



恤搭配牛仔裤，轻松穿
出青春活力范

图 3.7 A800 训练 10 个小时的效果

3.6 拓展: Qwen3-VL

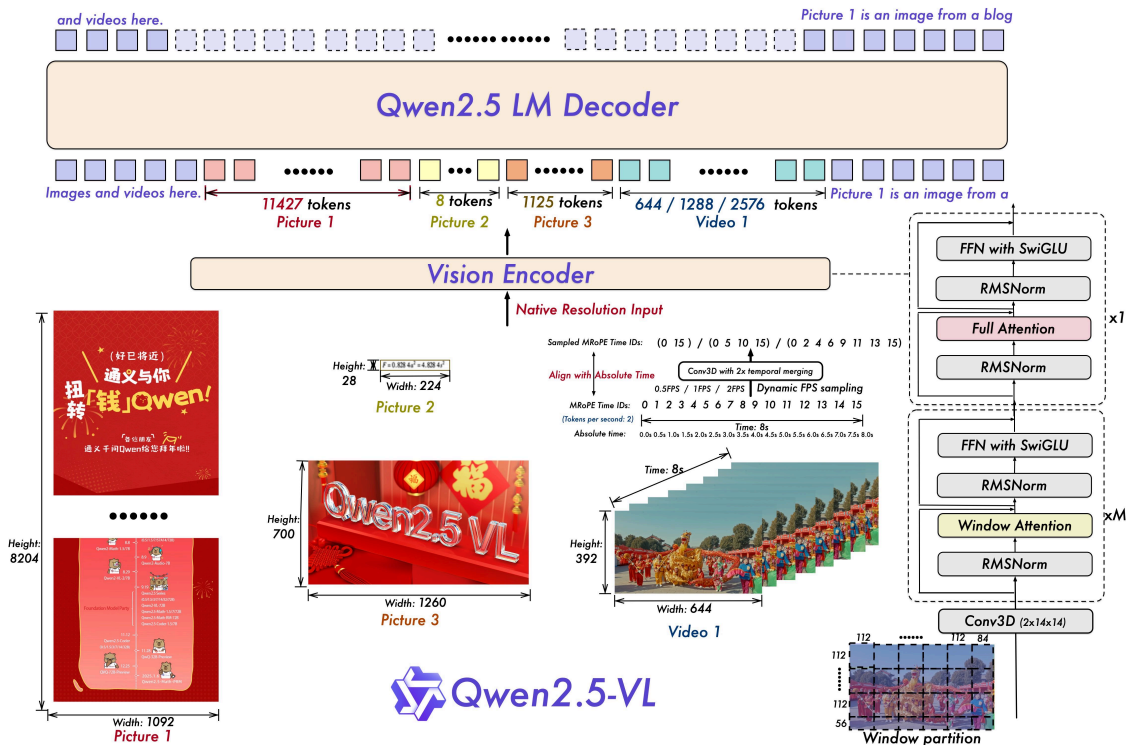


图 3.8 Qwen2.5-VL 框架展示了视觉编码器与语言模型解码器的集成，用以处理包括图像和视频在内的多模态输入。视觉编码器被设计为处理原生分辨率的输入并支持动态帧率采样。不同尺寸的图像和具有不同时帧率的视频帧被动态映射为长度各异的 token 序列。值得注意的是，MRoPE 在时间维度上将时间 ID 与绝对时间对齐，使模型能够更好地理解时间动态，例如事件的节奏和精确的时刻定位。处理后的视觉数据随后被输入到 Qwen2.5 LM 解码器。我们对 ViT 架构进行了重构，加入了诸如带 SwiGLU 激活的前馈网络（FFN）、用于归一化的 RMSNorm 以及基于窗口的注意力机制等先进组件，以提升性能和效率。

Qwen2.5-VL 的整体模型架构由三部分组成：

1. 大语言模型：Qwen2.5-VL 系列以大语言模型（LLM）作为其基础组件。该模型以 Qwen2.5 LLM 的预训练权重进行初始化。为了更好地满足多模态理解的需求，我们将一维 RoPE（旋转位置嵌入）修改为与绝对时间对齐的多模态旋转位置嵌入（Multimodal Rotary Position Embedding Aligned to Absolute Time）。
2. 视觉编码器：Qwen2.5-VL 的视觉编码器采用了重新设计的 ViT 架构。在结构上，我们引入了二维 RoPE 和窗口注意力，以支持原生输入分辨率并加速整个视觉编码器的计算。在训练和推理过程中，输入图像的高度和宽度在送入 ViT 之前会被调整为 28 的倍数。视觉编码器通过以 14 的步幅将图像切分为补丁来处理图像，生成一组图像特征。
3. 基于 MLP 的视觉-语言合并器：为了解决图像特征长序列带来的效率问题，我们采用一种简单但有效的方法：在将特征序列输入大型语言模型（LLM）之前对其进行压缩。具体来说，我们不是直接使用由 ViT 提取的原始的补丁嵌入，我们首先将空间上相邻的四个补丁嵌入分为一组。然后将这些分组嵌入串联起来，并通过一个两层的多层感知器（MLP）投影到与在 LLM 中使用的文本嵌入对齐的维度。该方法不仅降低了计算成本，还为动态压缩不同长度的图像特征序列提供了一种灵活的途径。

4. 扩散模型

扩散模型的兴起可以看作是近年来 AI 生成艺术作品领域取得突破的主要因素。

在图像创作方面,扩散模型已成为内容生成领域的前沿技术。尽管该模型于 2015 年首次推出,但已取得显著进展,并已成为 DALL·E 和 Midjourney 等知名模型的核心机制。



DDPM 算法

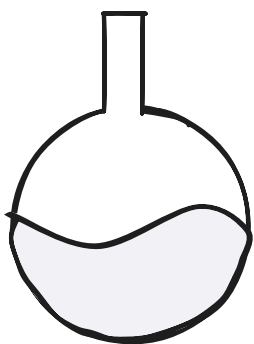
Denoising Diffusion Probabilistic Models, 去噪扩散概率模型

4.1 通俗讲解

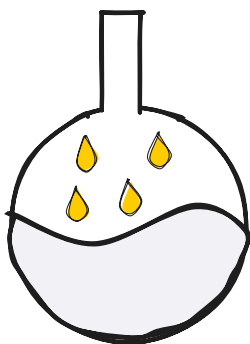
4.1.1 扩散

4.1.1.1 物理学中的类比

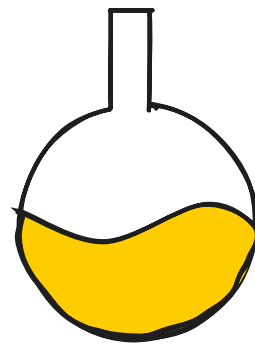
想象一下一杯透明的水。如果我们加入少量其他颜色的液体,比如黄色的液体,会发生什么?黄色液体会逐渐均匀地扩散到整个玻璃杯中,最终的混合物会呈现出略带透明的黄色。



透明的水



添加了一点黄色液体



水不再透明

图 4.1 水滴的扩散过程

上面的过程被称为 **正向扩散**：我们通过添加少量其他液体来改变环境状态。然而，进行 **反向扩散**——将混合物恢复到其原始状态——是否同样容易？事实证明并非如此。即使在最好的情况下，实现这一点也需要高度复杂的机制。

4.1.1.2 将类比应用于机器学习

扩散也可以应用于图像。想象一下一张高质量的狗狗照片。我们可以通过逐渐添加随机噪声来轻松地变换这幅图像。结果，像素值会发生变化，使图像中的狗狗变得不那么明显，甚至无法辨认。这个变换过程称为 **正向扩散**。

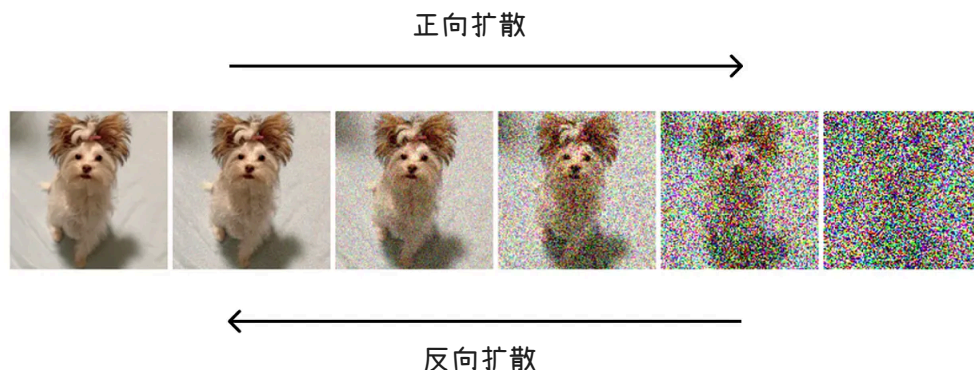


图 4.2 高清图像的扩散过程

我们也可以考虑反向操作：给定一张噪声图像，目标是重建原始图像。这项任务更具挑战性，因为与大量可能的噪声变化相比，可高度识别的图像状态要少得多。用前面提到的物理类比，这个过程称为 **反向扩散**。

在本文中，我将通过示意图来解释它的工作原理。

4.1.2 扩散模型的架构

为了更好地理解扩散模型的结构，让我们分别检查两个扩散过程。

4.1.2.1 正向扩散

如前所述，前向扩散涉及逐步向图像添加噪声。然而，在实践中，这个过程要更加微妙一些。

最常见的方法是从均值为 0 的 **高斯分布** 中为图片中的每个像素采样一个随机值。然后将这个采样值（可以是正值也可以是负值）添加到像素的原始值中。对所有像素重复此操作会得到原始图像的噪声版本。

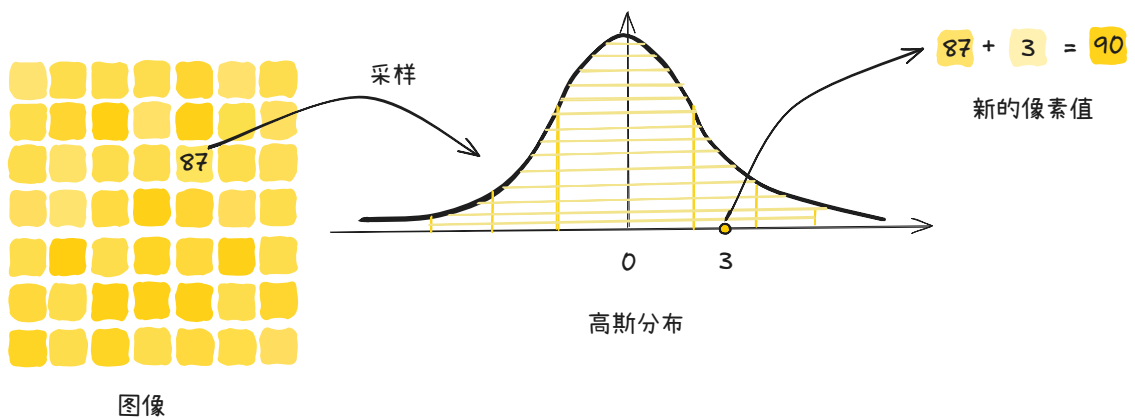


图 4.3 从高斯分布中采样一个随机值

 通知

所选的高斯分布通常方差较小，这意味着采样值通常较小。因此，每一步只会对图像产生微小的变化。

图中有 49 个像素点，那么需要从一个相同的高斯分布中采样 49 次噪声，然后将 49 个噪声分别加到 49 个像素点上。

正向扩散是一个迭代过程，其中噪声被多次应用于图像。随着每次迭代，生成的图像与原始图像的差异越来越大。经过数百次迭代（这在实际扩散模型中很常见）后，图像最终变得无法从纯噪声中识别出来。

4.1.2.2 反向扩散

现在你可能会问：执行所有这些正向扩散变换的目的是什么？答案是，每次迭代生成的图像都用于训练神经网络。

具体来说，假设我们在正向扩散过程中应用了 100 次连续噪声变换。然后，我们可以在每一步获取图像，并训练神经网络重建上一步的图像。预测图像与实际图像之间的差异使用损失函数计算——例如均方误差（MSE），它衡量两幅图像之间的平均像素差异。

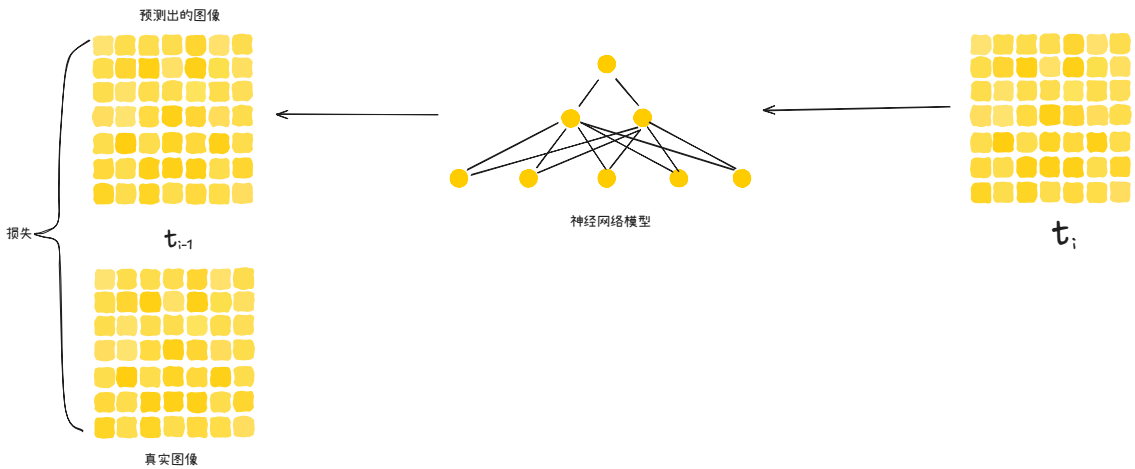


图 4.4 预测图像与实际图像之间的差异使用损失函数计算

🔔 通知

该模型的目标是检测添加的噪声并重建先前的图像。然后将预测图像与实际图像进行比较以计算损失。

这个例子展示了扩散模型重建原始图像的过程。同时，扩散模型可以被训练来预测添加到图像中的噪声。在这种情况下，要重构原始图像，只需要从前一次迭代的图像中减去预测的噪声就足够了。

虽然这两个任务看起来可能相似，但预测添加的噪声比图像重构要简单。

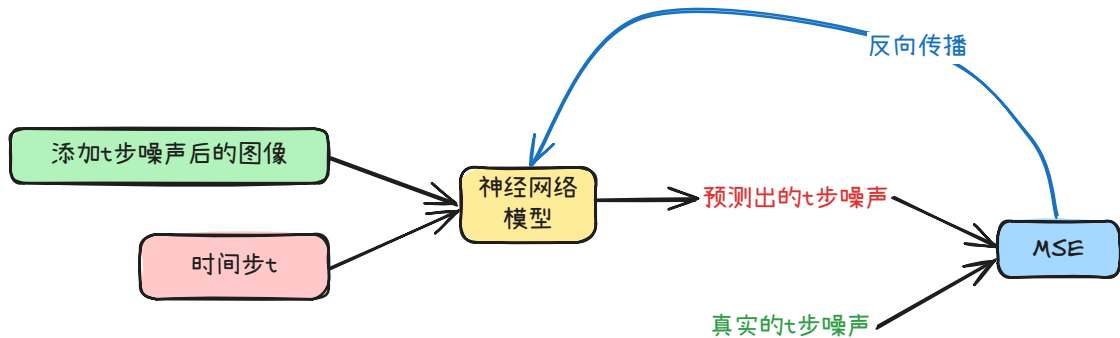


图 4.5 将噪声作为预测目标

4.1.3 模型设计

在对扩散技术有了基本的了解之后，有必要探索一些更高级的概念，以更好地理解扩散模型设计。

4.1.3.1 迭代次数

迭代次数是扩散模型中的关键参数之一：

🔔 通知

一方面，使用更多迭代次数意味着相邻步骤中的图像对差异会更小，从而使模型的学习任务更容易。另一方面，更高的迭代次数会增加计算成本。

虽然较少的迭代次数可以加快训练速度，但模型可能无法学习步骤之间的平滑过渡，从而导致性能不佳。

通常，迭代次数选择在 50 到 1000 之间。

4.1.3.2 神经网络架构

最常见的是，U-Net 架构被用作扩散模型的主干。以下是一些原因：

- U-Net 保留了输入和输出图像的尺寸，确保在整个逆扩散过程中图像大小保持一致。
- 其瓶颈架构能够在将整幅图像压缩到潜在空间后将其重建。同时，通过残差连接保留关键图像特征。
- U-Net 最初设计用于生物医学图像分割，其中像素级精度至关重要，它的优势可以很好地转化为需要精确预测单个像素值的扩散任务。

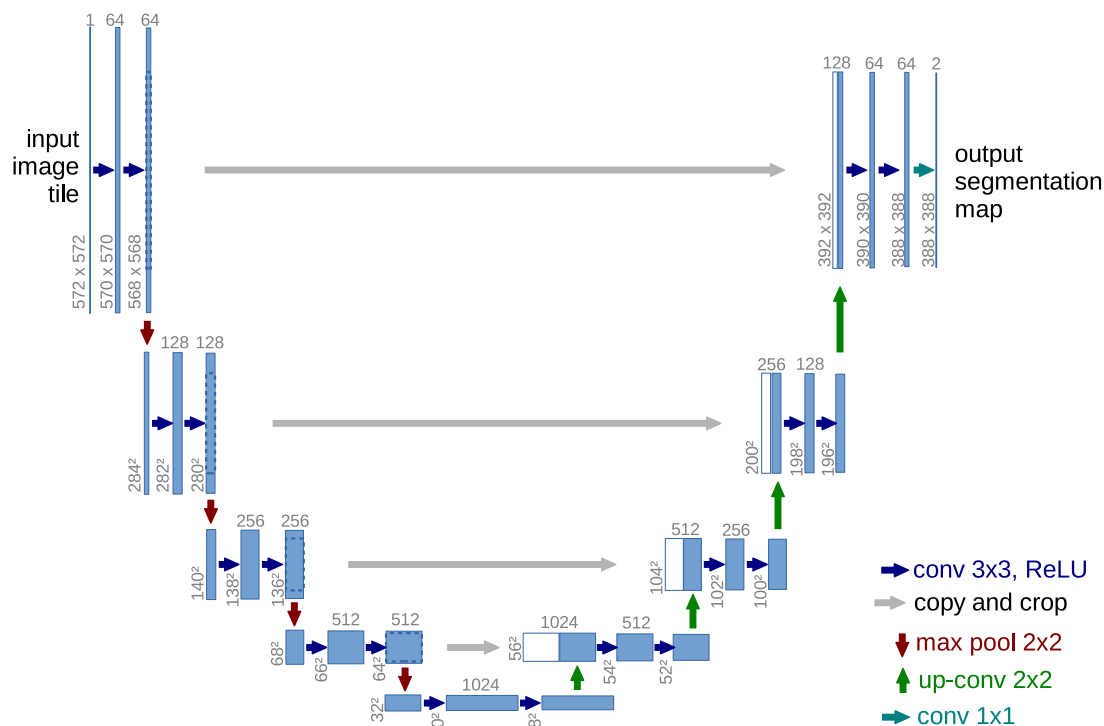


图 4.6 U-net 架构 (以最低分辨率为 32×32 像素为例)。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 $x-y$ 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。

4.1.3.3 共享网络

乍一看,似乎有必要为扩散过程的每次迭代训练一个单独的神经网络。虽然这种方法可行,并且可以产生高质量的推理结果,但从计算角度来看,它效率极低。例如,如果扩散过程包含 1000 个时间步,我们就需要训练 1000 个 U-Net 模型——这是一项极其耗时且资源密集的任务。

然而,我们可以观察到,不同迭代中的任务配置本质上是相同的:在每种情况下,我们都需要重建一个尺寸相同、且经过相似幅度噪声改变的图像。这一重要洞见促成了在所有迭代中使用单个共享神经网络的想法。

实际上,这意味着我们使用一个具有共享权重的 U-Net 模型,该模型基于来自不同扩散步骤的图像对进行训练。在推理过程中,含噪图像会多次通过同一个经过训练的 U-Net 模型,逐步进行优化,直到生成高质量的图像。

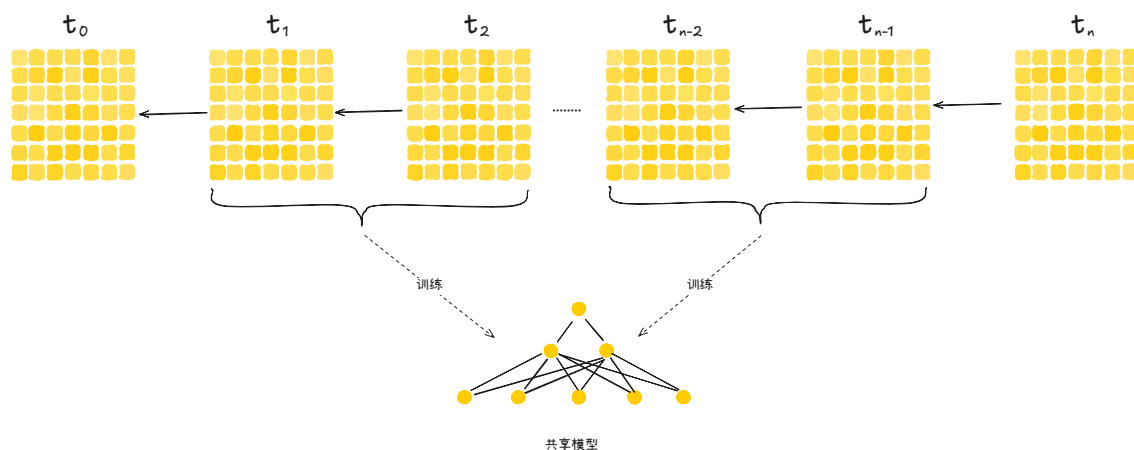


图 4.7 共享模型

虽然由于仅使用单一模型，生成质量可能会略有下降，但训练速度的提升却非常显著。

4.2 扩散模型理论简介

4.2.1 概述

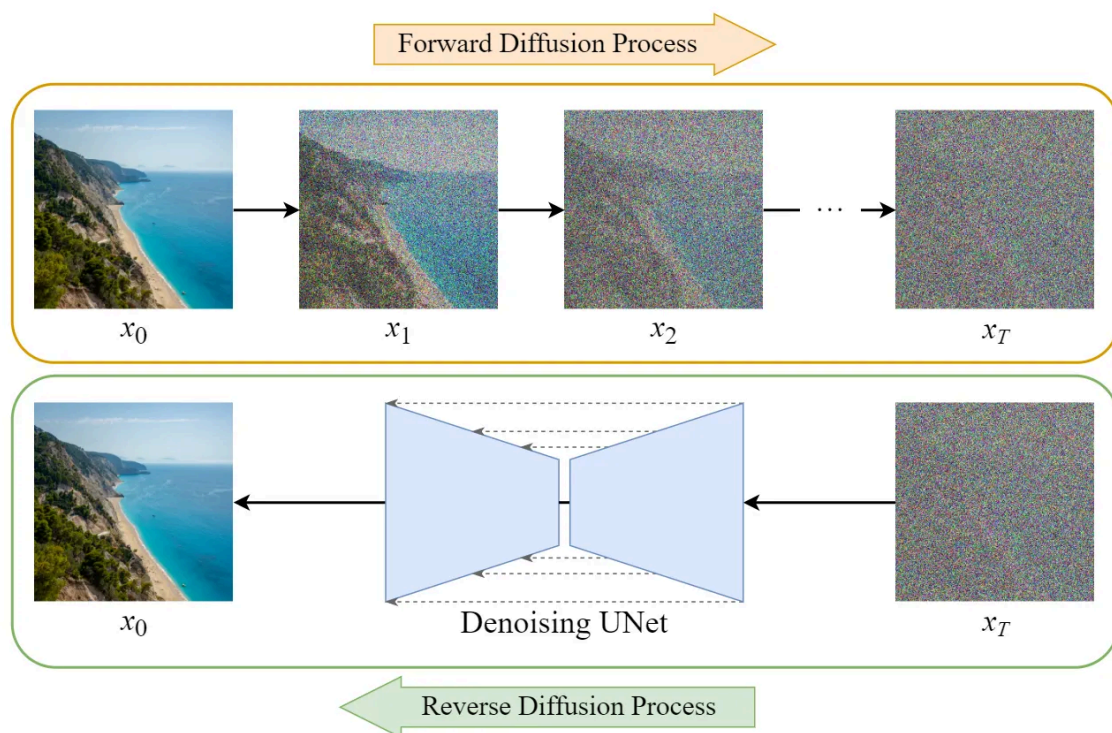


图 4.8 正向扩散和逆向扩散

扩散模型的训练可以分为两部分：

1. 正向扩散过程 → 给图像添加噪声。
2. 反向扩散过程 → 从图像中去除噪声。

4.2.2 正向扩散过程

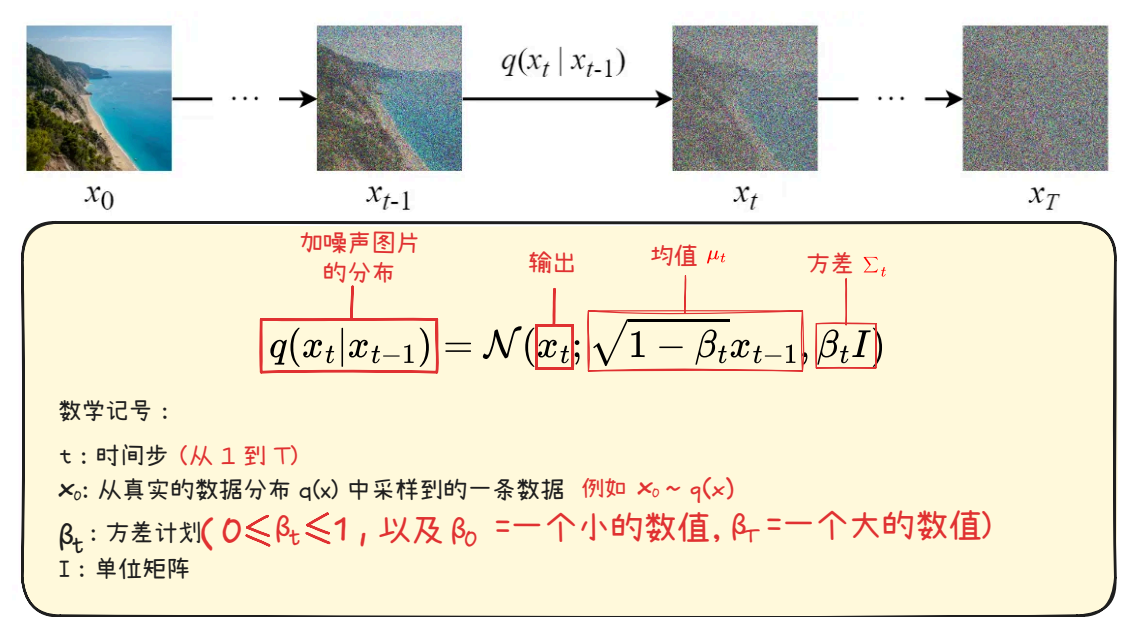


图 4.9 正向扩散公式图解

🔔 方差计划

假设 $\beta_{\text{start}} = 0.0002, \beta_{\text{end}} = 0.04$, 则第 1 步添加的高斯噪声的方差是 0.0002 , 第 2 步添加的高斯噪声的方差是 $0.0002 + \frac{0.04 - 0.0002}{1000}$ 。

前向扩散过程逐步将高斯噪声添加到输入图像 x_0 中, 总共会有 T 步。该过程将产生一系列带噪声的图像样本 $x_1, x_2, \dots, x_T$ 。

当 $T \rightarrow \infty$ 时, 最终结果将变成完全噪声图像, 就像从各向同性的高斯分布中采样出来的噪声一样。

首先, 如果 $z \sim \mathcal{N}(\mu, \sigma^2)$ 的话, 那么正态分布可以写成如下公式:

$$z = \mu + \sigma \varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0, 1) \tag{4.1}$$

利用这个技巧, 我们可以将采样图像 x_t 表示如下:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1} \tag{4.2}$$

式 (4.2) 的代码实现, 逐步向图片中添加噪声

```
1 import torch
2 import torchvision.transforms as transforms
3 import matplotlib.pyplot as plt
4
5 image = plt.imread("./flower.png")
6 print(image.shape)
7
8 preprocess = transforms.ToTensor()
9 x = preprocess(image)
10 print(image.shape)
```

Python

```

11
12 def reverse_to_img(x):
13     x = x * 255
14     x = x.clamp(0, 255)
15     x = x.to(torch.uint8)
16     to_pil = transforms.ToPILImage()
17     return to_pil(x)
18
19 # 最大时间步
20 T = 1000
21 # 方差计划的起始值
22 beta_start = 0.0001
23 # 方差计划的结束值
24 beta_end = 0.02
25 betas = torch.linspace(beta_start, beta_end, T)
26 print(betas)
27
28 imgs = []
29
30 for t in range(T):
31     if t % 100 == 0:
32         img = reverse_to_img(x)
33         imgs.append(img)
34
35     beta = betas[t]
36     eps = torch.randn_like(x) # 生成和x形状相同的噪声
37     x = torch.sqrt(1 - beta) * x + torch.sqrt(beta) * eps
38
39 # 2行5列的方式显示10张图片
40 plt.figure(figsize=(15, 6))
41 for i, img in enumerate(imgs[:10]):
42     plt.subplot(2, 5, i + 1)
43     plt.imshow(img)
44     plt.title(f"Noise: {i * 100}")
45     plt.axis("off")
46
47 plt.show()

```

⚡ 一步一步的添加噪声太麻烦了！

根据上面的公式，如果想要从原始图片 x_0 得到添加了 500 步噪声的图片 x_{500} 需要迭代 500 次！

但我们不需要设计一种算法来迭代地向图像中添加噪声，而是可以使用闭式公式（解析解）在特定的时间步长 t 直接对噪声图像进行采样。

给定原始图片 x_0 和时间步 t 可以直接得到添加了 t 步噪声的图像 x_t 。公式如下：

🔥 给定原始图片 x_0 和时间步 t 直接采样出 x_t 的公式

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (4.3)$$

其中:

- $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1$
- $\alpha_t = 1 - \beta_t$
- $\varepsilon \sim \mathcal{N}(0, 1)$

现在我们可以使用此公式在任何时间步直接对 x_t 进行采样，这使得前向扩散过程更快。

式 (4.3) 的实现，使用闭式解添加噪声

Python

```
1 import torch
2 import torchvision.transforms as transforms
3 import matplotlib.pyplot as plt
4
5 image = plt.imread("./flower.png")
6 print(image.shape)
7
8 preprocess = transforms.ToTensor()
9 x = preprocess(image)
10 print(image.shape)
11
12
13 def reverse_to_img(x):
14     x = x * 255
15     x = x.clamp(0, 255)
16     x = x.to(torch.uint8)
17     to_pil = transforms.ToPILImage()
18     return to_pil(x)
19
20
21 # 最大时间步
22 T = 1000
23 # 方差计划的起始值
24 beta_start = 0.0001
25 # 方差计划的结束值
26 beta_end = 0.02
27 betas = torch.linspace(beta_start, beta_end, T)
28 print(betas)
29
30 # 一步得到  $x_t$ , 使用闭式解 (closed form)
31 def add_noise(x_0, t, betas):
32     T = len(betas)
33
34     alphas = 1 - betas # [a_1, a_2, ...]
35     # cumprod功能: [1,2,3,4] → [1,2,6,24]
36     alpha_bars = torch.cumprod(alphas, dim=0)
37     t_idx = t - 1
38     alpha_bar = alpha_bars[t_idx] # alpha_bar_t
```

```

39
40     eps = torch.randn_like(x_0)
41     # 闭式解
42     x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar)*eps
43
44     return x_t
45
46 t = 100
47 x_t = add_noise(x, t, betas)
48
49 img = reverse_to_img(x_t)
50 plt.imshow(img)
51 plt.title(f"Noise: {t}")
52 plt.axis("off")
53 plt.show()

```

4.2.3 反向扩散过程

反向扩散过程是从一张完全的高斯噪声图片中，逐步去除噪声，来生成一张图片。这个逆向过程很难，相当于一滴墨水在水里面扩散开来（正向扩散），想要将变黑的水逆向为原来的墨水刚滴入水中的状态。这几乎不可能做到，所以我们用神经网络来解决这个问题。下面是训练神经网络的原理。

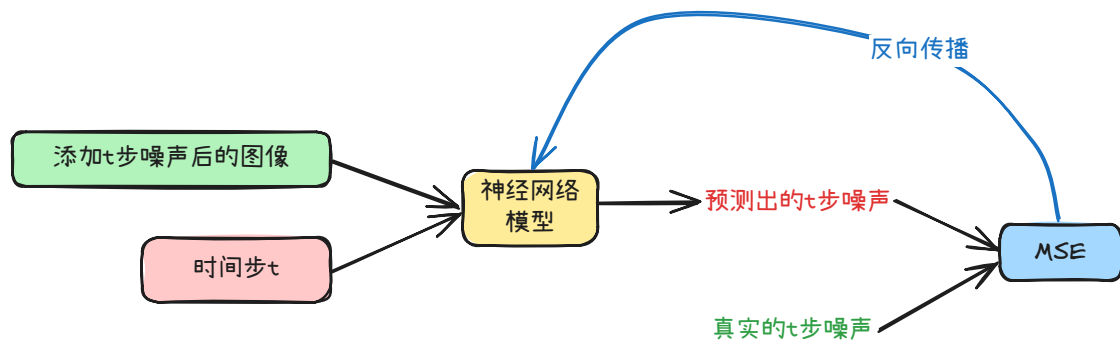


图 4.10 以噪声为预测目标

因此最终的训练目标如下：

$$\begin{aligned}
 & \text{时间步 } t \text{ 的图片} \\
 & x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon \\
 & \mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2]
 \end{aligned} \tag{4.4}$$

4.2.4 U-Net 模型

4.2.4.1 数据集

在每个 Epoch:

1. 将为每个训练样本（图像）选择一个随机时间步长 t 。
2. 对每幅图像添加高斯噪声（对应于 t ）。
3. 将时间步转换为嵌入（向量）。

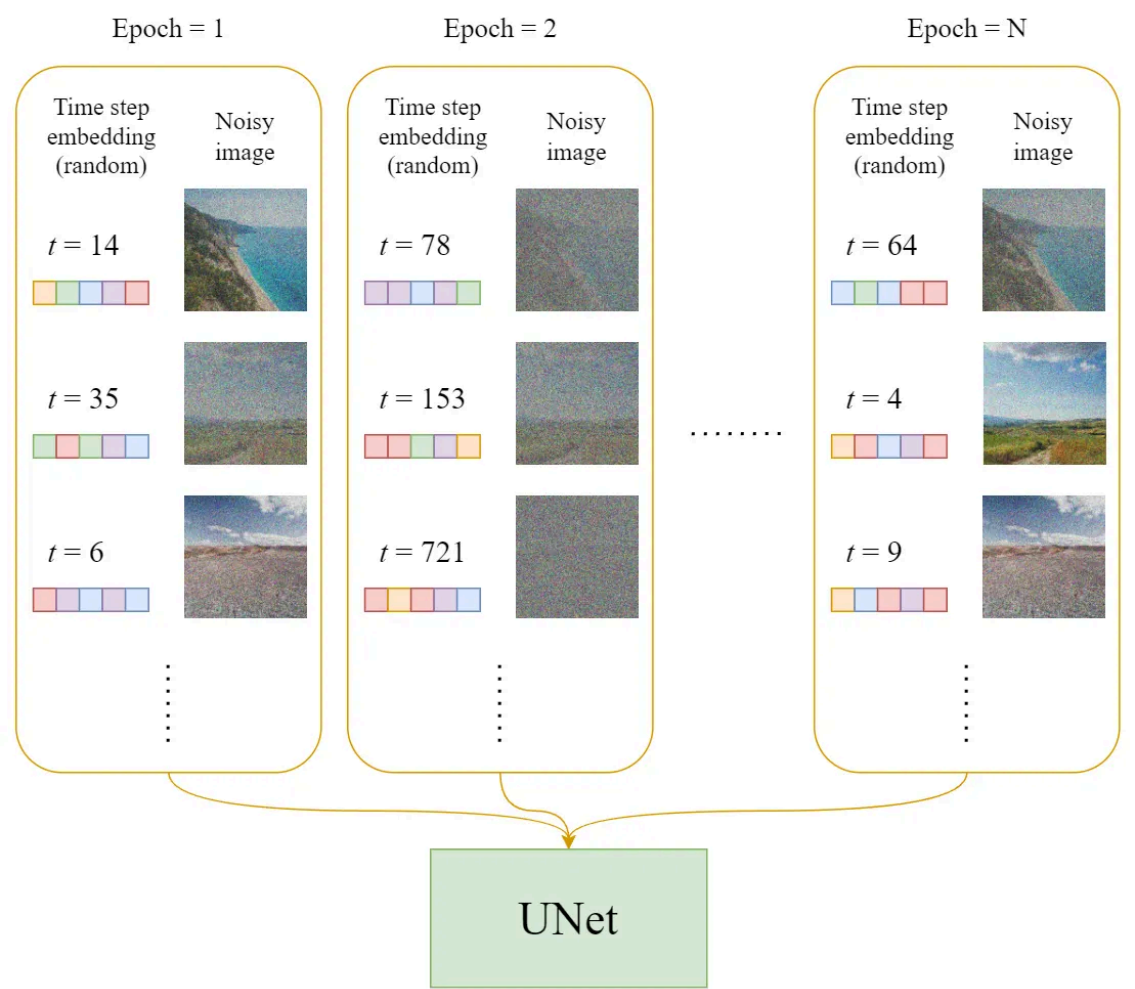


图 4.11 训练 unet

4.2.4.2 训练

1 Repeat

2 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$

3 $t \sim \text{Uniform}(\{1, \dots, T\})$

4 $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

5 使用梯度下降法，梯度为

6 $\nabla_{\theta} \|\varepsilon - \varepsilon_{\theta}(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\varepsilon, t)\|^2 \quad (4.5)$

7 Until 收敛

▷ 从数据集中抽取一张图片

▷ 从均匀分布中采样一个时间步

▷ 从正态分布中采样一个噪声

图 4.12 U-net 训练算法

官方的训练算法如上，下图是一个训练步骤的示意图：

针对每一个训练步骤（step）

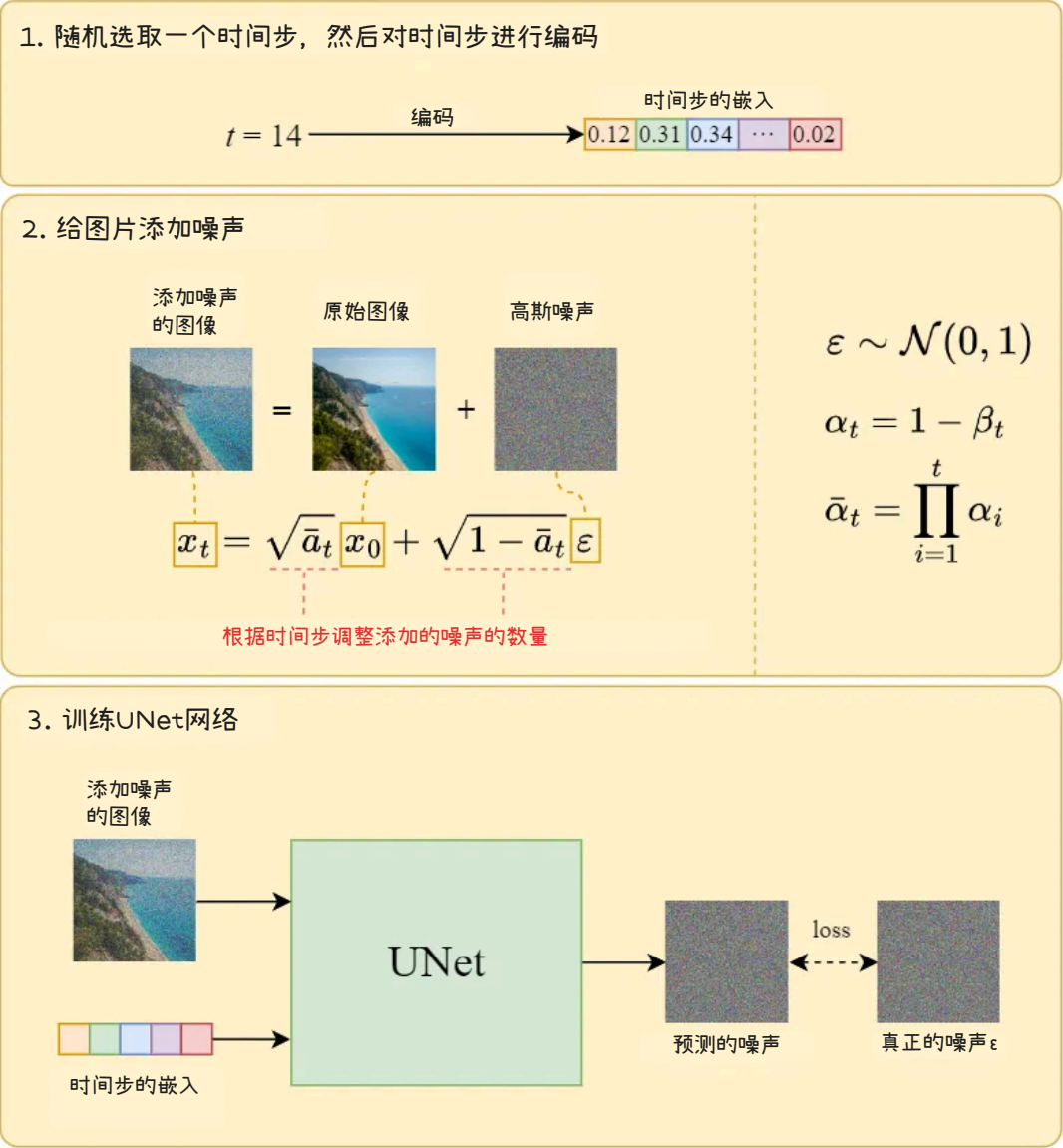


图 4.13 扩散模型训练步骤

4.2.4.3 反向扩散

```

1   $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$  ▷ 从高斯分布中采样一张纯噪声图片
2  for  $t = T, \dots, 1$  do
3       $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = 0$ 
4       $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(\mathbf{x}, t) \right) + \sigma_t \mathbf{z}$  ▷  $\sigma_t$ 一般取 $\sqrt{\beta_t}$ 
5  end for
6  return  $\mathbf{x}_0$ 

```

图 4.14 采样算法（去噪算法，反向扩散算法）

我们可以使用上述算法从噪声中生成图像。下图是它的说明：

反向扩散/去噪/采样

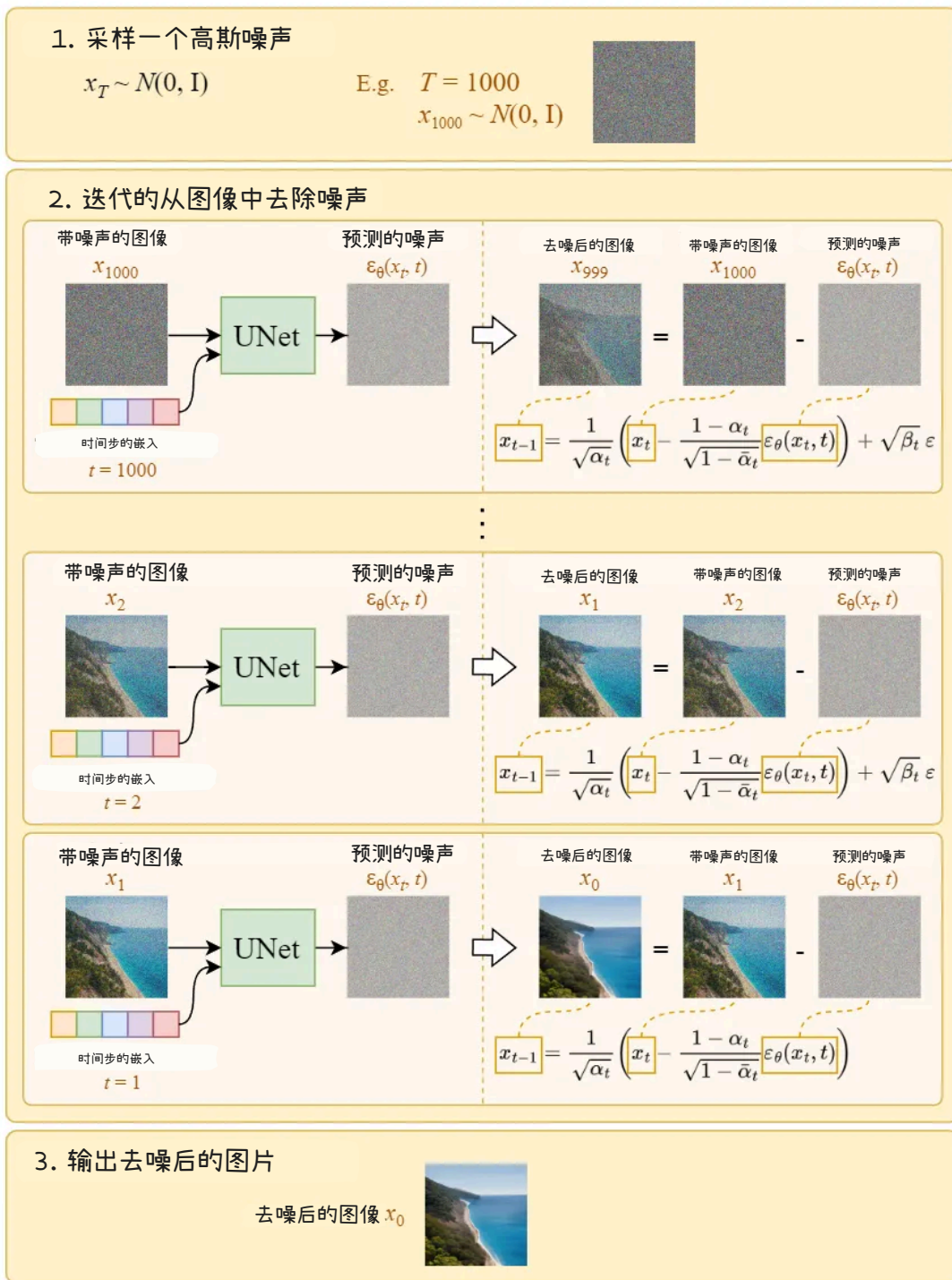


图 4.15 扩散模型的采样过程

请注意，在最后一步，我们只是输出学习到的平均值 $\mu_{\theta}(x_1, 1)$ ，而不向其中添加噪声。

4.3 代码实现

4.3.1 扩散过程相关代码

我们需要针对扩散的时间步来制定一个方差的调度计划。每个时间步，都在前面一个时间步的图像中添加一个高斯噪声。在开始向图像中添加噪声的时候，需要高斯噪声的方差小一些，不至于一开始就把图像变得很模糊。而到了后面，添加噪声就可以大胆一些了，反正已经模糊了。所以后面添加的高斯噪声需要方差大一些。

🔥 高斯噪声的方差

公式中的 β_t 就是第 t 个时间步要添加的噪声的方差。

在代码中如下

```
1 # self.betas: 方差计划调度表
2 self.betas = torch.linspace(
3     beta_start, # beta_start: 起始β值, 论文中等于0.0001
4     beta_end, # beta_end: 结束β值, 论文中等于0.02
5     num_timesteps, # num_timesteps: 时间步的数量: 1000
6     device=device
7 )
```

Python

而由于 $\alpha_t = 1 - \beta_t$ ，所以有如下代码

```
1 self.alphas = 1 - self.betas
```

Python

这样就计算出了每个时间步的 α_t 。

而由于 $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，所以使用 `torch.cumprod` 来计算

```
1 self.alphaBars = torch.cumprod(self.alphas, dim=0)
```

Python

由于我们已经使用重参数技巧来给图像添加噪声，也就是通过解析解可以直接得到添加了 t 个时间步的噪声的图像。所以前向扩散过程就用了这个公式。

🔥 给定原始图片 x_0 和时间步 t 直接采样出 x_t 的公式

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (4.6)$$

其中：

- $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1 = \prod_{s=1}^t \alpha_s$
- $\alpha_t = 1 - \beta_t$
- $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

公式和代码基本是对应的。

```
1 def add_noise(self, x_0, t):
2     # x_0是原始图片, t是时间步
3     # T: 时间步的数量
4     T = self.num_timesteps
5     # 确保 1 ≤ t ≤ T
6     assert (t ≥ 1).all() and (t ≤ T).all()
7
8     t_idx = t - 1 # alphaBars[0] is for t=1
```

Python

```

9      #  $\bar{\alpha}_t$ 
10     alpha_bar = self.alpha_bars[t_idx] # (N,)
11     N = alpha_bar.size(0)
12     alpha_bar = alpha_bar.view(N, 1, 1, 1) # (N, 1, 1, 1)
13     #  $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ 
14     # 噪声的形状或者点数等于图片的像素点
15     noise = torch.randn_like(x_0, device=self.device)
16     #  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$ 
17     x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar) * noise
18     # 返回第t步的图像 $x_t$ 和添加的噪声noise, 后面作为训练数据
19     return x_t, noise

```

接下来编写反向去噪过程的代码, 也就是从 x_t 预测 x_{t-1} 。

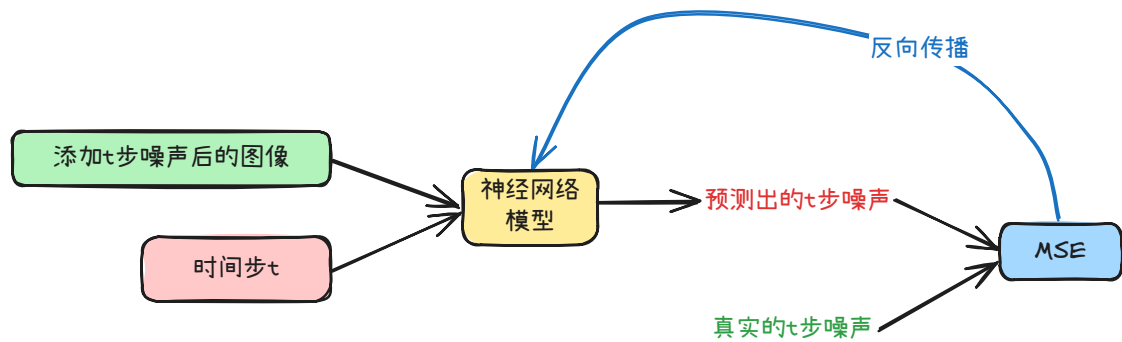


图 4.16 以噪声为预测目标

给定了时间步和添加噪声的图像, 模型可以预测出这幅图片中的噪声有多少, 那么我们从 x_t 中将预测出的噪声减掉, 就可以去噪了!

```

1  def denoise(self, model, x, t):
2      # x是图片, 模型会认为x是添加了t步噪声的图片
3      # t是时间步
4      T = self.num_timesteps
5      assert (t >= 1).all() and (t <= T).all()
6
7      t_idx = t - 1 # alphas[0] is for t=1
8      alpha = self.alphas[t_idx] #  $\alpha_t$ 
9      alpha_bar = self.alpha_bars[t_idx] #  $\bar{\alpha}_t$ 
10     alpha_bar_prev = self.alpha_bars[t_idx-1] #  $\bar{\alpha}_{t-1}$ 
11
12     N = alpha.size(0)
13     alpha = alpha.view(N, 1, 1, 1)
14     alpha_bar = alpha_bar.view(N, 1, 1, 1)
15     alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
16
17     model.eval()
18     with torch.no_grad():
19         eps = model(x, t) # 根据时间步和图像x, 预测噪声
20         model.train()
21         # 公式中的z

```

```

22 noise = torch.randn_like(x, device=self.device)
23 # 如果时间步为1的话, 我们就不再添加 $\sigma_z$ 噪声
24 noise[t == 1] = 0 # no noise at t=1
25 # 均值 $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(x_t, t) \right)$ 
26 mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps)
27 / torch.sqrt(alpha)
28 # 标准差 $\sigma_t = \sqrt{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\alpha_t}}$ 
29 std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) \
30 / (1-alpha_bar))
31 # 返回 $x_{t-1}$ 
32 return mu + noise * std

```

```

1  $x_T \sim \mathcal{N}(0, \mathbf{I})$  ▷ 从高斯分布中采样一张纯噪声图片
2 for  $t = T, \dots, 1$  do
3      $z \sim \mathcal{N}(0, \mathbf{I})$  if  $t > 1$ , else  $z = 0$ 
4      $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(x, t) \right) + \sigma_t z$  ▷ 在 DDPM 论文中 $\sigma_t = \sqrt{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\alpha_t}}$ 
5 end for
6 return  $x_0$ 

```

图 4.17 采样算法（去噪算法，反向扩散算法）

上面的代码实现了伪代码中的第 4 步。

4.3.2 U-Net 神经网络

U-Net 最初是为医学图像的语义分割而开发的模型。语义分割是为图像中的每个像素分配特定的类别标签的任务，如图所示。

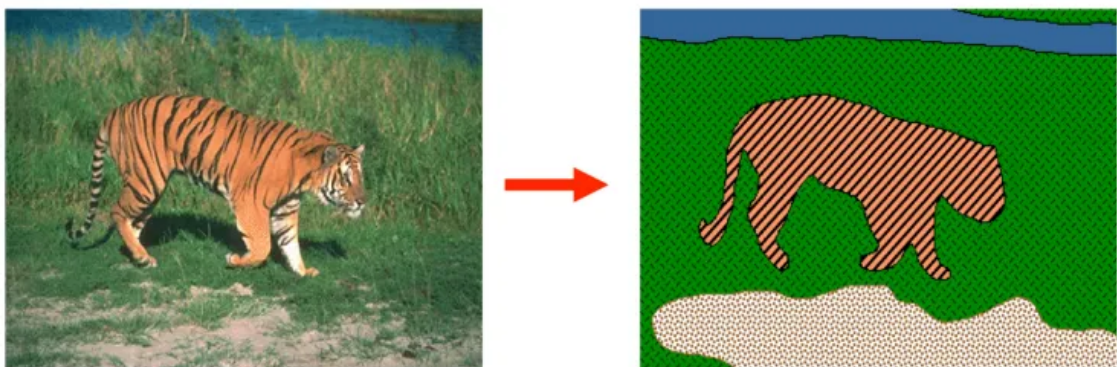


图 4.18 U-Net 执行语义分割

图中 U-Net 的输入是形状为 (c, h, w) 的图像数据，其中， c 是输入图像的通道数（RGB 图像为 3）， h 是图像的高度， w 是图像的宽度。输出是形状为 (d, h, w) 的张量，其中 d 是要分类的类别数。模型对每个像素输出 d 个类别的概率分布。而在扩散模型中，输出被设置为 (c, h, w) （ $D=C$ ），即输入和输出的通道数都被设置为 c 。

U-Net 的名称源于网络结构的形状，因为它与字母“U”的形状相似。

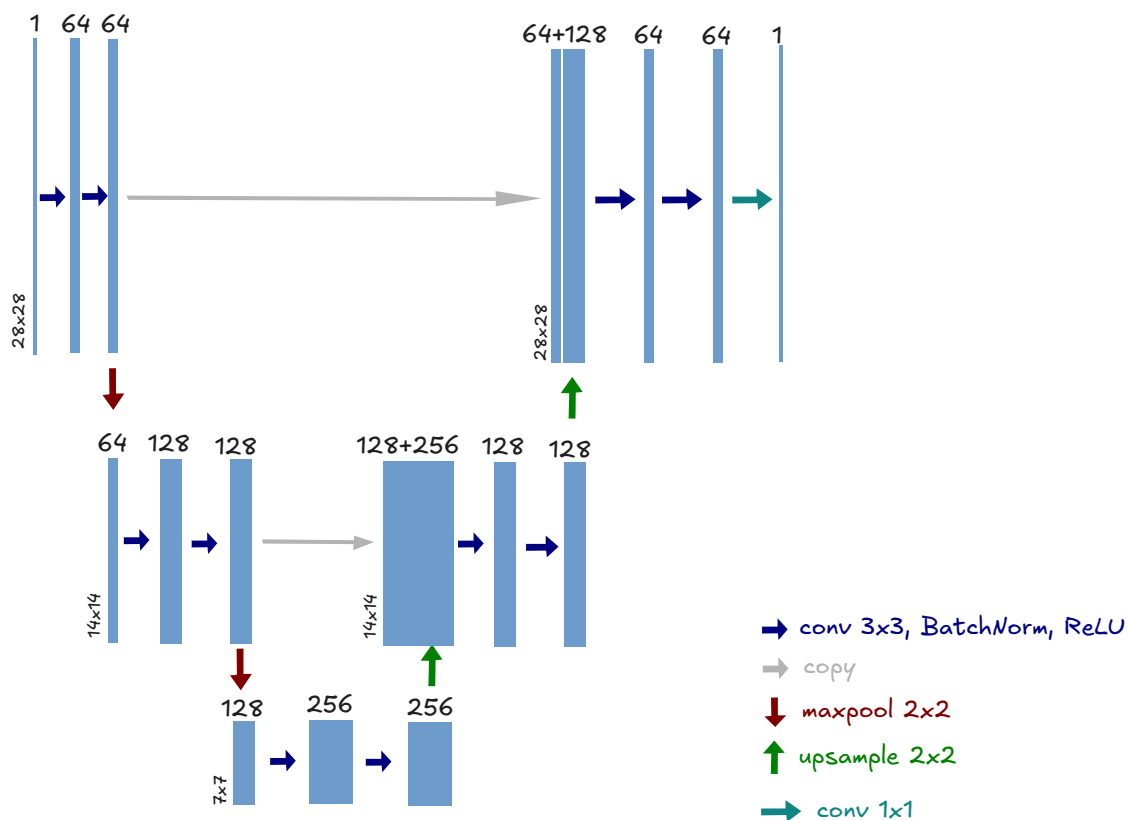


图 4.19 本章要实现的 U-Net

U-Net 的处理过程分为前半部分的“缩小阶段”和后半部分的“扩大阶段”。在前半部分的缩小阶段中，在卷积层进行处理的同时，特征图会逐渐缩小。缩小特征图的层称为“下采样层”。在后半部分的扩大阶段中，在卷积层进行特征抽取的同时，特征图会逐渐扩大，这与前半部分正好相反。扩大特征图的层称为“上采样层”。

U-Net 的重要特征是跳跃连接 (skip connection)。这是一种在网络的缩小阶段和扩大阶段之间直接传递特征图的机制。这种跳跃连接使得 U-Net 能够捕捉对象整体的特征，同时使用更精细的空间位置信息进行处理。

如上图所示，U-Net 包含两个缩小阶段和两个扩大阶段。每个阶段由两个卷积层进行处理。为了实现这个 U-Net，首先要实现一个名为 ConvBlock 的类。如下图所示，该类分别对卷积层，批量归一化层和 ReLU 函数的处理执行了两次。

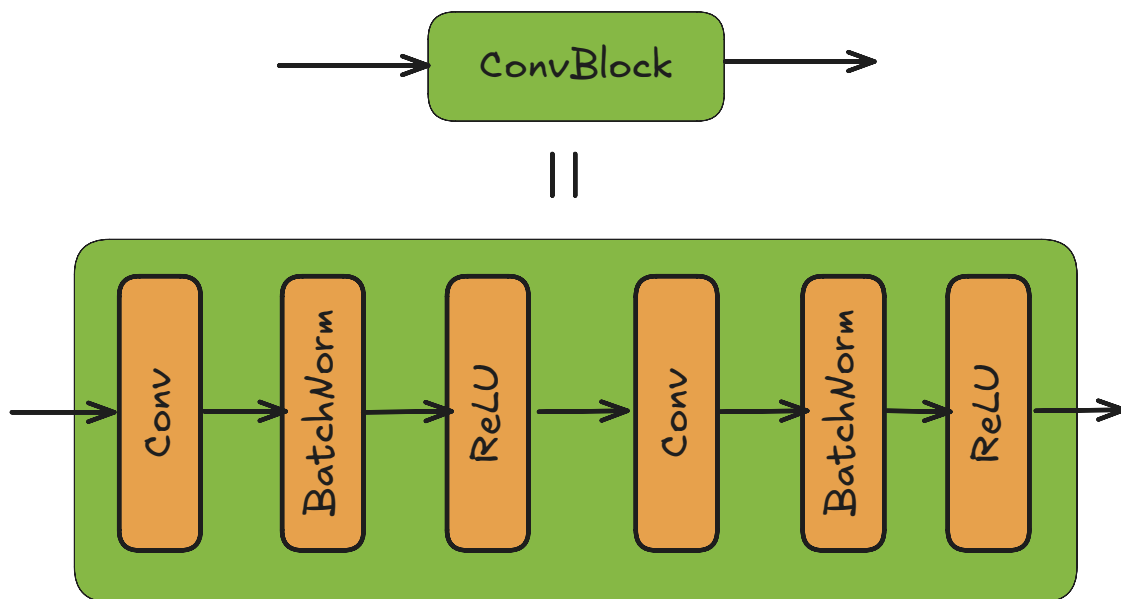


图 4.20 ConvBlock 类执行的处理

然后是 ConvBlock 类的代码，如下所示。

```

1  import torch
2  from torch import nn
3
4  class ConvBlock(nn.Module):
5      def __init__(self, in_ch, out_ch):
6          super().__init__()
7          self.convs = nn.Sequential(
8              nn.Conv2d(in_ch, out_ch, 3, padding=1)
9              nn.BatchNorm2d(out_ch)
10             nn.ReLU()
11             nn.Conv2d(out_ch, out_ch, 3, padding=1)
12             nn.BatchNorm2d(out_ch)
13             nn.ReLU()
14         )
15
16     def forward(self, x):
17         return self.convs(x)

```

这段代码使用 `nn.Sequential()` 串联了多个层。这样做的目的是使数据按顺序逐层通过。使用 `ConvBlock` 类，我们可以实现上图所示的 UNet 类。代码如下所示。

```

1  class UNet(nn.Module):
2      def __init__(self, in_ch=1):
3          super().__init__()
4
5          self.down1 = ConvBlock(in_ch, 64)
6          self.down2 = ConvBlock(64, 128)
7          self.bot1 = ConvBlock(128, 256)
8          self.up2 = ConvBlock(128 + 256, 128)

```

```

9         self.up1 = ConvBlock(128 + 64, 64)
10        self.out = nn.Conv2d(64, in_ch, 1)
11
12        self.maxpool = nn.MaxPool2d(2)
13        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")
14
15        def forward(self, x):
16            x1 = self.down1(x)
17            x = self.maxpool(x1)
18            x2 = self.down2(x)
19            x = self.maxpool(x2)
20            x = self.bot1(x)
21
22            x = self.upsample(x)
23            x = torch.cat([x, x2], dim=1)
24            x = self.up2(x)
25            x = self.upsample(x)
26            x = torch.cat([x, x1], dim=1)
27            x = self.up1(x)
28            x = self.out(x)
29            return x

```

这段代码使用最大池化（`nn.MaxPool2d`）来缩小数据。这会使张量的大小缩小 $1/2$ 。而在扩大数据的处理中，代码使用了双线性插值的上采样（`nn.Upsample`）。这会使张量的大小扩大 2 倍。



双线性插值

双线性插值是一种用于扩大图像大小的技术。双线性插值可以创建新的像素，其值基于原始图像中像素的值计算得出。这样就能平滑地扩大图像。

进行 U-Net 的跳跃连接的代码是 `torch.cat([x, x1], dim=1)`。`torch.cat()` 函数是连接张量的函数，其中 `dim=1` 指定了连接的维度。在本例中，如果 `x` 的形状是 (N, C, H, W) ，`x1` 的形状是 (N, I, H, W) ，那么连接后的张量的形状就是 $(N, C + I, H, W)$ 。

上面我们实现了处理 x_t 的 U-Net。剩下的任务是将时刻 t 引入 U-Net 中。需要注意的是这里的 t 是整数。在神经网络中，将整数变换为向量可以提高训练和预测的效率。扩散模型在对时刻 t 进行编码时，常常使用正弦位置编码。

正弦位置编码因在论文“Attention Is All You Need”的 Transformer 模型中使用而闻名。正弦位置编码的意思是“使用正弦波（sin 波）对位置信息进行编码”。位置信息是指序列数据中每个元素出现的位置。例如，在自然语言处理任务中，单词出现的位置就相当于位置信息。扩散模型的时刻 t 也是位置信息。

正弦位置编码可以将整数 t 变换为向量 $\mathbf{v}$ 。如果变换后的向量 $\mathbf{v}$ 的维度为 D ，则其第 i 个元素的数学式如下所示。

$$\mathbf{v}_i = \begin{cases} \sin\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i \text{ 为偶数时} \\ \cos\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i \text{ 为奇数时} \end{cases} \quad (4.7)$$

上面的正弦位置编码不以绝对值来编码位置信息，而是通过具有循环特性的 `sin` 函数和 `cos` 函数来编码位置信息。这样一来，位置信息中的相对差异和循环模式就能清晰地显示出来，使模型能够更有效地学习序列数据中的相对位置关系。

下面我们来实现正弦位置编码。首先，我们将实现对单个时刻数据（整数）进行编码的函数 `_pos_encoding()`。然后，我们将使用它来实现一个名为 `pos_encoding()` 的处理批量数据的函数。以下是 `_pos_encoding()` 的代码。

```
1 import torch
2
3 def _pos_encoding(t, output_dim, device="cpu"):
4     D = output_dim
5     v = torch.zeros(D, device=device)
6     i = torch.arange(0, D, device=device) ①
7     div_term = 10000 ** (i / D)
8
9     v[0::2] = torch.sin(t / div_term[0::2]) ②
10    v[1::2] = torch.cos(t / div_term[1::2])
11    return v
```

这段代码基于输入时刻 (`t`) 和输出维度 (`output_dim`) 进行位置编码。代码①处通过 `torch.arange(D)` 创建张量 `[0, 1, ..., D]`。代码②处使用切片语法 `v[0::2]` 来指定“从 `v` 的第 0 个元素开始，每次跳过一个元素后的所有元素”。也就是说，指定向量 `v` 的偶数索引 (`0, 2, 4, ...`) 对应的元素，并对这些元素进行正弦编码。

接下来，我们将实现用于处理批量数据的正弦位置编码。为了易于理解，这里使用 `for` 语句来实现，代码如下所示。

```
1 def pos_encoding(ts, output_dim, device="cpu"):
2     batch_size = len(ts)
3     v = torch.zeros(batch_size, output_dim, device=device)
4     for i in range(batch_size):
5         v[i] = _pos_encoding(ts[i], output_dim, device)
6     return v
```

参数 `ts` 是张量。对于该张量的每个元素，调用刚刚实现的 `_pos_encoding` 函数。这样我们就完成了正弦位置编码的实现。

最后将正弦位置编码嵌入到 **U-Net** 中。这里我们将正弦位置编码嵌入到之前实现的 **ConvBlock** 类中。**ConvBlock** 是拥有两个卷积层的处理单元。我们使用 **ConvBlock** 实现了 **U-Net**，如图所示。

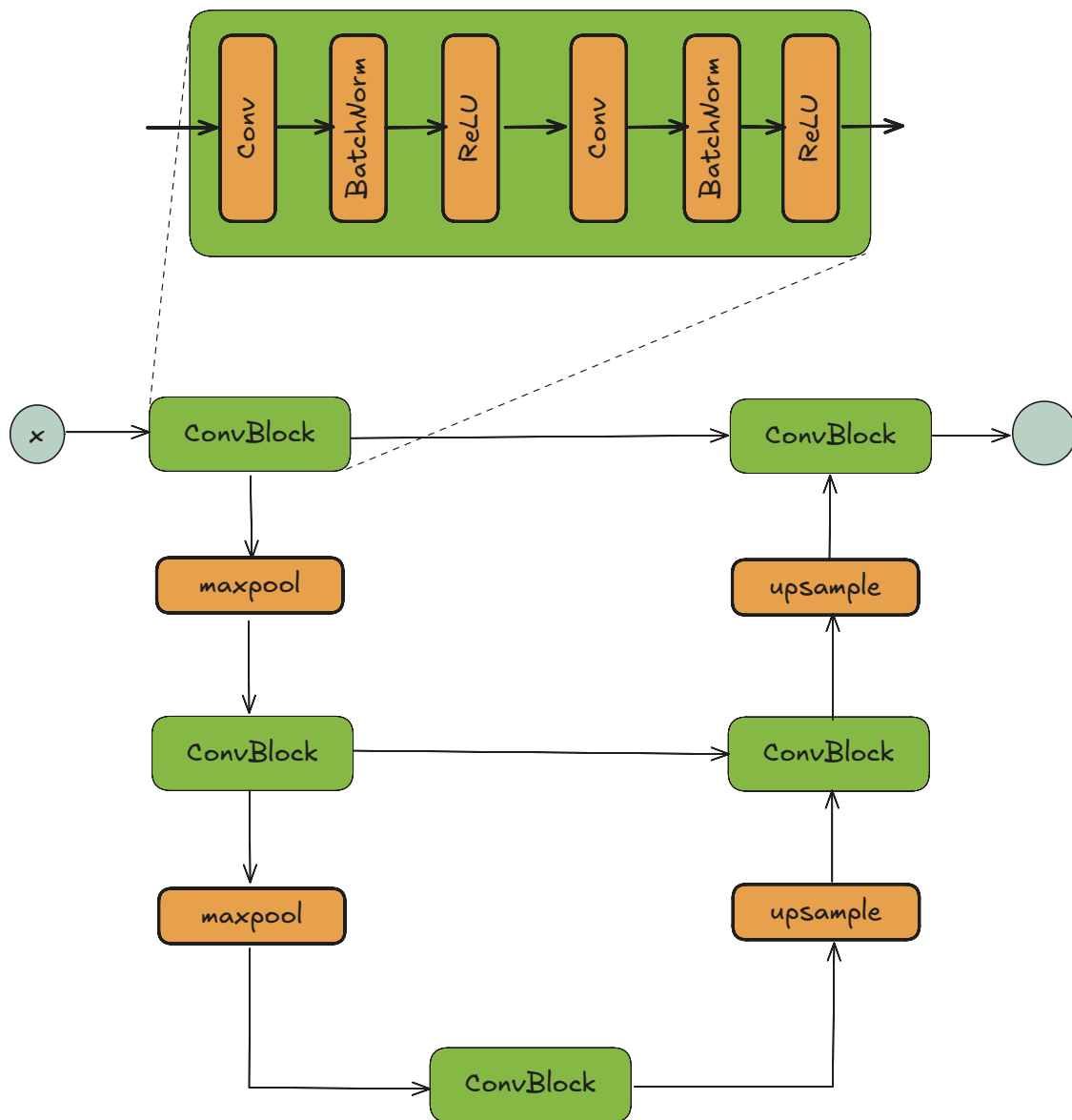


图 4.21 我们实现的 U-Net

这里要向 5 个 **ConvBlock** 添加正弦位置编码信息。

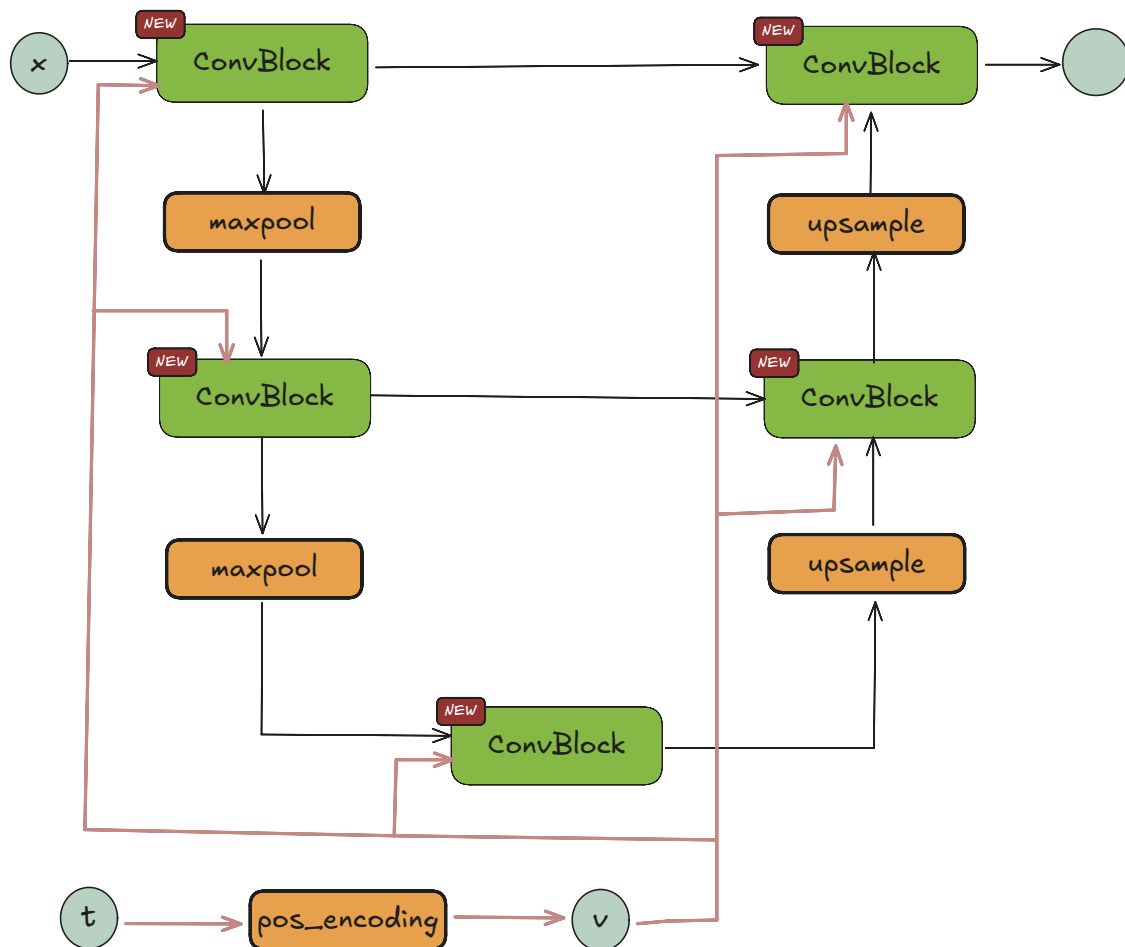


图 4.22 加入正弦位置编码的 U-Net

如图所示，新的 **ConvBlock** 中也将接收正弦位置编码信息 v 作为输入。然后在这个新的 **ConvBlock** 内部执行下图所示的处理。

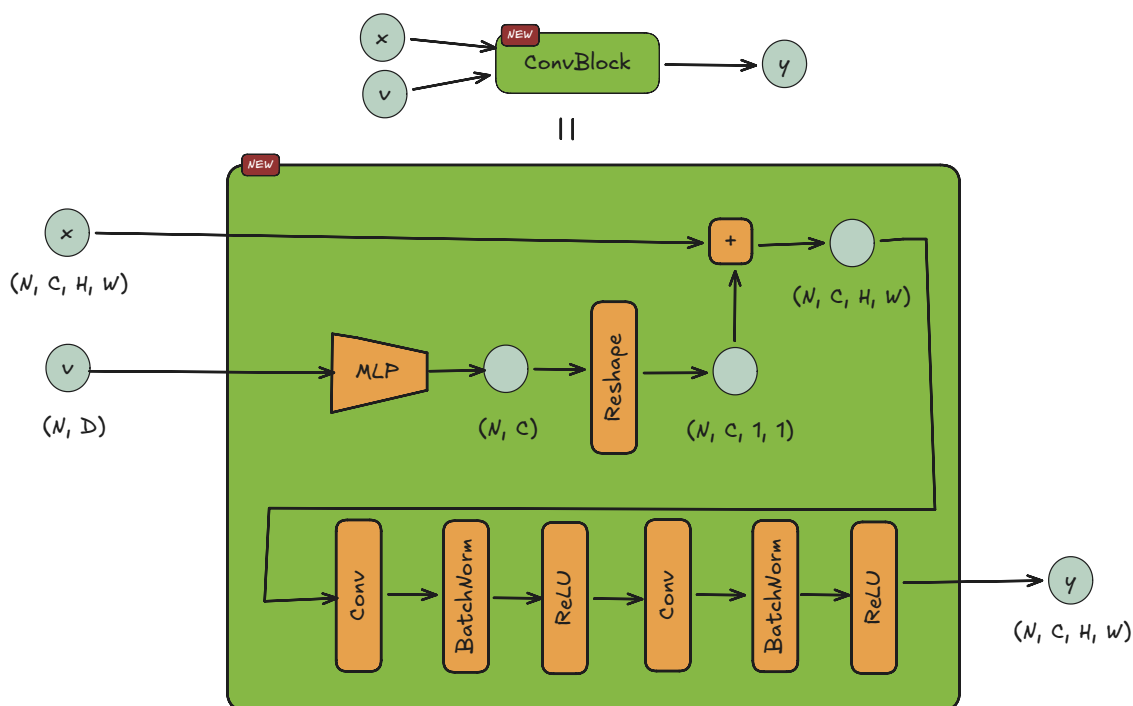


图 4.23 新的 ConvBlock 类

图中的 x 的形状为 (N, C, H, W) ， v 的形状为 (N, D) 。 v 被 MLP（多层感知机，这里指由全连接层组成的神经网络）变换为 (N, C) 的形状，然后再被变换为 $(N, C, 1, 1)$ 的形状。通过这个变形，广播函数得以应用于随后的加法运算中。新的 `ConvBlock` 的实现如下所示。

```

1  class ConvBlock(nn.Module):
2      def __init__(self, in_ch, out_ch, time_embed_dim):
3          super().__init__()
4          self.convs = nn.Sequential(
5              nn.Conv2d(in_ch, out_ch, 3, padding=1),
6              nn.BatchNorm2d(out_ch),
7              nn.ReLU(),
8              nn.Conv2d(out_ch, out_ch, 3, padding=1),
9              nn.BatchNorm2d(out_ch),
10             nn.ReLU()
11         )
12         self.mlp = nn.Sequential(
13             nn.Linear(time_embed_dim, in_ch),
14             nn.ReLU(),
15             nn.Linear(in_ch, in_ch)
16         )
17
18     def forward(self, x, v):
19         N, C, _, _ = x.shape
20         v = self.mlp(v)
21         v = v.view(N, C, 1, 1)
22         y = self.convs(x + v)
23         return y

```

参数 `time_embed_dim` 是通过正弦位置编码变换后的向量的维度。名为 `self.mlp` 的全连接层可以将维度为 `time_embed_dim` 的向量变换为维度为 `in_ch` 的向量。

最后使用这个新的 `ConvBlock` 类来实现 `UNet` 类。代码如下所示。

```

1  class UNet(nn.Module):
2      def __init__(self, in_ch=1, time_embed_dim=100):
3          super().__init__()
4          self.time_embed_dim = time_embed_dim
5
6          self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
7          self.down2 = ConvBlock(64, 128, time_embed_dim)
8          self.bot1 = ConvBlock(128, 256, time_embed_dim)
9          self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
10         self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
11         self.out = nn.Conv2d(64, in_ch, 1)
12
13         self.maxpool = nn.MaxPool2d(2)
14         self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
15
16     def forward(self, x, timesteps):
17         v = pos_encoding(timesteps, self.time_embed_dim, x.device)
18
19         x1 = self.down1(x, v)
20         x = self.maxpool(x1)
21         x2 = self.down2(x, v)
22         x = self.maxpool(x2)
23
24         x = self.bot1(x, v)
25
26         x = self.upsample(x)
27         x = torch.cat([x, x2], dim=1)
28         x = self.up2(x, v)
29         x = self.upsample(x)
30         x = torch.cat([x, x1], dim=1)
31         x = self.up1(x, v)
32         x = self.out(x)
33         return x

```

这样我们就完成了扩散模型使用的神经网络的实现。

4.4 完整代码

```

1  import math
2  import torch
3  import torchvision
4  import matplotlib.pyplot as plt
5  from torchvision import transforms
6  from torch.utils.data import DataLoader
7  from torch.optim import Adam
8  import torch.nn.functional as F
9  from torch import nn

```

```
10 from tqdm import tqdm
11
12
13 img_size = 28
14 batch_size = 128
15 num_timesteps = 1000
16 epochs = 10
17 lr = 1e-3
18 device = "cuda"
19
20
21 def show_images(images, rows=2, cols=10):
22     fig = plt.figure(figsize=(cols, rows))
23     i = 0
24     for r in range(rows):
25         for c in range(cols):
26             fig.add_subplot(rows, cols, i + 1)
27             plt.imshow(images[i], cmap='gray')
28             plt.axis('off')
29             i += 1
30     plt.show()
31
32 # 位置编码部分
33 def _pos_encoding(time_idx, output_dim, device='cpu'):
34     t, D = time_idx, output_dim
35     v = torch.zeros(D, device=device)
36
37     i = torch.arange(0, D, device=device)
38     div_term = torch.exp(i / D * math.log(10000))
39
40     v[0::2] = torch.sin(t / div_term[0::2])
41     v[1::2] = torch.cos(t / div_term[1::2])
42     return v
43
44 def pos_encoding(timesteps, output_dim, device='cpu'):
45     batch_size = len(timesteps)
46     device = timesteps.device
47     v = torch.zeros(batch_size, output_dim, device=device)
48     for i in range(batch_size):
49         v[i] = _pos_encoding(timesteps[i], output_dim, device)
50     return v
51
52 class ConvBlock(nn.Module):
53     def __init__(self, in_ch, out_ch, time_embed_dim):
54         super().__init__()
55         self.convs = nn.Sequential(
56             nn.Conv2d(in_ch, out_ch, 3, padding=1),
57             nn.BatchNorm2d(out_ch),
58             nn.ReLU(),
59             nn.Conv2d(out_ch, out_ch, 3, padding=1),
```



```

60         nn.BatchNorm2d(out_ch),
61         nn.ReLU()
62     )
63     self.mlp = nn.Sequential(
64         nn.Linear(time_embed_dim, in_ch),
65         nn.ReLU(),
66         nn.Linear(in_ch, in_ch)
67     )
68
69     def forward(self, x, v):
70         N, C, _, _ = x.shape
71         v = self.mlp(v)
72         v = v.view(N, C, 1, 1)
73         y = self.convs(x + v)
74         return y
75
76     class UNet(nn.Module):
77         def __init__(self, in_ch=1, time_embed_dim=100):
78             super().__init__()
79             self.time_embed_dim = time_embed_dim
80
81             self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
82             self.down2 = ConvBlock(64, 128, time_embed_dim)
83             self.bot1 = ConvBlock(128, 256, time_embed_dim)
84             self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
85             self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
86             self.out = nn.Conv2d(64, in_ch, 1)
87
88             self.maxpool = nn.MaxPool2d(2)
89             self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
90
91         def forward(self, x, timesteps):
92             v = pos_encoding(timesteps, self.time_embed_dim, x.device)
93
94             x1 = self.down1(x, v)
95             x = self.maxpool(x1)
96             x2 = self.down2(x, v)
97             x = self.maxpool(x2)
98
99             x = self.bot1(x, v)
100
101             x = self.upsample(x)
102             x = torch.cat([x, x2], dim=1)
103             x = self.up2(x, v)
104             x = self.upsample(x)
105             x = torch.cat([x, x1], dim=1)
106             x = self.up1(x, v)
107             x = self.out(x)
108             return x
109

```

```

110
111 class Diffuser:
112     def __init__(self,
113         num_timesteps=1000, # 时间步T=1000
114         beta_start=0.0001, # 方差的起始值 $\beta_0 = 0.0001$ 
115         beta_end=0.02, # 方差的最終值 $\beta_T = 0.02$ 
116         device="cpu"
117     ):
118         self.num_timesteps = num_timesteps
119         self.device = device
120         # 方差调度计划
121         self.betas = torch.linspace(
122             beta_start,
123             beta_end,
124             num_timesteps,
125             device=device
126         )
127         #  $\alpha_t = 1 - \beta_t$ 
128         self.alphas = 1 - self.betas
129         #  $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1$ 
130         self.alpha_bars = torch.cumprod(self.alphas, dim=0)
131
132     def add_noise(self, x_0, t):
133         T = self.num_timesteps
134         assert (t >= 1).all() and (t <= T).all()
135
136         t_idx = t - 1 # alpha_bars[0] is for t=1
137         alpha_bar = self.alpha_bars[t_idx] # (N,)
138         N = alpha_bar.size(0)
139         alpha_bar = alpha_bar.view(N, 1, 1, 1) # (N, 1, 1, 1)
140
141         noise = torch.randn_like(x_0, device=self.device)
142         x_t = torch.sqrt(alpha_bar) * x_0 \
143             + torch.sqrt(1 - alpha_bar) * noise
144         return x_t, noise
145
146     def denoise(self, model, x, t):
147         """去除一步噪声, x为时间步t的带噪声图片 $x_t$ """
148         T = self.num_timesteps
149         assert (t >= 1).all() and (t <= T).all()
150
151         t_idx = t - 1 # alphas[0] is for t=1
152         alpha = self.alphas[t_idx]
153         alpha_bar = self.alpha_bars[t_idx]
154         alpha_bar_prev = self.alpha_bars[t_idx-1]
155
156         N = alpha.size(0)
157         alpha = alpha.view(N, 1, 1, 1)
158         alpha_bar = alpha_bar.view(N, 1, 1, 1)
159         alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)

```

```

160
161     model.eval()
162     with torch.no_grad():
163         eps = model(x, t)
164     model.train()
165
166     noise = torch.randn_like(x, device=self.device)
167     noise[t == 1] = 0 # no noise at t=1
168
169     mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) \
170         / torch.sqrt(alpha)
171     std = torch.sqrt(
172         (1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar)
173     )
174     #  $x_{t-1}$ 
175     return mu + noise * std
176
177     def reverse_to_img(self, x):
178         x = x * 255
179         x = x.clamp(0, 255)
180         x = x.to(torch.uint8)
181         x = x.cpu()
182         to_pil = transforms.ToPILImage()
183         return to_pil(x)
184
185     def sample(self, model, x_shape=(20, 1, 28, 28)):
186         """从纯噪声图片 $x_{1000}$ 反向扩散出 $x_0$ """
187         batch_size = x_shape[0]
188         # 采样一张白噪声图片 $x_{1000}$ 出来
189         x = torch.randn(x_shape, device=self.device)
190         # for t = T, T-1, ..., 0
191         for i in tqdm(range(self.num_timesteps, 0, -1)):
192             t = torch.tensor(
193                 [i] * batch_size,
194                 device=self.device,
195                 dtype=torch.long
196             )
197             # 一步去噪,  $x_t \rightarrow x_{t-1}$ 
198             x = self.denoise(model, x, t)
199
200         images = [
201             self.reverse_to_img(x[i])
202             for i in range(batch_size)
203         ]
204         return images
205
206
207     preprocess = transforms.ToTensor()
208     dataset = torchvision.datasets.MNIST(
209         root=" ../datasets",

```

```

210     download=True,
211     transform=preprocess
212 )
213 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
214
215 diffuser = Diffuser(num_timesteps, device=device)
216 model = UNet()
217 model.to(device)
218 optimizer = Adam(model.parameters(), lr=lr)
219
220 losses = []
221 for epoch in range(epochs):
222     loss_sum = 0.0
223     cnt = 0
224
225     # 每个 epoch 都生成采样的图像 =====
226     images = diffuser.sample(model)
227     show_images(images)
228     # =====
229
230     for images, labels in tqdm(dataloader):
231         optimizer.zero_grad()
232         x = images.to(device)
233         # 随机取一个时间步
234         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
235         # x_noisy是 $x_t$ , noise是添加的真正的噪声
236         x_noisy, noise = diffuser.add_noise(x, t)
237         # 模型根据 $x_t$ 和时间步 $t$ , 预测给 $x_t$ 添加的噪声
238         noise_pred = model(x_noisy, t)
239         # 添加的真实噪声和预测噪声之间进行均方误差计算
240         loss = F.mse_loss(noise, noise_pred)
241
242         loss.backward()
243         optimizer.step()
244
245         loss_sum += loss.item()
246         cnt += 1
247
248     loss_avg = loss_sum / cnt
249     losses.append(loss_avg)
250     print(f"Epoch {epoch} | Loss: {loss_avg}")
251
252 # 画出损失
253 plt.plot(losses)
254 plt.xlabel("Epoch")
255 plt.ylabel("Loss")
256 plt.show()
257
258 # 从完全噪声的图片反向扩散
259 images = diffuser.sample(model)

```

```
260 show_images(images)
```


5. 扩散模型背后的数学理论

由于我们正在创建一个生成模型，我们希望找到真实图像的分布。

给定一个目标和未知的概率分布，可以构造一条马尔可夫链，用来近似该目标概率分布。执行近似时包含的步骤越多，样本的分布就越接近真实分布。

当当前决策仅取决于你所在的位置而非起点时，过程就是马尔可夫过程。存在概率转移矩阵，我们可以构建一个链结构，称为马尔可夫链。

$$\begin{aligned} p(x) \propto e^{-f(x)} &\Leftrightarrow \log p(x) = -f(x) + \text{常数} & (1) \\ \therefore x_t &= x_{t-1} - \eta_t \nabla f(x_{t-1}) & (2) \\ x_{t+1} &= x_t - \frac{\varepsilon}{2} \nabla f(x_t) + \sqrt{\varepsilon} \mathcal{N}(0, I) & (3) \end{aligned} \tag{5.1}$$

图 5.1 郎之万动力学

郎之万动力学是目前最流行的 MCMC（马尔可夫蒙特卡洛采样）方法之一。如果我们有一个概率分布 $p(x)$ 且是马尔可夫分布，则它属于指数分布类。这在图 5.1(1)中给出，其中 $f(x)$ 是某个使得该方程成立的函数。假设我们没有直接从该分布中采样的方法。假设我们只能知道函数在每一点的梯度 $\nabla f(x)$ 。如果是这样，我们可以应用梯度下降来求出分布的模式。这由图 5.1(2)给出，其中 η 是时间步 t 的学习率。如果我们能找到分布的模式，那么我们就找到密度中具有峰值的区域。由于在我们的训练数据中，会有很多“好”图像，这意味着去到峰值时，会得到这些好图像。郎之万采样是对该梯度下降算法的有趣修改，由图 5.1(3)给出。这里我们在梯度下降公式的每个步骤中添加一个噪声项。

一个问题是，如果想加快过程，可能会选择 ε 的值过高，这可能导致上述方程(3)不稳定并发散。所以，如果我们能提供某种形式的保证，保证更新规则会收敛，那就更好了。

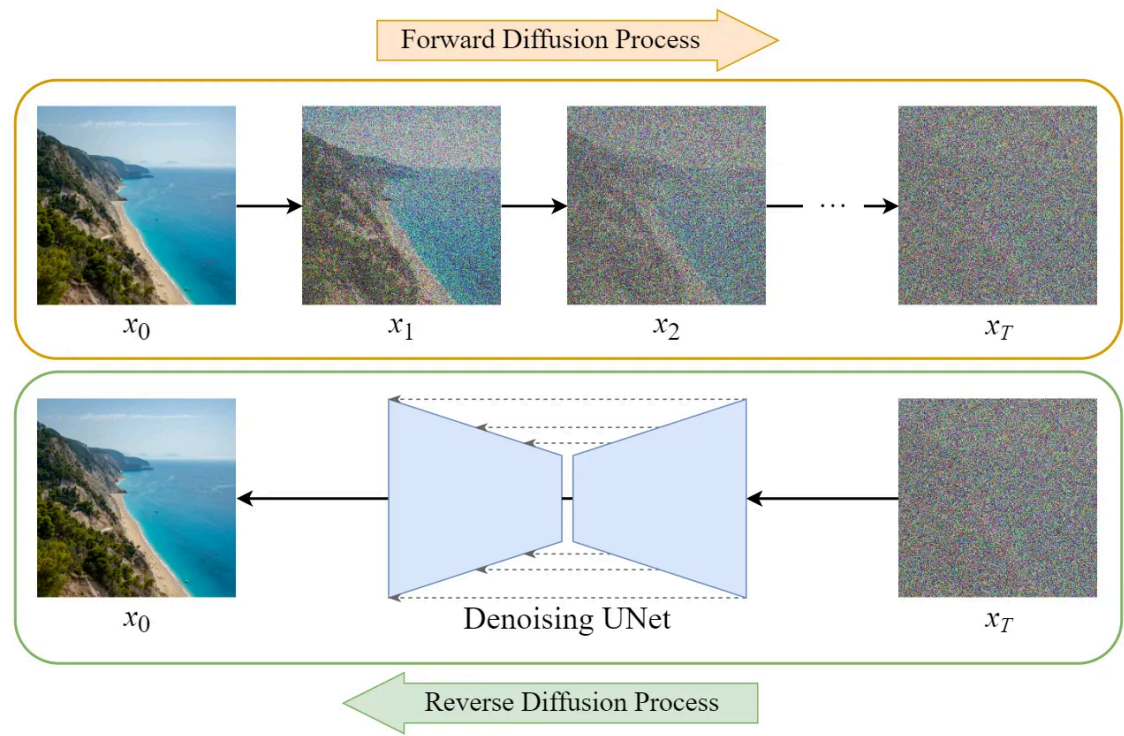


图 5.2 正向扩散和逆向扩散

扩散模型的训练可以分为两部分：

- 1. 正向扩散过程 → 给图像添加噪声。
- 2. 反向扩散过程 → 从图像中去除噪声。

5.1 正向扩散过程

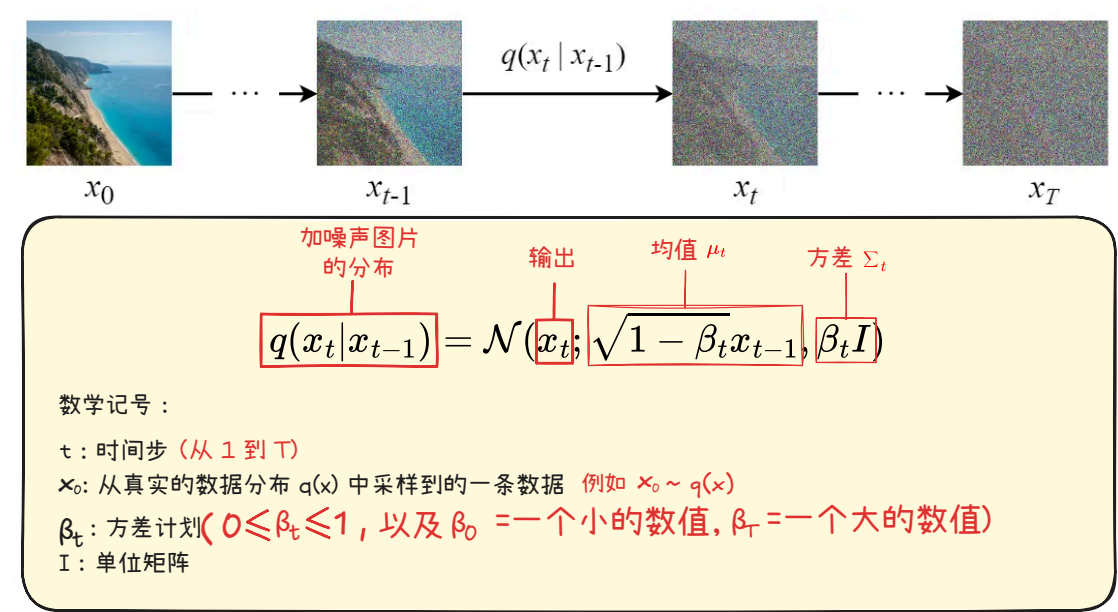


图 5.3 正向扩散公式图解

前向扩散过程逐步将高斯噪声添加到输入图像 x_0 中，总共会有 T 步。该过程将产生一系列带噪声的图像样本 $x_1, \dots, x_T$ 。

当 $T \rightarrow \infty$ 时，最终结果将变成完全噪声图像，就像从各向同性的高斯分布中采样出来的噪声一样。

但我们不需要设计一种算法来迭代地向图像中添加噪声，而是可以使用闭式公式（解析解）在特定的时间步长 t 直接对噪声图像进行采样。

前向扩散的闭式公式（解析解）

可以使用**重参数化技巧**推导出解析形式的采样公式。

首先，如果 $z \sim \mathcal{N}(\mu, \sigma^2)$ 的话，那么有下面的结论

$$z = \mu + \sigma \varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0, 1) \quad (5.2)$$

利用这个技巧，我们可以将采样图像 x_t 表示如下：

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1} \quad (5.3)$$

然后我们可以递归地展开它来得到闭合形式的公式：

我们先来写一些前置结论

$$\begin{aligned} \varepsilon_0, \dots, \varepsilon_{t-2}, \varepsilon_{t-1} &\sim \mathcal{N}(0, I) \\ \bar{\varepsilon}_0, \dots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} &\sim \mathcal{N}(0, I) \\ \varepsilon &\sim \mathcal{N}(0, I) \\ \alpha_t &= 1 - \beta_t \\ \bar{\alpha}_t &= \prod_{i=1}^t \alpha_i \end{aligned} \quad (5.4)$$

然后我们开始推导

$$\begin{aligned} x_t &= \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1} \\ &= \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_{t-1} \\ &= \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-2}) + \sqrt{1 - \alpha_t} \varepsilon_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-2} + \sqrt{1 - \alpha_t} \varepsilon_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\varepsilon}_{t-2} \\ &\vdots \\ &= \sqrt{\alpha_t \alpha_{t-1} \dots \alpha_1} x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \dots \alpha_1} \varepsilon \\ &= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \end{aligned} \quad (5.5)$$

how?

🔥 提示

所有 ε 都是 **i.i.d.**（独立同分布）标准正态随机变量。

使用不同的符号和下标来区分它们非常重要，因为它们是独立的，并且在采样后它们的值可能会有所不同。

但是我们如何从第 4 行跳到第 5 行呢？

也就是公式中的 **how?** 怎么解决呢?

$$\begin{aligned}
 x_t &= \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1} & \varepsilon_0, \dots, \varepsilon_{t-2}, \varepsilon_{t-1} &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t} \boxed{x_{t-1}} + \sqrt{1 - \alpha_t} \varepsilon_{t-1} & \bar{\varepsilon}_0, \dots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-2} \right) + \sqrt{1 - \alpha_t} \varepsilon_{t-1} & \varepsilon &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \underbrace{\sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-2}}_{X+Y} + \underbrace{\sqrt{1 - \alpha_t} \varepsilon_{t-1}}_{X+Y} & \alpha_t &= 1 - \beta_t \\
 & & \bar{\alpha}_t &= \prod_{i=1}^t \alpha_i
 \end{aligned}$$

重参数化技巧 对应的正态分布

$$\begin{aligned}
 0 + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-2} &\text{-----} \rightarrow X \sim \mathcal{N}(0, \alpha_t (1 - \alpha_{t-1}) I) \\
 0 + \sqrt{1 - \alpha_t} \varepsilon_{t-1} &\text{-----} \rightarrow Y \sim \mathcal{N}(0, (1 - \alpha_t) I)
 \end{aligned}$$

因为

If $X \sim \mathcal{N}(\mu_X, \sigma_X^2)$ $Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$

$Z = X + Y$

所以 $Z \sim \mathcal{N}(\mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$

$$\begin{aligned}
 Z &\sim \mathcal{N}(0, \boxed{\sigma_X^2 + \sigma_Y^2}) \\
 Z &\sim \mathcal{N}(0, \boxed{(1 - \alpha_t \alpha_{t-1}) I})
 \end{aligned}$$

⋮

重参数化技巧

↓

$$0 + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\varepsilon}_{t-2}$$

$$\begin{aligned}
 \sigma_X^2 + \sigma_Y^2 &= \alpha_t (1 - \alpha_{t-1}) + (1 - \alpha_t) \\
 &= \cancel{\alpha_t} - \alpha_t \alpha_{t-1} + 1 - \cancel{\alpha_t} \\
 &= 1 - \alpha_t \alpha_{t-1}
 \end{aligned}$$

$$\begin{aligned}
 &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \boxed{\sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\varepsilon}_{t-2}} \\
 &\vdots \\
 &= \sqrt{\alpha_t \alpha_{t-1} \dots \alpha_1} x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \dots \alpha_1} \varepsilon \\
 &= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon
 \end{aligned}$$

图 5.4 重参数技巧的使用

我们用 X 和 Y 来表示这两个项。它们可以被视为来自两个不同正态分布的样本。即

$$X \sim \mathcal{N}(0, \alpha_t (1 - \alpha_{t-1}) I) \quad (5.6)$$

和

$$Y \sim \mathcal{N}(0, (1 - \alpha_t) I) \quad (5.7)$$

回想一下，两个正态分布（独立）随机变量的和也是正态分布的。即，如果 $Z = X + Y$ ，那么有下面的公式

$$Z \sim \mathcal{N}(0, \sigma_X^2 + \sigma_Y^2) \quad (5.8)$$

因此，我们可以将它们合并在一起，并以重新参数化的形式表示合并后的正态分布。这就是我们将这两个项合并的方法。

重复这些步骤将为我们提供以下仅取决于输入图像 x_0 的公式：

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\varepsilon \quad (5.9)$$

现在我们可以使用此公式在任何时间步骤直接对 x_t 进行采样，这使得前向过程更快。

5.2 逆向扩散过程

前向过程的联合概率分布是

$$q(x_0x_1\cdots x_T) = q(x_0)q(x_1|x_0)\cdots q(x_T|x_{T-1}) \quad (5.10)$$

反向过程的联合概率分布是

$$q(x_0x_1\cdots x_T) = q(x_T)q(x_{T-1}|x_T)\cdots q(x_0|x_1) \quad (5.11)$$

联合概率的乘法公式是

$$q(x_t, x_{t-1}) = q(x_t|x_{t-1})q(x_{t-1}) = q(x_{t-1}|x_t)q(x_t) \quad (5.12)$$

贝叶斯公式

$$q(x_{t-1}|x_t) = \frac{q(x_t|x_{t-1})q(x_{t-1})}{q(x_t)} \quad (5.13)$$

贝叶斯推断

$$q(x_{t-1}|x_t) \propto q(x_t|x_{t-1})q(x_{t-1}) \quad (5.14)$$

贝叶斯推断为什么要忽略掉分母 $q(x_t)$ 呢？因为我们要计算的是，在 **固定** x_t 的情况下，求 x_{t-1} 的概率，所以 $q(x_t)$ 是个常数。可以忽略掉。

现在的问题是， $q(x_t|x_{t-1})$ 我们已经知道公式了，但 $q(x_{t-1})$ 我们不知道怎么计算。

$$q(x_{t-1}|x_t) \propto \overset{\text{已经知道公式了}}{q(x_t|x_{t-1})} \overset{\text{算不出来了}}{q(x_{t-1})} \quad (5.15)$$

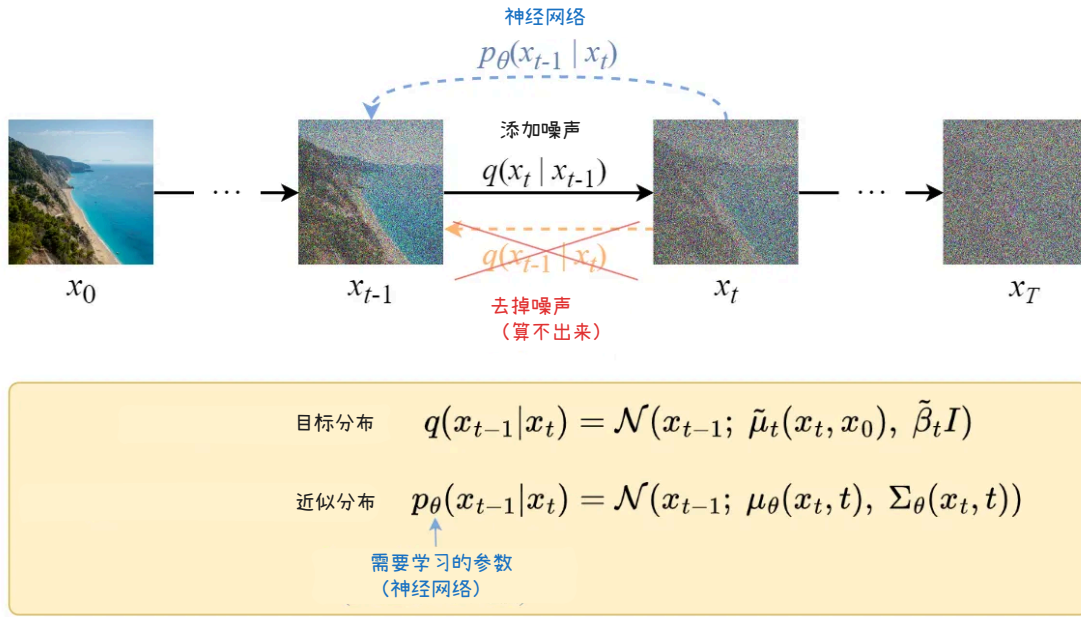


图 5.5 去噪的解析解无法计算

Theorem 5.1 — Feller. 如果前向过程的马尔可夫链的转移概率分布 $q(x_t | x_{t-1})$ 是高斯分布，且 β_t 充分小，那么反向过程的马尔可夫链的转移概率分布 $q(x_{t-1} | x_t)$ 也是高斯分布。

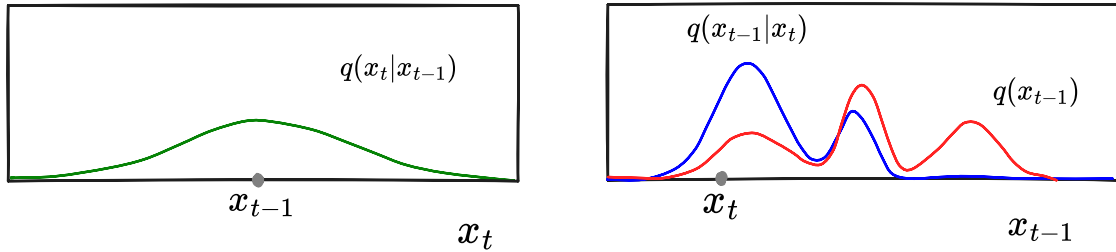


图 5.6 通过贝叶斯定理可以对反向分布 $q(x_{t-1} | x_t)$ 进行推断。右图上的红色曲线显示了用 3 个高斯分布混合表示的边缘分布 $q(x_{t-1})$ ，而左图显示了以 x_{t-1} 为中心的高斯前向噪声过程 $q(x_t | x_{t-1})$ 作为 x_t 的分布。通过将它们相乘并进行归一化，我们得到了蓝色曲线所示的对于特定选择的 x_t 的分布 $q(x_{t-1} | x_t)$ 。因为左图的分布相对较宽，对应着较大的方差 β_t ，所以分布 $q(x_{t-1} | x_t)$ 具有复杂的多峰结构。

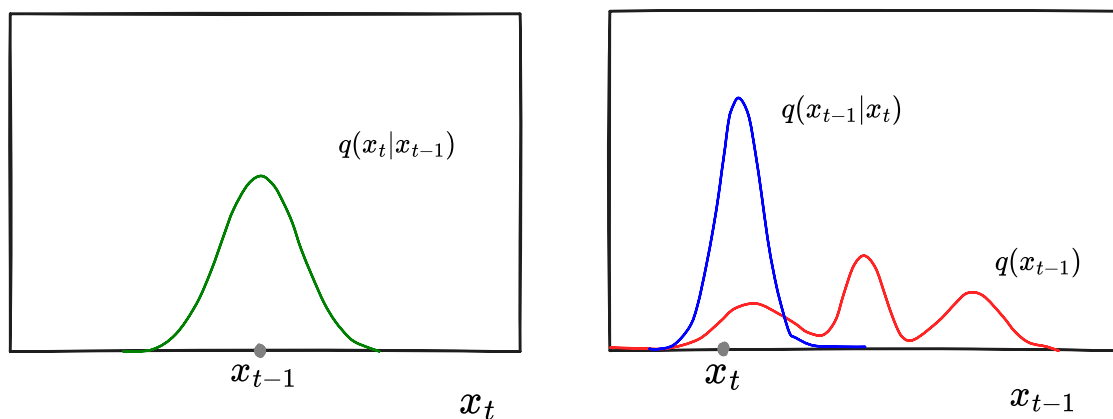


图 5.7 左图中的高斯分布 $q(x_t|x_{t-1})$ 的方差 β_t 要小的多。我们可以看到右图中相应的分布 $q(x_t|x_{t-1})$ （蓝色）接近高斯分布，具有与 $q(x_t|x_{t-1})$ 类似的方差

我们可以将边缘分布 $q(x_{t-1})$ 写成以下形式

$$q(x_{t-1}) = \int q(x_{t-1}|\mathbf{x}_0)q(\mathbf{x}_0)d\mathbf{x}_0 \quad (5.16)$$

其中 $q(x_{t-1}|\mathbf{x}_0)$ 可以由前向扩散的闭式解搞定。然而，上式的分布是难以处理的。因为我们必须对未知的数据密度 $q(\mathbf{x}_0)$ 进行积分。

如果我们使用训练数据集的样本来近似积分，我们得到一个复杂的分布，表示为高斯混合分布。

🔥 提示

无论多么复杂的概率分布，都可以由多个高斯分布加权相加来拟合。概率分布的万能逼近定理。

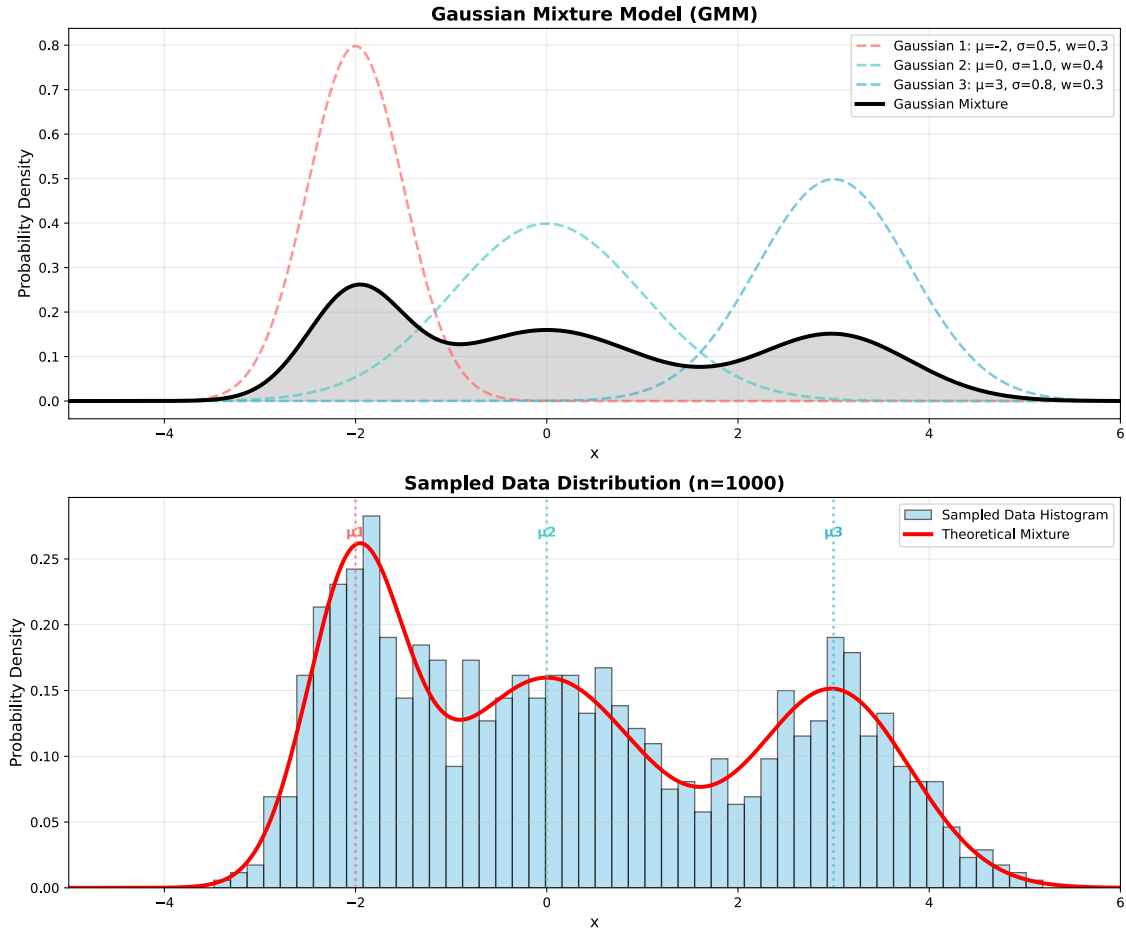


图 5.8 高斯混合模型 (GMM)

相反，我们考虑条件分布的反向分布，条件是数据向量 $\mathbf{x}_0$ ，定义为 $q(x_{t-1}|x_t, \mathbf{x}_0)$ ，我们很快将看到它实际上是一个简单的高斯分布。直观上讲，这是合理的，因为对于给定的噪声图像，很难猜测是哪个更低噪声的图像产生了它，而如果我们还知道起始图像，那么问题就变得容易得多。

我们可以使用贝叶斯定理计算这个条件分布：

$$q(x_{t-1}|x_t, \mathbf{x}_0) = \frac{q(x_t|x_{t-1}, \mathbf{x}_0)q(x_{t-1}|\mathbf{x}_0)}{q(x_t|\mathbf{x}_0)} \quad (5.17)$$

现在我们利用前向过程的马尔可夫性质来写

$$q(x_t|x_{t-1}, \mathbf{x}_0) = q(x_t|x_{t-1}) \quad (5.18)$$

引理 5.1 第 t 步到第 $t-1$ 步在第0步的条件下的转移概率分布 $q(x_{t-1}|x_t, \mathbf{x}_0)$ 是以下高斯分布：

$$q(x_{t-1}|x_t, \mathbf{x}_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}) \quad (5.19)$$

其均值是

$$\tilde{\mu}_t(x_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0 \quad (5.20)$$

方差的系数是

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \quad (5.21)$$

证明 使用贝叶斯定理，利用前向过程的转移概率是高斯分布的性质，可以推导出反向过程的条件转移概率分布也是高斯分布，并得到其概率密度函数。

$$\begin{aligned} & q(x_{t-1}|x_t, \mathbf{x}_0) \\ &= \frac{q(x_{t-1}|\mathbf{x}_0)q(x_t|x_{t-1}, \mathbf{x}_0)}{q(x_t|\mathbf{x}_0)} \\ &= \frac{q(x_{t-1}|\mathbf{x}_0)q(x_t|x_{t-1})}{q(x_t|\mathbf{x}_0)} \\ &= \frac{\mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbf{I})\mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)\mathbf{I})}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})} \\ &\propto \exp \left\{ -\frac{1}{2} \left[\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{1 - \bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right\} \\ &= \exp \left\{ -\frac{1}{2} \left[\frac{x_t^2 - 2\sqrt{\alpha_t}x_{t-1} + \alpha_t x_{t-1}^2}{\beta_t} + \frac{x_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}x_{t-1}\mathbf{x}_0 + \bar{\alpha}_{t-1}\mathbf{x}_0^2}{1 - \bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right\} \\ &= \exp \left\{ -\frac{1}{2} \left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) x_{t-1}^2 - 2 \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \right) x_{t-1} + C(x_t, \mathbf{x}_0) \right] \right\} \end{aligned} \quad (5.22)$$

$C(x_t, \mathbf{x}_0)$ 是对于 x_{t-1} 的常数。高斯分布的方差系数是

$$\begin{aligned} \bar{\beta}_t &= \frac{1}{\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}} \\ &= \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \end{aligned} \quad (5.23)$$

均值是

$$\begin{aligned} \tilde{\mu}_t(x_t, \mathbf{x}_0) &= \frac{\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0}{\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}} \\ &= \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} (1 - \alpha_t) \\ &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0 \end{aligned} \quad (5.24)$$

证明完毕

根据前向扩散的闭式解

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\varepsilon \quad (5.25)$$

可以得到

$$x_0 = \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1 - \alpha_t}\varepsilon) \quad (5.26)$$

代入 $\tilde{\mu}_t(x_t, \mathbf{x}_0)$ 可以得到其只依赖 x_t 的形式:

$$\begin{aligned} \tilde{\mu}_t(x_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1 - \alpha_{t-1})}{1 - \alpha_t}x_t + \frac{\sqrt{\alpha_{t-1}}(1 - \alpha_t)}{1 - \alpha_t} \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1 - \alpha_t}\varepsilon) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}}\varepsilon \right) \end{aligned} \quad (5.27)$$

假设前向过程和反向过程都是马尔可夫链, 前向过程的转移概率分布是高斯分布。在此基础上可以得出, 反向过程的转移概率分布也是高斯分布, 并且能够求出第 t 步前向的转移概率分布 $q(x_t|x_0)$ 和反向的条件转移概率分布 $q(x_{t-1}|x_t, x_0)$ 。

前向过程表示一步步加噪的随机过程, 由超参数 $\beta_t(1, 2, \dots, T)$ 控制, 是事先确定的。反向过程表示一步步去噪的随机过程, 由学习得到的神经网络控制。

DDPM用神经网络表示反向的转移概率分布 $p_\theta(x_{t-1}|x_t), t = 1, 2, \dots, T$ 。由定理可知, 这个转移概率分布是高斯分布

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}, \boldsymbol{\mu}_\theta(x_t, t), \boldsymbol{\Sigma}_\theta(x_t, t)) \quad (5.28)$$

其均值和方差由神经网络决定。神经网络的输入是样本 x_t 和步数 t , 输出是均值 $\boldsymbol{\mu}_\theta$ 和方差 $\boldsymbol{\Sigma}_\theta$, 参数是 θ 。

反向过程的联合概率分布表示为

$$p(x_0x_1 \cdots x_T) = p(x_T)\mathcal{N}(x_{T-1}; \boldsymbol{\mu}_\theta(x_T, T), \boldsymbol{\Sigma}_\theta(x_T, T)) \cdots \mathcal{N}(x_1; \boldsymbol{\mu}_\theta(x_1, 1), \boldsymbol{\Sigma}_\theta(x_1, 1)) \quad (5.29)$$

假设每一步的协方差矩阵是对角阵。

$$\boldsymbol{\Sigma}_\theta(x_t, t) = \sigma_t^2 \mathbf{I} \quad (5.30)$$

前向过程从第 $t-1$ 步到第 t 步根据 $q(x_t|x_{t-1})$ 采样, 由 x_{t-1} 得到 x_t 。反向过程从第 t 步到第 $t-1$ 步根据 $p_\theta(x_{t-1}|x_t)$ 采样, 由 x_t 得到 x_{t-1} 。原理上 $p_\theta(x_{t-1}|x_t)$ 应该近似转移概率分布 $q(x_{t-1}|x_t)$ 。但 $q(x_{t-1}|x_t)$ 难以计算, DDPM实际上使用 $p_\theta(x_{t-1}|x_t)$ 近似条件转移概率分布 $q(x_{t-1}|x_t, x_0)$ 。

我们的目的是希望 $p_\theta(\mathbf{x}_0)$ 能够逼近理想中的那个 $p(\mathbf{x}_0)$ 。

Theorem 5.2 — 詹生不等式. $\mathbb{E}_{q(x)}[\log f(x)] \leq \log \mathbb{E}_{q(x)}[f(x)]$

推导证据下界: ELBO。



ELBO

ELBO: Evidence Lower Bound, 证据下界, 变分下界

$$\begin{aligned}
& \log p_\theta(x_0) \\
&= \log \int p_\theta(x_0 x_1 \cdots x_T) dx_1 dx_2 \cdots dx_T \\
&= \log \int p_\theta(x_0 x_1 \cdots x_T) \frac{q(x_1 x_2 \cdots x_T | x_0)}{q(x_1 x_2 \cdots x_T | x_0)} dx_1 dx_2 \cdots dx_T \\
&= \log \mathbb{E}_{q(x_1 x_2 \cdots x_T | x_0)} \left[\frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right] \quad (\text{显示为期望值}) \\
&\geq \mathbb{E}_{q(x_1 x_2 \cdots x_T | x_0)} \left[\log \frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right] \quad (\text{詹生不等式})
\end{aligned} \tag{5.31}$$

那么就有

$$\underbrace{\mathbb{E}_{q(x_0)} \log p_\theta(x_0)}_{\text{证据}} \geq \underbrace{\mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right]}_{\text{证据下界}} \tag{5.32}$$

直接最大化左边是不可行的，也就是不可求导，无法优化。所以 DDPM 通过最大化变分下界来优化目标。所以损失函数得到一下结果，其中用 $p_\theta(x_{t-1}|x_t)$ 近似 $q(x_{t-1}|x_t, x_0)$ 。

$$\begin{aligned}
& L(\theta) \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{q(x_1 x_2 \cdots x_T | x_0)}{p_\theta(x_0 x_1 \cdots x_T)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{\prod_{t=1}^T q(x_t | x_{t-1})}{p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=1}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \tag{5.33} \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \sum_{t=2}^T \log \frac{q(x_t | x_0)}{q(x_{t-1} | x_0)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_T | x_0)}{q(x_1 | x_0)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{q(x_T | x_0)}{p_\theta(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} - \log p_\theta(x_0 | x_1) \right] \\
&= \mathbb{E}_{q(x_0, x_T)} \left[\frac{q(x_T | x_0)}{p_\theta(x_T)} \right] + \sum_{t=2}^T \mathbb{E}_{q(x_0, x_{t-1}, x_t)} \left[\log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \right] - \mathbb{E}_{q(x_0, x_1)} [\log p_\theta(x_0 | x_1)]
\end{aligned}$$

这里的分布 $q(x_{t-1}|x_t, x_0)$ 的定义如下：

$$\begin{aligned}
q(x_{t-1}|x_t, x_0) &= \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I}) \\
\tilde{\mu}_t(x_t, x_0) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \varepsilon \right)
\end{aligned} \tag{5.34}$$

与之对应，分布 $p_\theta(x_{t-1}|x_t)$ 的定义如下：

$$\begin{aligned}
p_\theta(x_{t-1}|x_t) &= \mathcal{N}(x_{t-1}; \tilde{\mu}_\theta(x_t, t), \sigma_\theta^2 \mathbf{I}) \\
\tilde{\mu}_\theta(x_t, t) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(x_t, t) \right)
\end{aligned} \tag{5.35}$$

假设两个分布 $q(x_{t-1}|x_t, x_0)$ 和 $p_\theta(x_{t-1}|x_t)$ 均值具有相同的形式：方差相同，即设 $\tilde{\beta}_t = \sigma_t^2$ 。因此，神经网络简化为 $\varepsilon_\theta(x_t, t)$ ，其输入是样本 x_t 和步数 t ，输出是噪声 ε ， θ 是参数。

DDPM 的学习实际通过简化的损失函数的最小化进行，训练一个预测噪声的神经网络。

将损失函数展开，第一项损失 L_T 如下：

$$L_T(\theta) = \mathbb{E}_{q(x_0)}[\text{KL}(q(x_T|x_0)||p_\theta(x_T))] \quad (5.36)$$

这个损失是常数，所以对最小化不起作用。

中间各项的损失 $L_{t-1}(t = T, T-1, \dots, 2)$ 如下。通过计算分布 $q(x_{t-1}|x_t, x_0)$ 和 $p_\theta(x_{t-1}|x_t)$ 的 KL 散度的期望得到，期望是针对分布 $q(x_0)$ 和 $p(\varepsilon)$ 的。

$$\begin{aligned} L_{t-1}(\theta) &= \mathbb{E}_{q(x_0), p(\varepsilon)}[\text{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))] \\ &= \mathbb{E}_{q(x_0), p(\varepsilon)}\left[\frac{1}{2\sigma^2(t)} \|\mu(x_t, x_0) - \mu_\theta(x_t, t)\|^2\right] \\ &= \mathbb{E}_{q(x_0), p(\varepsilon)}\left[\frac{1}{2\sigma^2(t)} \left\| \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon \right] - \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right] \right\|^2 \right] \quad (5.37) \\ &= \mathbb{E}_{q(x_0), p(\varepsilon)}\left[\frac{1}{2\sigma^2(t)} \frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right] \\ &= \mathbb{E}_{q(x_0), p(\varepsilon)}\left[\frac{1}{2\sigma^2(t)} \frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, t)\|^2\right] \end{aligned}$$

最后一项损失 L_0 如下：

$$L_0(\theta) = \mathbb{E}_{q(x_0), p(\varepsilon_1)}[-\log p_\theta(x_0|x_1)] \quad (5.38)$$

针对损失 $L_{t-1}, t = T, T-1, \dots, 2$ ，忽略系数，只对平方损失部分进行优化。针对损失 L_0 ，假设进行同样的平方损失优化。忽略损失 L_T ，这样得到以下简化的整体损失函数：

$$\begin{aligned} L'(\theta) &= \sum_{t=1}^T \mathbb{E}_{q(x_0), p(\varepsilon_1)} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2] \\ &= \sum_{t=1}^T \mathbb{E}_{q(x_0), p(\varepsilon_1)} [\|\varepsilon - \varepsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, t)\|^2] \quad (5.39) \end{aligned}$$

神经网络 $\varepsilon_\theta(x_t, t)$ 预测的是前向过程第 t 步的高斯噪声，其直观的解释是，这样的神经网络也能对反向过程的第 t 步 ($t = 1, 2, \dots, T$) 进行有效的去噪。

反向过程的第 t 步到第 $t-1$ 步的转移概率分布，可以利用学习得到的神经网络 $\varepsilon_\theta(x_t, t)$ 计算。使用随机变量表示形式：

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t \varepsilon \quad (5.40)$$

上面的式子称为 DDPM 反向过程的迭代公式，用于数据生成。设每一步的方差系数与对应的前向过程的方差系数相同， $\sigma_t = \sqrt{\beta_t}$ 。

6. 条件扩散模型

我们之前对数据 x 的概率 $p(x)$ 进行了建模。但在实用层面，我们更希望对条件概率 $p(x|y)$ 进行建模（其中 y 表示条件）。如果能成功建立 $p(x|y)$ 的模型，那么就可以通过条件 y 控制想生成的 x 。

条件 y 可以是文本、图像或标签等。如果 y 是一张低分辨率的图像，那么可以考虑将其变换为高分辨率的图像。这就是被称为超分辨率成像（**super-resolution imaging**）的技术。使用扩散模型进行超分辨率处理的模型包括级联扩散模型（**cascaded diffusion model**）等。

本节将以 y 作为标签的例子进行说明。具体来说，我们将创建一个模型，在给定 MNIST 数据的数字标签后，该模型能够生成与该标签相对应的图像。

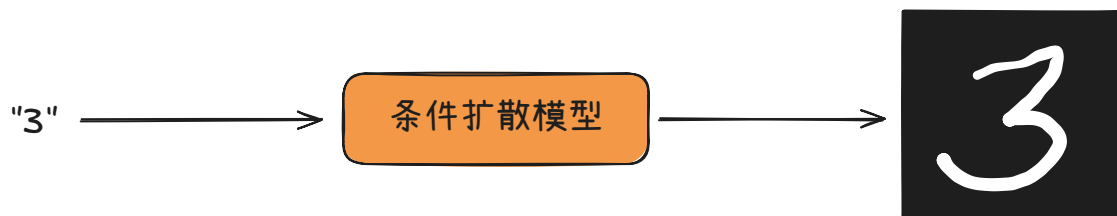


图 6.1 本章将要实现的条件扩散模型

6.1 向扩散模型添加条件

首先，我们从复习开始。在扩散模型中，我们可以选择在何处使用神经网络。例如，可以考虑使用神经网络对 $\mu_{\theta}(x_t, t)$ 和 $\epsilon_{\theta}(x_t, t)$ 进行建模。

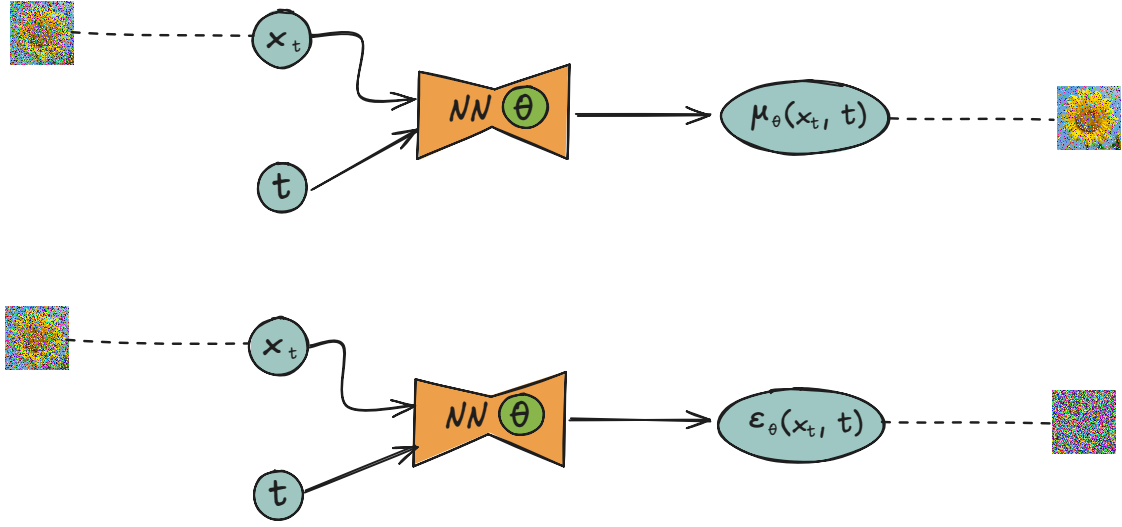


图 6.2 预测时刻 $t-1$ 的图像（正态分布的均值向量）的模型（上）和预测添加到 x_t 中的噪声的模型（下）

下面思考使用神经网络对 $\mu_\theta(x_t, t)$ 建模的情况。在这种情况下， $p_\theta(x_{t-1}|x_t)$ 的数学式如下所示。

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_q^2(t)\mathbf{I}) \quad (6.1)$$

数据 x_0 的概率 $p_\theta(x_0)$ 的数学式如下所示：

$$\begin{aligned} p_\theta(x_0) &= \int p_\theta(x_0, x_1, \dots, x_T) dx_1 \cdots dx_T && \text{(概率的边际化)} \\ &= \int p_\theta(x_0|x_1) \cdots p_\theta(x_{T-1}|x_T) p(x_T) dx_1 \cdots dx_T && \text{(马尔可夫性质)} \end{aligned} \quad (6.2)$$

这里有 $p(x_T) = \mathcal{N}(x_T; \mathbf{0}, \mathbf{I})$ 。

为了得到条件扩散模型，我们要建模的对象是 $p_\theta(x_0|y)$ ，而不是 $p_\theta(x_0)$ 。实现这个目标的最简单的方法是在每个时刻的 $p_\theta(x_{t-1}|x_t)$ 中添加条件 y 。具体的数学式如下所示：

$$p_\theta(x_0|y) = \int p_\theta(x_0|x_1, y) \cdots p_\theta(x_{T-1}|x_T, y) p(x_T) dx_1 dx_T \quad (6.3)$$

此时的 $p_\theta(x_{t-1}|x_t, y)$ 的数学式如下所示：

$$p_\theta(x_{t-1}|x_t, y) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, y), \sigma_q^2(t)\mathbf{I}) \quad (6.4)$$

在常规的扩散模型中， $\mu_\theta(x_t, t)$ 通常由神经网络进行建模。而在条件扩散模型中，如 $\mu_\theta(x_t, t, y)$ 所示，会额外添加参数 y 。换言之，通过向神经网络中添加 y ，可以使模型“进化”为条件扩散模型。同样，对于预测噪声的神经网络，也可以通过将参数 y 添加到 $\epsilon_\theta(x_t, t)$ 中，使其变为 $\epsilon_\theta(x_t, t, y)$ 来实现相应的功能（参见图 6.3）。

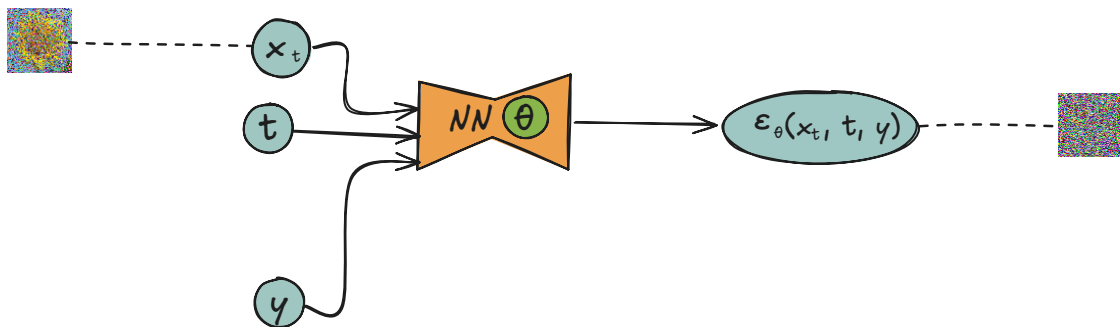


图 6.3 条件扩散模型的神经网络

6.2 条件扩散模型的实现

我们已经使用神经网络实现了 $\varepsilon_{\theta}(x_t, t)$ 。 $\varepsilon_{\theta}(x_t, t)$ 的参数 t 是整数，可以通过正弦位置编码变换为向量。而新添加的条件 y 是标签，也是整数。这个整数 y 可以通过嵌入层 (embedding layer) 变换为向量。具体来说， y 被变换成向量，然后与变换后的 t 向量相加。

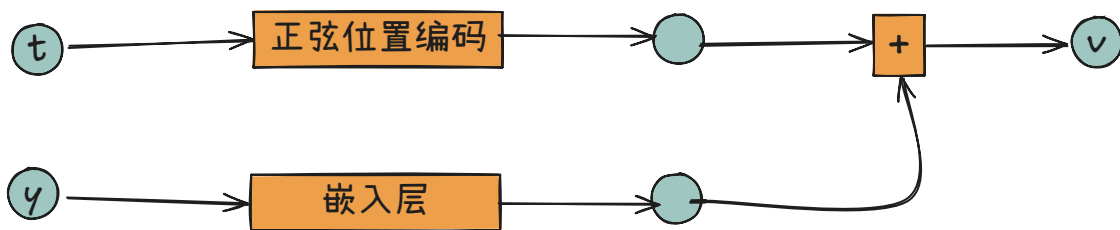


图 6.4 在扩散模型中添加嵌入层

提示

嵌入层的初始值被设置为随机值，然后通过训练进行优化。这样就可以训练得到与每个标签相对应的匹配任务的向量。而由于与时刻 t 相关的特定任务的训练要素很少，因此我们使用了一种称为正弦位置编码的固定向量变换的方法。

以下是代码示例。这里我们在上一章中实现的 UNet 类中添加了名为 UNetCond 类的代码，类名中的 Cond 是 Conditional (条件) 的缩写。

```
1 class UNetCond(nn.Module):
2     def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
3         super().__init__()
4         self.time_embed_dim = time_embed_dim
5
6         self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
7         self.down2 = ConvBlock(64, 128, time_embed_dim)
8         self.bot1 = ConvBlock(128, 256, time_embed_dim)
9         self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
10        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
11        self.out = nn.Conv2d(64, in_ch, 1)
12
13        self.maxpool = nn.MaxPool2d(2)
```

Python

```

14         self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")
15
16         # ① 处理标签的嵌入层
17         if num_labels is not None:
18             self.label_emb = nn.Embedding(num_labels, time_embed_dim)
19
20     def forward(self, x, timesteps, labels=None):
21         t = pos_encoding(timesteps, self.time_embed_dim)
22
23         # ② 标签的处理
24         if labels is not None:
25             t += self.label_emb(labels)
26
27         x1 = self.down1(x, t)
28         x = self.maxpool(x1)
29         x2 = self.down2(x, t)
30         x = self.maxpool(x2)
31
32         x = self.bot1(x, t)
33
34         x = self.upsample(x)
35         x = torch.cat([x, x2], dim=1)
36         x = self.up2(x, t)
37         x = self.upsample(x)
38         x = torch.cat([x, x1], dim=1)
39         x = self.up1(x, t)
40         x = self.out(x)
41         return x

```

代码的主要修改有两处。在代码①处，我们准备了处理标签的嵌入层（`nn.Embedding`）。具体来说，就是通过 `nn.Embedding(num_labels, time_embed_dim)` 将共计 `num_labels` 个不同整数变换为 `time_embed_dim` 维的向量。在代码②处，通过 `nn.Embedding` 层处理标签，然后再加到变换后的时间数据的向量中。

接下来是 `Diffuser` 类。对于 `Diffuser` 类，修改生成数据的方法。下面只展示了修改部分的代码。

```

1  class Diffuser: Python
2      def denoise(self, model, x, t, labels):
3          # ... (省略)
4          with torch.no_grad():
5              eps = model(x, t, labels) # 同时提供labels
6              # ...
7
8      def sample(self, model, x_shape=(20, 1, 28, 28), labels=None):
9          # ...
10         if labels is None:
11             labels = torch.randint(0, 10, (len(x),), device=self.device)
12         # ...
13         for i in tqdm(range(self.num_timesteps, 0, -1)):
14             t = torch.tensor([i] * batch_size, device=self.device,
15                             dtype=torch.long)
16             x = self.denoise(model, x, t, labels) # 还提供labels

```

```

17         # ...
18         return images, labels

```

修改的部分还向模型提供了标签。具体来说，标签是在生成数据和去噪过程中提供的。最后是训练的代码。

```

1  diffuser = Diffuser(num_timesteps, device=device)
2  model = UNetCond(num_labels=10) ❶
3  model.to(device)
4  optimizer = Adam(model.parameters(), lr=lr)
5
6  losses = []
7  for epoch in range(epochs):
8      loss_sum = 0.0
9      cnt = 0
10
11     for images, labels in tqdm(dataloader):
12         optimizer.zero_grad()
13         x = images.to(device) ❷
14         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
15
16         x_noisy, noise = diffuser.add_noise(x, t)
17         noise_pred = model(x_noisy, t, labels) ❸
18         loss = F.mse_loss(noise, noise_pred)
19
20         loss.backward()
21         optimizer.step()
22
23         loss_sum += loss.item()
24         cnt += 1

```

训练代码的修改内容如下所示。

❶ UNetCond(num_labels=10)

创建一个具有 10 个分类的条件扩散模型。

❷ labels.to(device)

在 device 上准备标签数据。

❸ model(x_noisy, t, labels)

向模型提供 labels 进行训练。

以上就是条件扩散模型的实现。现在运行代码。经过 10 轮训练后，最终生成了下图所示的图像。

虽然仍有改进的余地，但生成的图像是符合条件的。在下一节中，我们将探讨进一步改进这个条件扩散模型的方法。

完整代码如下：

```

1  import math
2  import torch
3  import torchvision
4  import matplotlib.pyplot as plt
5  from torchvision import transforms
6  from torch.utils.data import DataLoader
7  from torch.optim import Adam
8  import torch.nn.functional as F
9  from torch import nn

```

```

10 from tqdm import tqdm
11
12
13 img_size = 28
14 batch_size = 128
15 num_timesteps = 1000
16 epochs = 10
17 lr = 1e-3
18 device = 'cuda' if torch.cuda.is_available() else 'cpu'
19
20
21 def show_images(images, labels=None, rows=2, cols=10):
22     fig = plt.figure(figsize=(cols, rows))
23     i = 0
24     for r in range(rows):
25         for c in range(cols):
26             ax = fig.add_subplot(rows, cols, i + 1)
27             plt.imshow(images[i], cmap='gray')
28             if labels is not None:
29                 ax.set_xlabel(labels[i].item())
30                 ax.get_xaxis().set_ticks([])
31                 ax.get_yaxis().set_ticks([])
32             i += 1
33     plt.tight_layout()
34     plt.show()
35
36 def _pos_encoding(time_idx, output_dim, device='cpu'):
37     t, D = time_idx, output_dim
38     v = torch.zeros(D, device=device)
39
40     i = torch.arange(0, D, device=device)
41     div_term = torch.exp(i / D * math.log(10000))
42
43     v[0::2] = torch.sin(t / div_term[0::2])
44     v[1::2] = torch.cos(t / div_term[1::2])
45     return v
46
47 def pos_encoding(timesteps, output_dim, device='cpu'):
48     batch_size = len(timesteps)
49     device = timesteps.device
50     v = torch.zeros(batch_size, output_dim, device=device)
51     for i in range(batch_size):
52         v[i] = _pos_encoding(timesteps[i], output_dim, device)
53     return v
54
55 class ConvBlock(nn.Module):
56     def __init__(self, in_ch, out_ch, time_embed_dim):
57         super().__init__()
58         self.convs = nn.Sequential(
59             nn.Conv2d(in_ch, out_ch, 3, padding=1),

```



```

60         nn.BatchNorm2d(out_ch),
61         nn.ReLU(),
62         nn.Conv2d(out_ch, out_ch, 3, padding=1),
63         nn.BatchNorm2d(out_ch),
64         nn.ReLU()
65     )
66     self.mlp = nn.Sequential(
67         nn.Linear(time_embed_dim, in_ch),
68         nn.ReLU(),
69         nn.Linear(in_ch, in_ch)
70     )
71
72     def forward(self, x, v):
73         N, C, _, _ = x.shape
74         v = self.mlp(v)
75         v = v.view(N, C, 1, 1)
76         y = self.convs(x + v)
77         return y
78
79     class UNetCond(nn.Module):
80         def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
81             super().__init__()
82             self.time_embed_dim = time_embed_dim
83
84             self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
85             self.down2 = ConvBlock(64, 128, time_embed_dim)
86             self.bot1 = ConvBlock(128, 256, time_embed_dim)
87             self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
88             self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
89             self.out = nn.Conv2d(64, in_ch, 1)
90
91             self.maxpool = nn.MaxPool2d(2)
92             self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
93             # 如果条件标签不为None
94             if num_labels is not None:
95                 # 将标签转换为嵌入，嵌入和时间步的嵌入形状相同
96                 self.label_emb = nn.Embedding(num_labels, time_embed_dim)
97
98             def forward(self, x, timesteps, labels=None):
99                 t = pos_encoding(timesteps, self.time_embed_dim)
100                 # 如果标签不为空，将标签转换为嵌入，然后加到时间步的嵌入上面。
101                 if labels is not None:
102                     t += self.label_emb(labels)
103
104                 x1 = self.down1(x, t)
105                 x = self.maxpool(x1)
106                 x2 = self.down2(x, t)
107                 x = self.maxpool(x2)
108
109                 x = self.bot1(x, t)

```

```

110
111     x = self.upsample(x)
112     x = torch.cat([x, x2], dim=1)
113     x = self.up2(x, t)
114     x = self.upsample(x)
115     x = torch.cat([x, x1], dim=1)
116     x = self.up1(x, t)
117     x = self.out(x)
118     return x
119
120
121 class Diffuser:
122     def __init__(
123         self,
124         num_timesteps=1000,
125         beta_start=0.0001,
126         beta_end=0.02,
127         device='cpu'
128     ):
129         self.num_timesteps = num_timesteps
130         self.device = device
131         self.betas = torch.linspace(
132             beta_start,
133             beta_end,
134             num_timesteps,
135             device=device
136         )
137         self.alphas = 1 - self.betas
138         self.alpha_bars = torch.cumprod(self.alphas, dim=0)
139
140     def add_noise(self, x_0, t):
141         T = self.num_timesteps
142         assert (t ≥ 1).all() and (t ≤ T).all()
143
144         t_idx = t - 1 # alpha_bars[0] is for t=1
145         alpha_bar = self.alpha_bars[t_idx] # (N,)
146         alpha_bar = alpha_bar.view(
147             alpha_bar.size(0), 1, 1, 1) # (N, 1, 1, 1)
148
149         noise = torch.randn_like(x_0, device=self.device)
150         x_t = torch.sqrt(alpha_bar) * x_0 \
151             + torch.sqrt(1 - alpha_bar) * noise
152         return x_t, noise
153
154     def denoise(self, model, x, t, labels):
155         T = self.num_timesteps
156         assert (t ≥ 1).all() and (t ≤ T).all()
157
158         t_idx = t - 1 # alphas[0] is for t=1
159         alpha = self.alphas[t_idx]

```

```

160     alpha_bar = self.alpha_bars[t_idx]
161     alpha_bar_prev = self.alpha_bars[t_idx-1]
162
163     N = alpha.size(0)
164     alpha = alpha.view(N, 1, 1, 1)
165     alpha_bar = alpha_bar.view(N, 1, 1, 1)
166     alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
167
168     model.eval()
169     with torch.no_grad():
170         eps = model(x, t, labels) # 添加了标签的嵌入
171     model.train()
172
173     noise = torch.randn_like(x, device=self.device)
174     noise[t == 1] = 0 # no noise at t=1
175
176     mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) \
177           / torch.sqrt(alpha)
178     std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar))
179     return mu + noise * std
180
181     def reverse_to_img(self, x):
182         x = x * 255
183         x = x.clamp(0, 255)
184         x = x.to(torch.uint8)
185         x = x.cpu()
186         to_pil = transforms.ToPILImage()
187         return to_pil(x)
188
189     def sample(self, model, x_shape=(20, 1, 28, 28), labels=None):
190         batch_size = x_shape[0]
191         x = torch.randn(x_shape, device=self.device)
192         if labels is None:
193             labels = torch.randint(0, 10, (len(x),), device=self.device)
194
195         for i in tqdm(range(self.num_timesteps, 0, -1)):
196             t = torch.tensor(
197                 [i] * batch_size,
198                 device=self.device,
199                 dtype=torch.long
200             )
201             # 带着标签去噪
202             x = self.denoise(model, x, t, labels)
203
204         images = [self.reverse_to_img(x[i]) for i in range(batch_size)]
205         return images, labels
206
207
208 preprocess = transforms.ToTensor()
209 dataset = torchvision.datasets.MNIST(

```

```

210     root='./../datasets', download=True, transform=preprocess)
211 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
212
213 diffuser = Diffuser(num_timesteps, device=device)
214 model = UNetCond(num_labels=10)
215 model.to(device)
216 optimizer = Adam(model.parameters(), lr=lr)
217
218 losses = []
219 for epoch in range(epochs):
220     loss_sum = 0.0
221     cnt = 0
222
223     # generate samples every epoch =====
224     images, labels = diffuser.sample(model)
225     show_images(images, labels)
226     # =====
227
228     for images, labels in tqdm(dataloader):
229         optimizer.zero_grad()
230         x = images.to(device)
231         # 使用标签数据
232         labels = labels.to(device)
233         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
234
235         x_noisy, noise = diffuser.add_noise(x, t)
236         # 带着标签预测噪声
237         noise_pred = model(x_noisy, t, labels)
238         loss = F.mse_loss(noise, noise_pred)
239
240         loss.backward()
241         optimizer.step()
242
243         loss_sum += loss.item()
244         cnt += 1
245
246     loss_avg = loss_sum / cnt
247     losses.append(loss_avg)
248     print(f'Epoch {epoch} | Loss: {loss_avg}')
249
250 # plot losses
251 plt.plot(losses)
252 plt.xlabel('Epoch')
253 plt.ylabel('Loss')
254 plt.show()
255
256 # generate samples
257 images, labels = diffuser.sample(model)
258 show_images(images, labels)

```

6.3 得分函数

在上面的内容里面，我们实现了一个简单的条件扩散模型。这个“简单”的意思是我们只是向模型提供了条件而已。因此，模型有可能会轻视我们提供的条件，甚至在最坏的情况下有可能会忽略我们提供的条件。接下来，我们将介绍一种称为“指引”的方法。指引是一种将给定条件纳入模型并给予更多重视的机制。

要理解指引机制，首先需要了解得分函数。

6.3.1 什么是得分函数

在实现扩散模型时，我们使用神经网络对 $\varepsilon_{\theta}(x_t, t)$ 进行了建模。 $\varepsilon_{\theta}(x_t, t)$ 基于 x_t 和 t 来推断噪声 ε 。下图是这个过程的示意图。

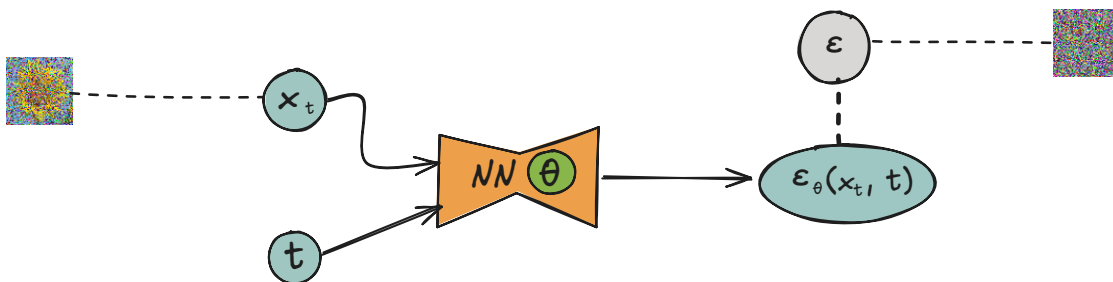


图 6.5 扩散模型中使用的推断噪声 ε 的神经网络

此时有以下与 ε 相关的数学式成立（稍后将给出证明）。

$$\varepsilon \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p(x_t) \quad (6.5)$$

∇ 是表示梯度的符号，读作“那勃乐”（Nabla）。 $\nabla_{x_t} \log p(x_t)$ 是对数似然 $\log p(x_t)$ 对输入数据 x_t 的梯度，称为“得分函数”或“得分”。

提示

$p(x_t)$ 是表示数据 x_t 为“真”的随机密度函数。而 $p_{\theta}(x_t)$ 表示使用参数对真的概率密度函数进行近似的概率密度函数。由于 $\nabla_{x_t} \log p(x_t)$ 和 $\nabla_{x_t} \log p_{\theta}(x_t)$ 表示关于输入 x_t 的梯度，因此称为“得分”。另外，在某些领域，关于参数的梯度（ $\nabla_{\theta} \log p_{\theta}(x_t)$ ）也称为“得分”。在本书中，我们将对输入的梯度称为“得分”。

根据式（6.5）， ε 可以近似表示为 $\nabla_{x_t} \log p(x_t)$ 。值得注意的是， ε 与 $\nabla_{x_t} \log p(x_t)$ 之间只相差负常数倍（ $-\sqrt{1 - \bar{\alpha}_t}$ ）。这说明以 $-\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon$ 来代替 ε 作为训练数据的神经网络也是可行的。

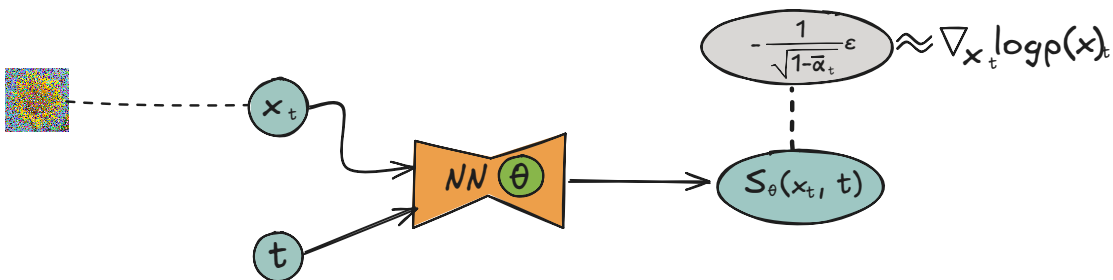


图 6.6 用于推断得分的神经网络

图中的神经网络将得分的近似值 $-\sqrt{1-\bar{\alpha}_t}\varepsilon$ 推断为 $s_\theta(x_t, t)$ 。因此，我们也可以通过推断得分的神经网络来实现扩散模型。

🔥 提示

生成模型中包括对得分进行建模的方法。这些模型统称为基于得分的模型。以对数似然最大化的方式可以推导出各种模型（GMM、VAE、扩散模型等）。这些模型可以被称为基于似然的模型。重要的是，扩散模型也可以作为基于得分的模型推导得出。

6.3.2 式 (6.5) 的证明

现在让我们来证明 式 (6.5) 成立。

$$\varepsilon \approx -\sqrt{1-\bar{\alpha}_t}\nabla_{x_t} \log p(x_t) \quad (6.6)$$

我们先来复习一下。时刻 t 的噪声数据 x_t 可以基于以下正态分布从原始数据 x_0 生成。

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}) \quad (6.7)$$

利用重参数化技巧，可以通过以下式子得到 x_t 的样本。

$$\begin{aligned} \varepsilon &\sim \mathcal{N}(0, \mathbf{I}) \\ x_t &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon \end{aligned} \quad (6.8)$$

接下来要用到的是 Tweedie 公式，这个公式如下所示。

🔥 Tweedie 公式

当基于 $x \sim \mathcal{N}(x; \mu, \Sigma)$ 得到样本 x 时，有以下式子成立。

$$\mathbb{E}[\mu|x] = x + \Sigma \nabla_x \log p(x) \quad (6.9)$$

式子中的 μ 被视为随机变量。左侧的 $\mathbb{E}[\mu|x]$ 是在 x 作为条件下的 μ 的期望值。右侧的 $\nabla_x \log p(x)$ 表示得分。我们将这个 Tweedie 公式应用于式 (6.7)。

🔥 Tweedie 公式的应用

当基于 $x_t \sim \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$ 得到样本 x_t 时，有以下式子成立。

$$\begin{aligned} \mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] &= x_t + (1-\bar{\alpha}_t)\mathbf{I}\nabla_{x_t} \log p(x_t) \\ &= x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \end{aligned} \quad (6.10)$$

然后，利用 式 (6.8) 中的 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ 成立这一事实将式子展开，如下所示。

$$\begin{aligned} \mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] &= x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow \mathbb{E}[x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow \mathbb{E}[x_t|x_t] - \mathbb{E}[\sqrt{1-\bar{\alpha}_t}\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow x_t - \sqrt{1-\bar{\alpha}_t}\mathbb{E}[\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \therefore \mathbb{E}[\varepsilon|x_t] &= (1-\bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \end{aligned} \quad (6.11)$$

期望值 $\mathbb{E}[\varepsilon|x_t]$ 可以用蒙特卡洛方法近似。这里我们用一个采样数据来近似 x_t 。这样就得到了以下数学式。

$$\varepsilon \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p(x_t) \quad (6.12)$$

这样我们就完成了推导。

6.4 分类器指引

在上面的内容中，我们了解到可以通过预测得分的神经网络来实现扩散模型。在本节中，我们将从得分的角度来研究扩散模型，并推导出一种称为指引的方法。指引有两种主要类型：分类器指引 (classifier guidance) 和无分类器指引 (classifier-free guidance)。我们先来介绍分类器指引。

6.4.1 什么是分类器

分类器指引是一种使用分类器指导数据生成的方法。分类器是一个能够对数据进行分类的预训练神经网络，如图所示。

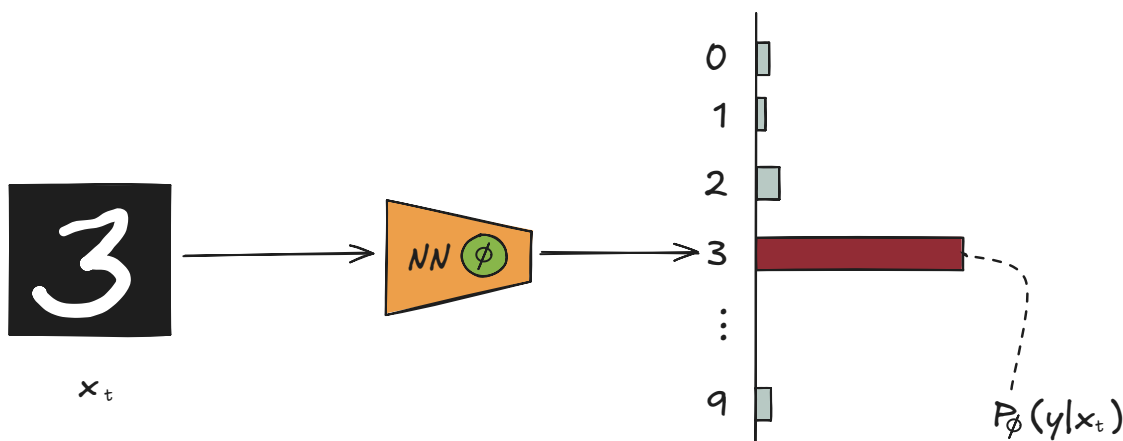


图 6.7 使用预训练神经网络进行分类的示例 (参数为 ϕ)

图中的神经网络将时刻 t 的噪声图像 x_t 作为输入，并输出类别 y 的概率 $p_\phi(y|x_t)$ 。利用这个分类器的神经网络，可以将普通的扩散模型变换为条件扩散模型。在这个过程中，我们需要掌握以下两点。

- 扩散模型可以通过预测得分 $\nabla_{x_t} \log p(x_t)$ 的神经网络来实现。
- (基于同样的原理) 条件扩散模型可以通过预测条件概率得分 $\nabla_{x_t} \log p(x_t|y)$ 的神经网络来实现。

在记住以上两点之后，我们进行式子的变形，推导出分类器指引。

6.4.2 分类器指引的推导

首先，根据贝叶斯定理，将条件概率 $p(x_t|y)$ 表示为下面的数学式。

$$p(x_t|y) = \frac{p(x_t)p(y|x_t)}{p(y)} \quad (6.13)$$

然后，计算关于 x_t 的梯度 (∇_{x_t})。

$$\begin{aligned}
\nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log \left(\frac{p(x_t)p(y|x_t)}{p(y)} \right) \\
&= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t) - \underbrace{\nabla_{x_t} \log p(y)}_0 \\
&= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)
\end{aligned} \tag{6.14}$$

中间的式子中出现 $\nabla_{x_t} \log p(y)$ ，但由于 $p(y)$ 中不包含 x_t ，因此 $\nabla_{x_t} \log p(y) = 0$ 。由此得到的数学式如下所示。

$$\underbrace{\nabla_{x_t} \log p(x_t|y)}_{\text{条件得分}} = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{①得分}} + \underbrace{\nabla_{x_t} \log p(y|x_t)}_{\text{②分类器的对数似然的梯度}} \tag{6.15}$$

根据这个式子可以推导出分类器指引。下面是对式子中的要点的说明。

- ① 此项可以使用预测得分的神经网络 $s_\theta(x_t, t)$ 来计算。
- ② 此项可以使用作为分类器的神经网络来计算。

由此可见,我们可以利用得分和分类器这两个神经网络来表示条件得分(而一旦有了条件得分,便可以实现条件扩散模型)。另外,②的 $\nabla_{x_t} \log p(y|x_t)$ 可以很容易地通过反向传播求出。

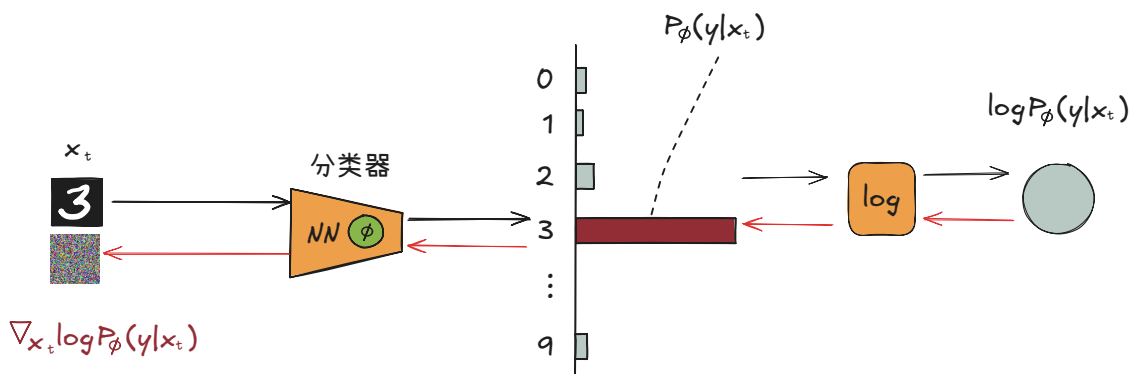


图 6.8 通过反向传播可以求出 $\nabla_{x_t} \log p(y|x_t)$

上图中的 $\nabla_{x_t} \log p(y|x_t)$ 显示了在当前 x_t 下,类别 y 的对数似然增加最快的方向。因此,如果沿着 $\nabla_{x_t} \log p(y|x_t)$ 的方向更新 x_t ,那么更新后的图像被分类为类别 y 的概率会更高。

分类器指引的理念是使用得分和分类器来表示条件得分。通常,我们会向分类器指引中引入权重 γ ,这样就能调整分类器的贡献度。引入后的数学式如下所示。

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \tag{6.16}$$

权重 γ 是人为设定的值(超参数),用于调整分类器向类别 y 方向引导的程度。 γ 值越大,条件 y 的作用就越显著。使用表示推断得分的神经网络 $s_\theta(x_t, t)$ 和表示分类器的 $p_\phi(y|x_t)$,可以将式子改写为如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_\theta(x_t, t) + \gamma \nabla_{x_t} \log p(y|x_t) \tag{6.17}$$

作为参考,上式所执行的处理如下图所示。

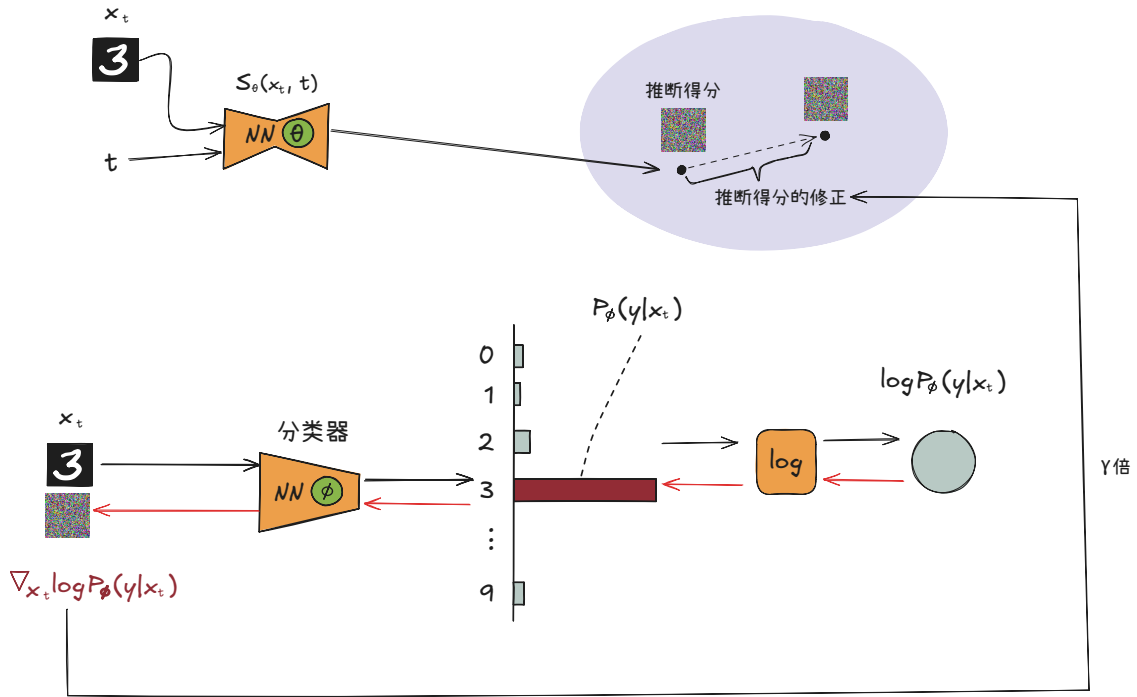


图 6.9 使用了推断得分的神经网络的分类器指引

如图所示，分类器可以与常规的扩散模型（推断得分的模型）相结合，作为条件扩散模型来生成数据。以上就是分类器指引的核心思想。

6.5 无分类器指引

分类器指引可以生成强调条件的数据。然而，分类器指引在实际应用中存在一个问题，即需要单独准备一个分类器。针对分类器指引的这一缺点进行改进的技术是无分类器指引。

6.5.1 无分类器指引的理论知识

顾名思义，无分类器指引就是不需要分类器的指引。它的机制可以通过以下过程得出。

$$\begin{aligned}
 & \nabla_{x_t} \log p(x_t|y) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log \frac{p(x_t|y)p(y)}{p(x_t)} \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \left(\nabla_{x_t} \log p(x_t|y) + \underbrace{\nabla_{x_t} \log p(y)}_0 - \nabla_{x_t} \log p(x_t) \right) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma (\nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t))
 \end{aligned} \tag{6.18}$$

上面我们使用贝叶斯定理进行了式子的展开，最后得到了上面的式子。根据上面的式子，可以推导出无分类器指引。上式的意思是从点 $\nabla_{x_t} \log p(x_t)$ 出发，沿着 $\nabla_{x_t} \log p(x_t|y)$ 的方向前进了 γ 倍的距离。下图是这个式子的示意图。

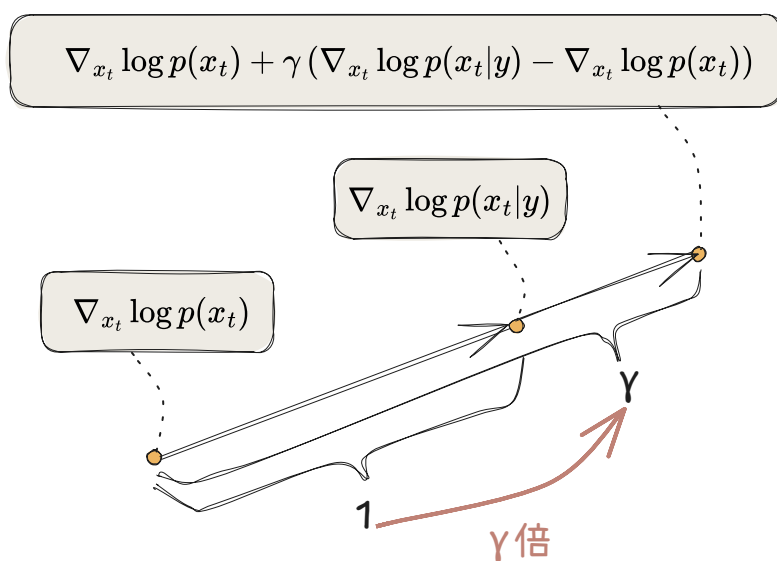


图 6.10 上面式子的可视化

式子中的 $\nabla_{x_t} \log p(x_t)$ 和 $\nabla_{x_t} \log p(x_t|y)$ 分别表示无条件得分和条件得分。它们可以由以下两个神经网络进行推断。

- 无条件得分推断器: $s_{\theta_1}(x_t, t)$
- 条件得分的推断器: $s_{\theta_2}(x_t, t, y)$

不过，准备两个模型很麻烦。更简单的方法是使用一个条件得分推断器 $s_{\theta_2}(x_t, t, y)$ ，并按如下方式建模。

- 无条件得分的推断器: $s_{\theta}(x_t, t, \emptyset)$
- 条件得分的推断器: $s_{\theta}(x_t, t, y)$

这里我们用 $\emptyset$ 表示“无条件”对应的类别。条件 y 可以被嵌入层变换为向量，但当类别为 $\emptyset$ 时，它会被变换为“零向量”，使其不包含任何信息。根据以上信息，无分类器指引的数学式可以表示成如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_{\theta}(x_t, t, \emptyset) + \gamma(s_{\theta}(x_t, t, y) - s_{\theta}(x_t, t, \emptyset)) \quad (6.19)$$

下图展示了这一计算过程。

上面介绍的是推断得分函数的神经网络，但由于得分和噪声之间只有常数倍的差别，因此同样的机制也适用于推断噪声的神经网络。

🔥 提示

在提供文本的图像生成服务中，我们经常会用到一种名为反向提示词（**negative prompt**）的技术。反向提示词是一种指定不希望生成的文本的技术。在上面的公式的 $\emptyset$ 中插入反向提示词即可实现这一技术。

6.5.2 无分类器指引的实现

下面我们通过修改节中的部分代码来实现无分类器指引。先回忆一下前面已经实现的 `UNetCond` 类，然后在这个类中实现 `forward()` 方法，代码如下所示。

```
1 class UNetCond(nn.Module):
```

Python

```

2     #... (省略)
3     def forward(self, x, timesteps, labels=None):
4         t = pos_encoding(timesteps, self.time_embed_dim)
5
6         if labels is not None:
7             t += self.label_emb(labels)
8         #...

```

从上面的代码来看，只有指定了参数 `labels`，才会处理类标签。如果没有指定参数（当 `labels=None` 时），则不做任何处理。这相当于“无条件”的处理。

无分类器指引的训练是在“无条件”和“有条件”两种情况下进行的条件扩散模型的训练。例如，我们可以以一定的比例进行“无条件”的训练，除此之外进行“有条件”的训练。基于这一点，我们可以按如下方式实现。

```

1  for epoch in range(epochs):
2      loss_sum = 0.0
3      cnt = 0
4
5      for images, labels in tqdm(dataloader):
6          optimizer.zero_grad()
7          x = images.to(device)
8          labels = labels.to(device)
9          t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
10
11         # 以10%的概率进行"无条件"的训练
12         if np.random.random() < 0.1:
13             labels = None
14
15         x_noisy, noise = diffuser.add_noise(x, t)
16         noise_pred = model(x_noisy, t, labels)
17         loss = F.mse_loss(noise, noise_pred)
18         loss.backward()
19         optimizer.step()
20
21         loss_sum += loss.item()
22         cnt += 1

```

上面的代码以 10% 的概率进行了“无条件”的训练。对于“无条件”的情况，设置 `labels=None`。最后是 `Diffuser` 类进行去噪处理的代码。

```

1  class Diffuser:
2      #... (省略)
3
4      def denoise(self, model, x, t, labels, gamma):
5          #...
6          with torch.no_grad():
7              eps_cond = model(x, t, labels)
8              eps_uncond = model(x, t)
9              eps = eps_uncond + gamma * (eps_cond - eps_uncond)
10         #...

```

参数 `gamma` 的值越大，就越应该重视条件部分。另外，这里实现的是如下数学式的代码。

$$\varepsilon \approx \varepsilon_{\theta}(x_t, t, \emptyset) + \gamma(\varepsilon_{\theta}(x_t, t, y) - \varepsilon_{\theta}(x_t, t, \emptyset)) \quad (6.20)$$

经过以上修改之后，我们就完成了无分类器指引的实现。这里以 $\gamma = 3$ (gamma=3.0) 声称了数据。

无分类器指引可以通过 γ 来调整条件的重视程度。而且由于无需另外训练分类器，因此在资源和时间方面更为高效。

完整代码如下：

```

1  import math
2  import numpy as np
3  import torch
4  import torchvision
5  import matplotlib.pyplot as plt
6  from torchvision import transforms
7  from torch.utils.data import DataLoader
8  from torch.optim import Adam
9  import torch.nn.functional as F
10 from torch import nn
11 from tqdm import tqdm
12
13
14 img_size = 28
15 batch_size = 128
16 num_timesteps = 1000
17 epochs = 10
18 lr = 1e-3
19 device = 'cuda' if torch.cuda.is_available() else 'cpu'
20
21
22 def show_images(images, labels=None, rows=2, cols=10):
23     fig = plt.figure(figsize=(cols, rows))
24     i = 0
25     for r in range(rows):
26         for c in range(cols):
27             ax = fig.add_subplot(rows, cols, i + 1)
28             plt.imshow(images[i], cmap='gray')
29             if labels is not None:
30                 ax.set_xlabel(labels[i].item())
31                 ax.get_xaxis().set_ticks([])
32                 ax.get_yaxis().set_ticks([])
33             i += 1
34     plt.tight_layout()
35     plt.show()
36
37 def _pos_encoding(time_idx, output_dim, device='cpu'):
38     t, D = time_idx, output_dim
39     v = torch.zeros(D, device=device)
40
41     i = torch.arange(0, D, device=device)
42     div_term = torch.exp(i / D * math.log(10000))
43
44     v[0::2] = torch.sin(t / div_term[0::2])

```

```

45     v[1::2] = torch.cos(t / div_term[1::2])
46     return v
47
48     def pos_encoding(timesteps, output_dim, device='cpu'):
49         batch_size = len(timesteps)
50         device = timesteps.device
51         v = torch.zeros(batch_size, output_dim, device=device)
52         for i in range(batch_size):
53             v[i] = _pos_encoding(timesteps[i], output_dim, device)
54         return v
55
56     class ConvBlock(nn.Module):
57         def __init__(self, in_ch, out_ch, time_embed_dim):
58             super().__init__()
59             self.convs = nn.Sequential(
60                 nn.Conv2d(in_ch, out_ch, 3, padding=1),
61                 nn.BatchNorm2d(out_ch),
62                 nn.ReLU(),
63                 nn.Conv2d(out_ch, out_ch, 3, padding=1),
64                 nn.BatchNorm2d(out_ch),
65                 nn.ReLU()
66             )
67             self.mlp = nn.Sequential(
68                 nn.Linear(time_embed_dim, in_ch),
69                 nn.ReLU(),
70                 nn.Linear(in_ch, in_ch)
71             )
72
73         def forward(self, x, v):
74             N, C, _, _ = x.shape
75             v = self.mlp(v)
76             v = v.view(N, C, 1, 1)
77             y = self.convs(x + v)
78             return y
79
80     class UNetCond(nn.Module):
81         def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
82             super().__init__()
83             self.time_embed_dim = time_embed_dim
84
85             self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
86             self.down2 = ConvBlock(64, 128, time_embed_dim)
87             self.bot1 = ConvBlock(128, 256, time_embed_dim)
88             self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
89             self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
90             self.out = nn.Conv2d(64, in_ch, 1)
91
92             self.maxpool = nn.MaxPool2d(2)
93             self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
94

```

```

95         if num_labels is not None:
96             self.label_emb = nn.Embedding(num_labels, time_embed_dim)
97
98     def forward(self, x, timesteps, labels=None):
99         t = pos_encoding(timesteps, self.time_embed_dim)
100
101         if labels is not None:
102             t += self.label_emb(labels)
103
104         x1 = self.down1(x, t)
105         x = self.maxpool(x1)
106         x2 = self.down2(x, t)
107         x = self.maxpool(x2)
108
109         x = self.bot1(x, t)
110
111         x = self.upsample(x)
112         x = torch.cat([x, x2], dim=1)
113         x = self.up2(x, t)
114         x = self.upsample(x)
115         x = torch.cat([x, x1], dim=1)
116         x = self.up1(x, t)
117         x = self.out(x)
118         return x
119
120
121 class Diffuser:
122     def __init__(self, num_timesteps=1000, beta_start=0.0001, beta_end=0.02,
123                  device='cpu'):
124         self.num_timesteps = num_timesteps
125         self.device = device
126         self.betas = torch.linspace(beta_start, beta_end, num_timesteps,
127                                     device=device)
128         self.alphas = 1 - self.betas
129         self.alpha_bars = torch.cumprod(self.alphas, dim=0)
130
131     def add_noise(self, x_0, t):
132         T = self.num_timesteps
133         assert (t >= 1).all() and (t <= T).all()
134
135         t_idx = t - 1 # alpha_bars[0] is for t=1
136         alpha_bar = self.alpha_bars[t_idx] # (N,)
137         alpha_bar = alpha_bar.view(alpha_bar.size(0), 1, 1, 1) # (N, 1, 1, 1)
138
139         noise = torch.randn_like(x_0, device=self.device)
140         x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar) * noise
141         return x_t, noise
142
143     def denoise(self, model, x, t, labels, gamma):
144         T = self.num_timesteps
145         assert (t >= 1).all() and (t <= T).all()

```

```

144         t_idx = t - 1 # alphas[0] is for t=1
145         alpha = self.alphas[t_idx]
146         alpha_bar = self.alpha_bars[t_idx]
147         alpha_bar_prev = self.alpha_bars[t_idx-1]
148
149
150         N = alpha.size(0)
151         alpha = alpha.view(N, 1, 1, 1)
152         alpha_bar = alpha_bar.view(N, 1, 1, 1)
153         alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
154
155         model.eval()
156         with torch.no_grad():
157             eps = model(x, t, labels)
158             eps_uncond = model(x, t)
159             eps = eps_uncond + gamma * (eps - eps_uncond)
160         model.train()
161
162         noise = torch.randn_like(x, device=self.device)
163         noise[t == 1] = 0 # no noise at t=1
164
165         mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) / torch.sqrt(alpha)
166         std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar))
167         return mu + noise * std
168
169     def reverse_to_img(self, x):
170         x = x * 255
171         x = x.clamp(0, 255)
172         x = x.to(torch.uint8)
173         x = x.cpu()
174         to_pil = transforms.ToPILImage()
175         return to_pil(x)
176
177     def sample(self, model, x_shape=(20, 1, 28, 28), labels=None, gamma=3.0):
178         batch_size = x_shape[0]
179         x = torch.randn(x_shape, device=self.device)
180         if labels is None:
181             labels = torch.randint(0, 10, (len(x),), device=self.device)
182
183         for i in tqdm(range(self.num_timesteps, 0, -1)):
184             t = torch.tensor([i] * batch_size, device=self.device,
185                               dtype=torch.long)
186             x = self.denoise(model, x, t, labels, gamma)
187
188         images = [self.reverse_to_img(x[i]) for i in range(batch_size)]
189         return images, labels
190
191 preprocess = transforms.ToTensor()
192 dataset = torchvision.datasets.MNIST(root='./data', download=True,

```

```

193 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
194
195 diffuser = Diffuser(num_timesteps, device=device)
196 model = UNetCond(num_labels=10)
197 model.to(device)
198 optimizer = Adam(model.parameters(), lr=lr)
199
200 losses = []
201 for epoch in range(epochs):
202     loss_sum = 0.0
203     cnt = 0
204
205     # generate samples every epoch =====
206     #images, labels = diffuser.sample(model)
207     #show_images(images, labels)
208     # =====
209
210     for images, labels in tqdm(dataloader):
211         optimizer.zero_grad()
212         x = images.to(device)
213         labels = labels.to(device)
214         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
215
216         if np.random.random() < 0.1:
217             labels = None
218
219         x_noisy, noise = diffuser.add_noise(x, t)
220         noise_pred = model(x_noisy, t, labels)
221         loss = F.mse_loss(noise, noise_pred)
222
223         loss.backward()
224         optimizer.step()
225
226         loss_sum += loss.item()
227         cnt += 1
228
229     loss_avg = loss_sum / cnt
230     losses.append(loss_avg)
231     print(f'Epoch {epoch} | Loss: {loss_avg}')
232
233 # plot losses
234 plt.plot(losses)
235 plt.xlabel('Epoch')
236 plt.ylabel('Loss')
237 plt.show()
238
239 # generate samples
240 images, labels = diffuser.sample(model)
241 show_images(images, labels)

```


6.6 Stable Diffusion

到目前为止，我们已经实现了处理 MNIST 的小型扩散模型以及小型条件扩散模型。当然，现代的扩散模型规模相当庞大，而且经过了多项改进。下面是一些进一步改进扩散模型的指引。这里我们介绍一个著名的模型——Stable Diffusion，它能够生成下图所示的高清图像。另外，它的代码和预训练的权重数据是公开的，任何人都可以运行这些代码。



图 6.11 Stable Diffusion 生成的高清图像



7. DALL-E2（文生图）

🔥 DALL-E2

- 输入：文本提示词
- 输出：图像

🔥 训练 DALL-E2 的数据集

图文对！

去噪扩散概率模型（DDPM）是一种流行的生成式人工智能模型，由 Ho 等人于 2020 年提出，并由 Nichol 等人于 2021 年对其进行了改进。这些模型背后的基本思想是，在正向扩散过程中将噪声添加到图像中，以便训练模型预测在反向扩散过程中应在特定时间步去除的噪声。在对图像进行采样（去噪）时，需要从纯噪声的图像开始，并在每个时间步迭代地去除模型预测的噪声，直到获得最终图像。

为了让 DDPM 生成多种类型的图像，同时仍允许用户选择所需的图像类型，模型需要根据某些输入进行条件调节（条件扩散模型）。Ramesh 等人提出了一种名为 unCLIP 的条件调节方法，该方法已用于 OpenAI 的 DALL-E 2 模型。在 Ramesh 等人描述的方法中，输入的图片标题或者描述首先被传递到一个先验网络（prior model），该网络将使用经过训练的 CLIP 模型来获取 CLIP 文本嵌入。然后，仅解码器架构的 Transformer 使用这些文本嵌入来生成可能的 CLIP 图像嵌入。先验网络生成的 CLIP 图像嵌入将被解码器网络（由 UNet 模型组成）用于条件调节所创建的图像。在本章中，我们将使用此过程构建一个简单的扩散模型。

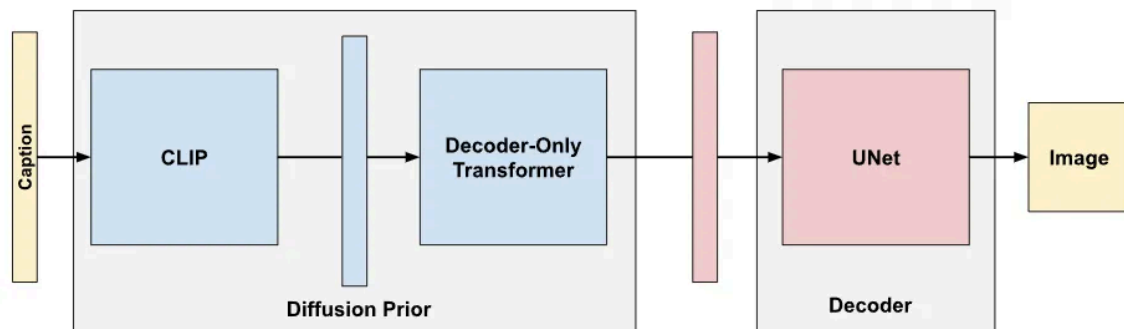


图 7.1 DALL-E2 模型整体架构

🔥 DALL-E2 需要训练 3 个模型

1. CLIP 模型
2. 先验模型: Decoder-Only Transformer
3. 生成图像的解码器网络: UNet

7.1 CLIP

为了从文本创建扩散图像 (文生图), 我们将使用 CLIP 模型中的嵌入。从 CLIP 获得的文本嵌入用于调节先验模型, 使其扩散相应的图像嵌入。然后, 这些图像嵌入用于调节解码器模型, 用来指导解码器生成对应的图像。

🔥 提示

CLIP 模型训练完毕后, 冻结起来供后续使用。

7.2 先验模型 (Prior Model)

⚡ 危险

训练先验模型时, 需要使用上一节训练的 CLIP 模型, CLIP 模型必须冻结!

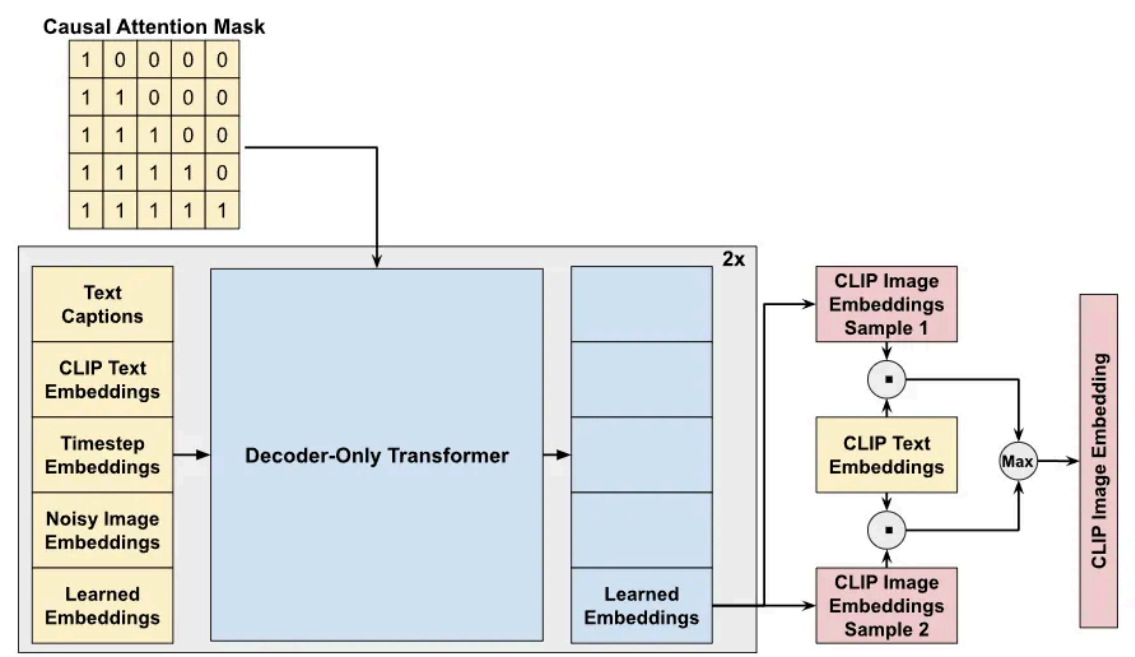


图 7.2 先验模型架构图

先验模型的作用：**根据文本标题预测 CLIP 图像嵌入**。也可以放弃先验模型，而以 CLIP 文本嵌入为条件，而不是用先验模型生成的 CLIP 图像嵌入作为条件，但使用先验模型的效果最佳。

先验模型是一个仅解码器架构的 Transformer。

7.2.1 先验模型的训练

- 要训练一个模型，我们必须搞清楚模型的输入和输出（预测目标）。
- 训练先验模型时，我们手上有的是：图文对和一个训练好的 CLIP 模型。
- 先验模型的输入有 6 个：
- 1. **Text Captions**: 图文对中的“文本”。
 - 2. **CLIP Text Embeddings**: 图文对中的“文本”经过上一节训练好的冻结的 CLIP 模型生成的 CLIP 文本嵌入。
 - 3. **Timestep Embeddings**: 随机选取的时间步 t 的嵌入。
 - 4. **Noisy Image Embeddings**: 添加了 t 步噪声的 CLIP 图像嵌入。这个 CLIP 图像嵌入是图文对中的“图像”经过上一节训练好的冻结的 CLIP 模型生成的 CLIP 图像嵌入。
 - 5. **Learned Embeddings**: 一个随机初始化的可学习的嵌入。
 - 6. **Causal Attention Mask**: 因果注意力掩码。因为我们要训练的是一个 Decoder-Only Transformer 架构的模型，所以需要使用因果注意力掩码。

⚡ 危险

Noisy Image Embeddings 是针对 CLIP 图片嵌入添加噪声，而不是针对图片添加噪声！

先验模型的输出（预测目标）：

- 图文对中的“图像”经过上一节训练好的冻结的 CLIP 模型生成的 CLIP 图像嵌入。

训练细节：

1. 6 个输入经过 Decoder-Only Transformer 之后会输出 5 个张量，我们取最后一个，也就是图中蓝色的 **Learned Embeddings**。
2. **Learned Embeddings** 会送入一个 MLP，输出的就是预测的 CLIP 图片嵌入。
3. **Learned Embeddings** 会送入两次 MLP，生成两个预测的 CLIP 图片嵌入。
4. 两个预测的 CLIP 图片嵌入分别和 CLIP 文本嵌入计算余弦相似度，然后选取最相似的一个预测。
5. 使用余弦相似度更大的那个预测出的 CLIP 图片嵌入和预测目标（真正的 CLIP 图片嵌入）计算损失，并反向传播更新网络。

7.2.2 先验模型的推理

提示

在推理时，我们手上只有：

1. 自己编写的提示词
2. 训练好的 CLIP 模型
3. 训练好的先验模型

先验模型的输入有 6 个：

1. **Text Captions**：自己编写的文本提示词。
2. **CLIP Text Embeddings**：“自己编写的文本提示词”经过上一节训练好的冻结的 CLIP 模型生成的 CLIP 文本嵌入。
3. **Timestep Embeddings**：最大时间步 $T = 1000$ 的时间步嵌入。
4. **Noisy Image Embeddings**：纯噪声构成的嵌入（和图片嵌入的形状相同，从高斯分布中采样得到）。
5. **Learned Embeddings**：训练好的参数（先验模型训练完之后就在模型中了）。
6. **Causal Attention Mask**：因果注意力掩码。

提示

所以只有 **Text Captions** 是我们自己编写的文本提示词，其它输入都是模型或者程序自动生成的。

7.2.3 关键代码解释

7.2.3.1 余弦调度的方差计划



图 7.3 线性调度方差和余弦调度方差的对比

对于方差调度，在 Ho 等人的原始 DDPM 论文中，采用了线性调度。虽然该调度在高分辨率图像上效果良好，但当图像较小（64x64 或更小）时，前向过程最终会产生过多的噪声。为了解决这个问题，Nichol 等人建议使用余弦调度，其公式如下所示。

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}$$

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}$$

$$f(t) = \cos \left(\left(\frac{t}{T} + s \right) \cdot \frac{\pi}{2} \right)^2$$
(7.1)

这些值被剪裁为小于或等于 0.999，以防止在 $t = T$ 附近出现奇点。因为我们使用包含低分辨率图像的 FashionMNIST 数据集，所以我们将使用余弦方差计划。

综合起来，前向扩散过程的代码看起来是这样的：

```

1  # Gets elements from indicies and makes sure output has a certain
   dimension
2  def extract_and_expand(x, idx, shape):
3      return x[idx].reshape(idx.shape[0], *((1, ) * (len(shape) - 1)))
4
5  # 返回方差调度计划
6  def get_beta_schedule(schedule, max_time, s=0.008):
7      if schedule == "linear":
8          scale = 1000 / max_time
9          betas = torch.linspace(1e-4 * scale, 0.02 * scale, max_time)
10     elif schedule == "cosine":
11         t = torch.linspace(0, max_time, max_time + 1)
12         a_bars = torch.cos(
13             (((t / max_time) + s) / (1 + s)) * (np.pi / 2)
14         ) ** 2
15         a_bars = a_bars / a_bars[0]
16         betas = 1 - (a_bars[1:] / a_bars[:-1])
17         betas = torch.clamp(betas, min=0, max=0.999)
18     else:
19         Exception("Beta schedule not implemented.")
20
21     return betas
22
23 def get_schedule_values(config):
24     schedule_values = {}
25     schedule_values["betas"] = get_beta_schedule(
26         config.prior.schedule,
27         config.prior.max_time
28     ).to(config.device)
29     schedule_values["alphas"] = 1.0 - schedule_values["betas"]
30     schedule_values["alpha_bars"] = torch.cumprod(
31         schedule_values["alphas"],
32         axis = 0
33     )
34     schedule_values["sqrt_recip_alphas"] = torch.sqrt(
35         1.0 / schedule_values["alphas"]
36     )
37     schedule_values["sqrt_alpha_bars"] = torch.sqrt(
38         schedule_values["alpha_bars"]
39     )
40     schedule_values["sqrt_one_minus_alpha_bars"] = torch.sqrt(

```

```

40         1.0 - schedule_values["alpha_bars"]
41     )
42     schedule_values["alpha_bars_prev"] = torch.cat((
43         torch.ones(1, device=config.device),
44         schedule_values["alpha_bars"][:-1])
45     )
46     schedule_values["sigma"] = schedule_values["betas"] \
47         * (1.0 - schedule_values["alpha_bars_prev"]) \
48         / (1.0 - schedule_values["alpha_bars"])
49     return schedule_values
50
51 # 获取x_t, 前向扩散过程
52 def forward_diffusion(x_0, schedule_values, t):
53     # 噪声和图片的尺寸一样
54     noise = torch.randn_like(x_0)
55     sqrt_alpha_bars = extract_and_expand(
56         schedule_values["sqrt_alpha_bars"], t, x_0.shape)
57     sqrt_one_minus_alpha_bars = extract_and_expand(
58         schedule_values["sqrt_one_minus_alpha_bars"], t, x_0.shape)
59     x_noisy = (sqrt_alpha_bars * x_0) \
60         + (sqrt_one_minus_alpha_bars * noise)
61     # 返回x_t, 和添加的噪声
62     return x_noisy, noise

```

7.2.3.2 时间步嵌入

时间步嵌入是扩散的重要组成部分。这是因为不同时间步的图像具有不同的噪声量。为了在我们的模型中利用这些信息，我们将使用正弦位置编码。这些位置编码与 Transformer 中常用的位置编码相同。主要区别在于，我们的输入时间步很可能不会按顺序排列，并且包含所有可能的时间步，因此我们只需要获取与输入时间步对应的位置编码。

```

1  class SinusoidalPositionalEncodings(nn.Module):
2      def __init__(
3          self,
4          max_seq_length,    # 最大序列长度
5          width              # 模型的宽度（嵌入的维度d_model）
6      ):
7          super().__init__()
8          # Create positional encodings
9          pe = torch.zeros(max_seq_length, width)
10         for pos in range(max_seq_length):
11             for i in range(width):
12                 if i % 2 == 0:
13                     pe[pos][i] = np.sin(pos/(10000**(i/width)))
14                 else:
15                     pe[pos][i] = np.cos(pos/(10000**((i-1)/width)))
16
17         self.register_buffer('pe', pe)
18
19     def forward(self, x):
20         # Get positional encodings corresponding to inputted timesteps

```



```

21         x = self.pe[x]
22         return x

```

然后，这些时间编码通过 MLP 来进一步捕获时间信息。

```

1  self.time_mlp = nn.Sequential(Python
2      SinusoidalPositionalEmbedding(
3          config.decoder.max_time,
4          config.decoder.model_channels
5      ),
6      nn.Linear(
7          config.decoder.model_channels,
8          config.decoder.cond_channels
9      ),
10     nn.SiLU(),
11     nn.Linear(
12         config.decoder.cond_channels,
13         config.decoder.cond_channels
14     )
15 )

```

有多种方法可以调节时间步嵌入的输入，这些方法在本章的 UNet：残差块部分中进行了描述。

7.2.3.3 冻结模型

先验模型首先将图像的文本标题作为输入，然后获取 CLIP 文本嵌入和图像嵌入。加载 CLIP 模型时，所有层都应冻结，并将模式设置为 eval。

```

1  def freeze_model(model, set_eval=True):Python
2      if set_eval:
3          model.eval()
4
5      for param in model.parameters():
6          param.requires_grad = False
7
8      # Constructor
9      self.clip = CLIP(config).to(config.device)
10     self.clip.load_state_dict(torch.load(
11         config.clip.model_location,
12         map_location=config.device
13     ))
14     # 冻结clip模型
15     freeze_model(self.clip)
16
17     # Forward
18     image_embeddings = self.clip.image_encoder(
19         images
20     ) # (B, C, H, W) → (B, latent_dim)
21     text_embeddings = self.clip.text_encoder(
22         captions,
23         mask=masks
24     ) # (B, text_seq_length) → (B, latent_dim)

```

7.2.3.4 加噪声的 CLIP 图片嵌入

然后, 它会获取批次中每个图像的随机时间步, 并使用它们从前向扩散中获取噪声 CLIP 图像嵌入。

⚡ 危险

不是对图片加噪声, 而是对 CLIP 图片嵌入加噪声!

```

1  # Forward
2  # 随机采样一个时间步
3  timesteps = torch.randint(
4      0, self.config.prior.max_time, (images.shape[0],)) # (B, )
5
6  noisy_image_embedding, _ = forward_diffusion(
7      image_embeddings, # 对图片的嵌入加噪声
8      self.schedule_values, # 方差调度计划
9      timesteps # 时间步
10 )

```

然后将时间步长通过 MLP 以获得时间步长嵌入。

```

1  # Constructor
2  self.time_mlp = nn.Sequential(
3      SinusoidalPositionalEmbedding(
4          config.prior.max_time,
5          config.latent_dim
6      ),
7      nn.Linear(
8          config.latent_dim,
9          config.latent_dim * config.prior.r_mlp,
10         bias=config.prior.bias
11     ),
12     nn.SiLU(),
13     nn.Linear(
14         config.latent_dim * config.prior.r_mlp,
15         config.latent_dim,
16         bias=config.prior.bias
17     )
18 )
19
20 # Forward
21 timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)

```

7.2.3.5 可学习的嵌入

先验模型的另一个重要部分是可以训练的嵌入层。这些嵌入是 Torch 的一个参数, 将用于预测最终输出。从构造函数中学习到的嵌入需要在 Forward 方法中进行扩展, 以便批次中的每个项目都有一个嵌入。

```

1  # 初始化一个随机的需要学习的嵌入参数
2  self.learned_embedding = nn.Parameter(torch.randn(config.latent_dim))
3

```

```

4 # Forward
5 # 为批次中的每一项复制一份可学习参数
6 learned_embeddings = self.learned_embedding.repeat(
7     images.shape[0],
8     1
9 ) # (latent_dim) → (B, latent_dim)

```

7.2.3.6 输入的拼接

文本标题、CLIP 文本嵌入、时间步嵌入、加噪声的 CLIP 图像嵌入以及需要学习的嵌入将被连接成一个序列。所有这些项都将具有形状 $(B, \text{latent_dim})$ ，但我们需要在中间添加一个额外的维度，使它们具有形状 $(B, 1, \text{latent_dim})$ 。我们将在这个新维度上进行连接，使序列具有形状 $(B, 5, \text{latent_dim})$ 。如果使用包含更高质量图像的数据集，一种提高模型质量的可能方法是将文本嵌入、图像嵌入和/或时间步嵌入传递到卷积层，以增加序列长度。

```

1 tokens = torch.cat((
2     captions,          # 图像标题
3     text_embeddings,    # CLIP文本嵌入
4     timestep_embeddings, # 时间步嵌入
5     noisy_image_embedding, # 加噪声的CLIP图像嵌入
6     learned_embeddings  # 需要学习的嵌入
7 ), dim=1) # (B, 5, latent_dim)

```

然后，该序列会输入到一个带有因果注意力掩码的仅解码器架构的 Transformer。因果注意力掩码是一个由 1 组成的下三角矩阵，它使得某个 token 只能关注在它之前出现的 token。

```

1 # 初始化
2 self.decoder = nn.ModuleList(
3     [TransformerBlock(
4         config.latent_dim,
5         cond_width=config.latent_dim,
6         n_heads=config.prior.n_heads,
7         dropout=config.prior.dropout,
8         r_mlp=config.prior.r_mlp,
9         bias=config.prior.bias
10    ) for _ in range(config.prior.n_layers)]
11 )
12
13 self.register_buffer(
14     "causal_attention_mask",
15     torch.tril(torch.ones(5, 5))[None, :])
16 )
17
18 # Forward
19 for block in self.decoder:
20     tokens = block(tokens, mask=self.causal_attention_mask)

```

7.2.3.7 预测目标

最后，我们从 Transformer 的输出中获取学习到的嵌入，并将其传递给 LayerNorm 和 Linear 层以获得预测的图像嵌入。

```

1  # 初始化
2  self.output = nn.Sequential(
3      nn.LayerNorm(config.latent_dim),
4      nn.Linear(
5          config.latent_dim,
6          config.latent_dim,
7          bias=config.decoder.bias
8      )
9  )
10
11 # Forward: 取最后一个训练出来的可学习嵌入
12 pred_image_embeddings = self.output(tokens[:, -1, :])

```

虽然通常情况下, 扩散模型会预测噪声, 并通过迭代去除噪声来生成样本, 但 unCLIP 论文中提到, 最好直接预测 CLIP 图像嵌入。该模型的损失函数应该是预测值与实际 CLIP 图像嵌入之间的均方误差损失。

```

1  loss = nn.functional.mse_loss(pred_image_embeddings, image_embeddings)

```

在采样过程中, 为了提高质量, 模型应该生成两个 CLIP 图像嵌入的样本, 并选择与 CLIP 文本嵌入点积较高的样本。

```

1  def get_one_sample(self, text_embeddings, captions):
2      # 获取一个和CLIP文本嵌入形状相同的白噪声
3      # 而clip的图像嵌入和文本嵌入的形状相同
4      noisy_image_embeddings = torch.randn(
5          text_embeddings.shape,
6          device=self.config.device
7      )
8
9      # 所有项的时间步都是最大时间步T, 因为图像嵌入目前都是纯噪声
10     timesteps = torch.full(
11         (captions.shape[0],),
12         self.config.prior.max_time - 1
13     )
14
15     # 获取时间步嵌入
16     timestep_embeddings = self.time_mlp(
17         timesteps
18     ) # (B, ) → (B, latent_dim)
19
20     # (B, latent_dim) → (B, 1, latent_dim)
21     timestep_embeddings = timestep_embeddings[:, None, :]
22
23     # 为批次中每一项都创建一个需要学习的嵌入
24     learned_embeddings = self.learned_embedding.repeat(
25         captions.shape[0],
26         1
27     ) # (latent_dim) → (B, latent_dim)
28
29     # (B, latent_dim) → (B, 1, latent_dim)
30     learned_embeddings = learned_embeddings[:, None, :]

```

```

31
32     tokens = torch.cat((
33         captions,          # 图像标题
34         text_embeddings,    # CLIP文本嵌入
35         timestep_embeddings, # 最大时间步的嵌入 (比如T=1000)
36         noisy_image_embeddings, # 白噪声
37         learned_embeddings  # 需要学习的嵌入
38     ), dim=1) # (B, 5, latent_dim)
39
40     # 和因果注意力掩码一起输入给transformer块
41     for block in self.decoder:
42         tokens = block(tokens, mask=self.causal_attention_mask)
43
44     # 预测的图像嵌入 (取出学习到的嵌入, 输入给投影层)
45     pred_image_embeddings = self.output(tokens[:, -1, :])
46
47     return pred_image_embeddings
48
49 def sample(self, captions, masks=None):
50     # 获取CLIP文本嵌入
51     t_emb = self.clip.text_encoder(
52         captions,
53         mask=masks
54     ) # (B, text_seq_length) → (B, latent_dim)
55
56     # (B, latent_dim) → (B, 1, latent_dim)
57     text_embeddings = t_emb[:, None, :]
58
59     # 使图片文本的长度等于 latent dimension
60     if self.config.text_seq_length ≥ self.config.latent_dim:
61         # (B, max_seq_len) → (B, latent_dim)
62         captions = captions[:, :self.config.latent_dim]
63     else:
64         captions = nn.functional.pad(
65             captions,
66             (0, self.config.latent_dim - self.config.text_seq_length)
67         ) # (B, max_seq_len) → (B, latent_dim)
68
69     # (B, latent_dim) → (B, 1, latent_dim)
70     captions = captions[:, None, :]
71
72     # 获取两个采样
73     sample_1 = self.get_one_sample(text_embeddings, captions)
74     sample_2 = self.get_one_sample(text_embeddings, captions)
75
76     gen_image_embeddings = torch.zeros(sample_1.shape)
77
78     # 选择和文本嵌入点积更大的样本
79     for i in range(gen_image_embeddings.shape[0]):

```

```

80         if sample_1[i] @ t_emb[i] >= sample_2[i] @ t_emb[i]:
81             gen_image_embeddings[i] = sample_1[i]
82         else:
83             gen_image_embeddings[i] = sample_2[i]
84         # 预测的图片嵌入
85     return gen_image_embeddings

```

7.2.3.8 先验模型完整代码

综合以上所有，扩散先验模型的代码看起来应该是这样的：

```

1  class DiffusionPrior(nn.Module): Python
2      def __init__(self, config):
3          super().__init__()
4          # 加载clip模型
5          self.clip = CLIP(config).to(config.device)
6          self.clip.load_state_dict(torch.load(
7              config.clip.model_location,
8              map_location=config.device
9          ))
10         freeze_model(self.clip)
11
12         self.config = config
13
14         self.time_mlp = nn.Sequential(
15             SinusoidalPositionalEmbedding(
16                 config.prior.max_time,
17                 config.latent_dim
18             ),
19             nn.Linear(
20                 config.latent_dim,
21                 config.latent_dim * config.prior.r_mlp,
22                 bias=config.prior.bias
23             ),
24             nn.SiLU(),
25             nn.Linear(
26                 config.latent_dim * config.prior.r_mlp,
27                 config.latent_dim,
28                 bias=config.prior.bias
29             )
30         )
31
32         self.learned_embedding = nn.Parameter(
33             torch.randn(config.latent_dim)
34         )
35
36         self.schedule_values = get_schedule_values(config)
37
38         # Transformer blocks
39         self.decoder = nn.ModuleList(
40             [TransformerBlock(

```

```

41         config.latent_dim,
42         cond_width=config.latent_dim,
43         n_heads=config.prior.n_heads,
44         dropout=config.prior.dropout,
45         r_mlp=config.prior.r_mlp,
46         bias=config.prior.bias
47     ) for _ in range(config.prior.n_layers)]
48 )
49
50 # Output Projection, 输出为预测的图片嵌入
51 self.output = nn.Sequential(
52     nn.LayerNorm(config.latent_dim),
53     nn.Linear(
54         config.latent_dim,
55         config.latent_dim,
56         bias=config.decoder.bias
57     )
58 )
59
60 self.register_buffer(
61     "causal_attention_mask",
62     torch.tril(torch.ones(5, 5))[None, :])
63 )
64
65 def get_one_sample(self, text_embeddings, captions):
66     # Get image embeddings that are pure noise
67     noisy_image_embeddings = torch.randn(text_embeddings.shape,
68                                         device=self.config.device)
69
70     # timestep is max for all items because image embeddings are pure noise
71     timesteps = torch.full((captions.shape[0],), self.config.prior.max_time -
72                            1)
73
74     # Get timestep embeddings
75     timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)
76     timestep_embeddings = timestep_embeddings[:, None, :] # (B, latent_dim) →
77     (B, 1, latent_dim)
78
79     # Expand learned embedding so that there is one for each item in batch
80     learned_embeddings = self.learned_embedding.repeat(captions.shape[0], 1) #
81     (latent_dim) → (B, latent_dim)
82     learned_embeddings = learned_embeddings[:, None, :] # (B, latent_dim) →
83     (B, 1, latent_dim)
84
85     tokens = torch.cat((
86         captions, # Image Caption
87         text_embeddings, # CLIP Text Embedding
88         timestep_embeddings, # Timestep Embedding
89         noisy_image_embeddings, # Noisy CLIP Image Embedding
90         learned_embeddings # Learned Embedding
91     ), dim=1) # (B, 5, latent_dim)

```

```

88         # Pass through transformer blocks with causal attention mask
89         for block in self.decoder:
90             tokens = block(tokens, mask=self.causal_attention_mask)
91
92         # Get learned embeddings and pass through output projection to get CLIP
93         # image embeddings
94         pred_image_embeddings = self.output(tokens[:, -1, :])
95
96         return pred_image_embeddings
97
98     def sample(self, captions, masks=None):
99         # Get CLIP text embeddings
100         t_emb = self.clip.text_encoder(captions, mask=masks) # (B,
101         # text_seq_length) → (B, latent_dim)
102         text_embeddings = t_emb[:, None, :] # (B, latent_dim) → (B, 1,
103         # latent_dim)
104
105         # Make caption length equal to latent dimension
106         if self.config.text_seq_length ≥ self.config.latent_dim:
107             captions = captions[:, :self.config.latent_dim] # (B, max_seq_len) →
108             # (B, latent_dim)
109         else:
110             captions = nn.functional.pad(captions, (0, self.config.latent_dim -
111             self.config.text_seq_length)) # (B, max_seq_len) → (B, latent_dim)
112
113         captions = captions[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)
114
115         # Getting two samples
116         sample_1 = self.get_one_sample(text_embeddings, captions)
117         sample_2 = self.get_one_sample(text_embeddings, captions)
118
119         gen_image_embeddings = torch.zeros(sample_1.shape)
120
121         # Choosing the samples with the higher dot product with text embeddings
122         for i in range(gen_image_embeddings.shape[0]):
123             if sample_1[i] @ t_emb[i] ≥ sample_2[i] @ t_emb[i]:
124                 gen_image_embeddings[i] = sample_1[i]
125             else:
126                 gen_image_embeddings[i] = sample_2[i]
127
128         return gen_image_embeddings
129
130     def forward(self, images, captions, masks=None):
131         # Get CLIP image embeddings
132         image_embeddings = self.clip.image_encoder(images) # (B, C, H, W) → (B,
133         # latent_dim)
134
135         # Get CLIP text embeddings
136         text_embeddings = self.clip.text_encoder(captions, mask=masks) # (B,
137         # text_seq_length) → (B, latent_dim)
138         text_embeddings = text_embeddings[:, None, :] # (B, latent_dim) → (B, 1,
139         # latent_dim)

```



```

132
133     # Make caption length equal to latent dimension
134     if self.config.text_seq_length > self.config.latent_dim:
135         captions = captions[:, :self.config.latent_dim] # (B, max_seq_len) →
136         (B, latent_dim)
137     else:
138         captions = nn.functional.pad(captions, (0, self.config.latent_dim -
139         self.config.text_seq_length)) # (B, max_seq_len) → (B, latent_dim)
140
141     captions = captions[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)
142
143     # Get random timesteps for forward diffusion
144     timesteps = torch.randint(0, self.config.prior.max_time,
145     (images.shape[0],)) # (B, )
146
147     # Get timestep embeddings
148     timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)
149     timestep_embeddings = timestep_embeddings[:, None, :] # (B, latent_dim) →
150     (B, 1, latent_dim)
151
152     # Perform forward diffusion to get noisy CLIP image embeddings
153     noisy_image_embedding, _ = forward_diffusion(image_embeddings,
154     self.schedule_values, timesteps)
155     noisy_image_embedding = noisy_image_embedding[:, None, :] # (B,
156     latent_dim) → (B, 1, latent_dim)
157
158     # Expand learned embedding so that there is one for each item in batch
159     learned_embeddings = self.learned_embedding.repeat(images.shape[0], 1) #
160     (latent_dim) → (B, latent_dim)
161     learned_embeddings = learned_embeddings[:, None, :] # (B, latent_dim) →
162     (B, 1, latent_dim)
163
164     tokens = torch.cat((
165         captions,          # Image Caption
166         text_embeddings,    # CLIP Text Embedding
167         timestep_embeddings, # Timestep Embedding
168         noisy_image_embedding, # Noisy CLIP Image Embedding
169         learned_embeddings   # Learned Embedding
170     ), dim=1) # (B, 5, latent_dim)
171
172     # Pass through transformer blocks with causal attention mask
173     for block in self.decoder:
174         tokens = block(tokens, mask=self.causal_attention_mask)
175
176     # Get learned embeddings and pass through output projection to get CLIP
177     image embeddings
178     pred_image_embeddings = self.output(tokens[:, -1, :])
179
180     loss = nn.functional.mse_loss(
181         pred_image_embeddings,
182         image_embeddings
183     )

```


7.3.2 残差块

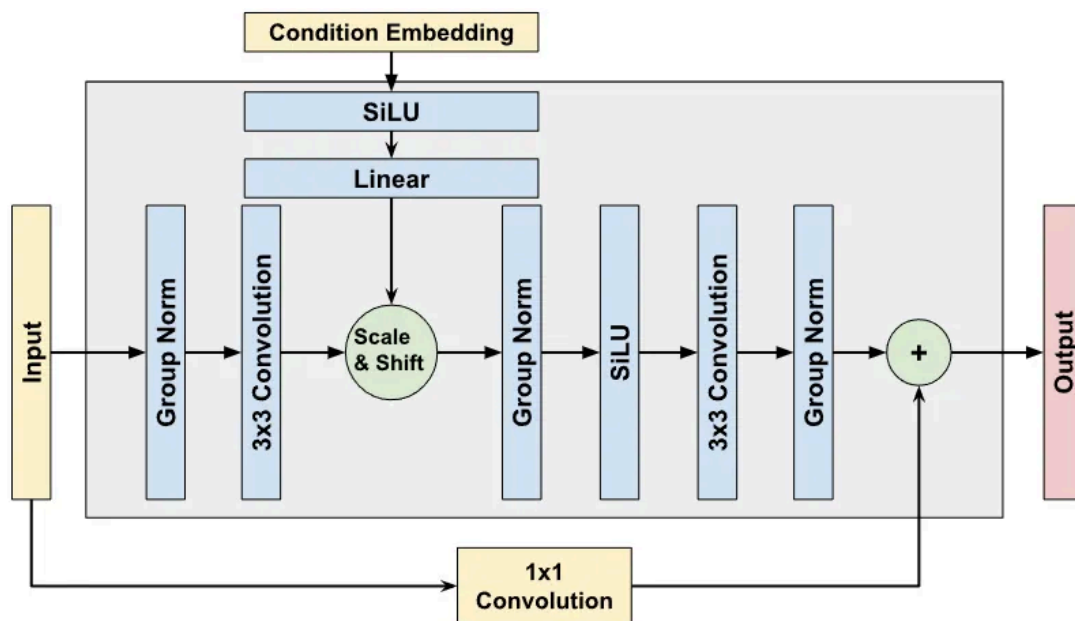


图 7.5 残差块的架构

编码器和解码器层均由残差块构成，这些残差块包含用于识别和定位特征的卷积。这些残差块通常由归一化、激活、卷积、归一化、激活、卷积组成；然而，我们的残差块将是归一化、卷积、归一化、激活、卷积、归一化（如图所示）。Han 等人的论文证明了这种架构可以在保持非线性的同时提升性能。

```

1 # 初始化
2 self.layers1 = nn.Sequential(
3     nn.GroupNorm(n_groups, d_in),
4     nn.Conv2d(d_in, d_out, kernel_size, padding=1)
5 )
6
7 self.layers2 = nn.Sequential(
8     nn.GroupNorm(n_groups, d_out),
9     nn.SiLU(),
10    nn.Dropout(p=dropout),
11    nn.Conv2d(d_out, d_out, kernel_size, padding=1),
12    nn.GroupNorm(n_groups, d_out)
13 )
14
15 # Forward
16 x = self.layers1(x_0)
17 x = self.layers2(x)

```

Python

对于残差块的归一化部分，我们将使用 **GroupNorm**。之所以使用 **GroupNorm** 而不是 **BatchNorm**，是因为它的性能与批次大小无关，这使得它在小批次或可变批次大小上表现更佳。使用 **GroupNorm** 也有助于提高训练期间的稳定性。

残差块的激活部分用于实现非线性特性。虽然 **ReLU** 通常用于残差网络，但我们将使用 **SiLU** 作为激活函数。在 Ramachandran 等人的论文中，尽管模型和超参数是专门为 **ReLU** 设置的，但 **SiLU**

的表现优于 ReLU 和其他激活函数。由于 SiLU 的简单性以及 ReLU 的相似性，因此只需用它来代替 ReLU 即可轻松实现。

对于卷积，我们将使用 3x3 核大小和 SAME 填充的 Conv2d 来保留空间维度。SAME 填充的工作原理是在输入的边界添加零，以确保输出形状与未填充时的输入形状相同。第一个卷积会将输入从 d_{in} 通道投影到 d_{out} 通道，而第二个卷积则仅保留 d_{out} 通道。

残差块也将根据输入的嵌入信息进行调节。执行调节的主要方法之一是将嵌入添加到输入中。对于我们的模型，我们将执行线性投影以获得比例和偏差值。然后将输入乘以比例，然后添加偏差。Nichol 和 Dhariwal 表明，与加法相比，使用这种调节方法可以提高 FID 分数。编写此部分代码时需要注意的一点是，调节嵌入的维度可能比输入的维度少。因此，我们需要在嵌入的末尾添加维度。例如，如果输入的形状为 (B, C, L) ，而嵌入的形状为 (B, C) ，则需要添加一个维度以使其形状为 $(B, C, 1)$ 。

```

1  # 初始化
2  self.use_scale_shift = use_scale_shift
3
4  self.cond_layers = nn.Sequential(
5      nn.SiLU(),
6      nn.Linear(model_channels, d_out * 2 if use_scale_shift else d_out)
7  )
8
9  # Forward
10 emb = self.cond_layers(emb)
11
12 while len(emb.shape) < len(x.shape):
13     emb = emb[ ..., None]
14
15 if self.use_scale_shift:
16     y_s, y_b = emb.chunk(2, dim=1)
17     x = y_s * x + y_b
18 else:
19     x += emb

```

调节是在第一次卷积之后进行的，而不是在残差块的开始处进行的，因为它允许模型在使用时间步信息对其进行细化之前获取基本特征，从而获得更好的表示。

最后，通过将 ResNet 模块的原始输入添加到输出来创建跳跃连接。如果需要更改通道数以匹配输出，则对输入执行 1x1 卷积。这些跳跃连接用于增强特征传播。

```

1  # 初始化
2  self.residual = nn.Conv2d(d_in, d_out, 1) if d_in != d_out else nn.Identity()
3
4  # Forward
5  x += self.residual(x_0)

```

将所有这些放在一起，残差块的最终代码应该是这样的：

```

1  class ResidualBlock(nn.Module):
2      def __init__(
3          self,
4          d_in,
5          d_out,
6          cond_channels=128,
7          n_groups=8, # 组归一化GroupNorm每一组的数据量

```

```
8         kernel_size=(3,3),
9         dropout=0.0,
10        use_scale_shift=True
11    ):
12        super().__init__()
13
14        self.use_scale_shift = use_scale_shift
15
16        self.layers1 = nn.Sequential(
17            nn.GroupNorm(n_groups, d_in),
18            nn.SiLU(),
19            nn.Conv2d(d_in, d_out, kernel_size, padding=1)
20        )
21
22        # Activation & Linear Projection for Embedding
23        self.cond_layers = nn.Sequential(
24            nn.SiLU(),
25            # d_out multiplied by 2 in order to split into scale & shift if
            # necessary
26            nn.Linear(cond_channels, d_out * 2 if use_scale_shift else d_out)
27        )
28
29        self.layers2 = nn.Sequential(
30            nn.GroupNorm(n_groups, d_out),
31            nn.SiLU(),
32            nn.Dropout(p=dropout),
33            nn.Conv2d(d_out, d_out, kernel_size, padding=1),
34            nn.GroupNorm(n_groups, d_out)
35        )
36
37        self.residual = nn.Conv2d(d_in, d_out, 1) \
38            if d_in != d_out \
39            else nn.Identity()
40
41    def forward(self, x_0, emb):
42        # x_0是原始图片, emb是图片的clip嵌入
43        x = self.layers1(x_0)
44
45        emb = self.cond_layers(emb)
46
47        # Adding dimensions to embedding
48        while len(emb.shape) < len(x.shape):
49            emb = emb[ ..., None]
50
51        # Conditioning input with embedding
52        if self.use_scale_shift:
53            # Getting scale and shift
54            y_s, y_b = emb.chunk(2, dim=1)
55            # Performing scale and shift
56            x = y_s * x + y_b
```

```

57     else:
58         # Adding embedding to input
59         x += emb
60
61     x = self.layers2(x)
62
63     # Skip Connection
64     x += self.residual(x_0)
65
66     return x

```

7.3.3 注意力块

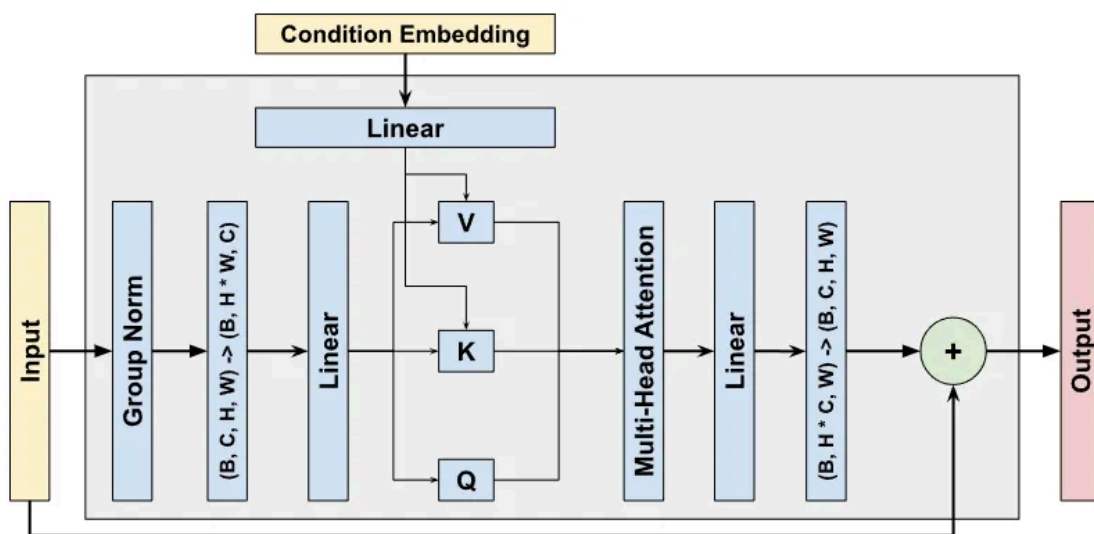


图 7.6 注意力块

在我们的模型中，我们将把注意力模块放置在编码器和解码器内层以及瓶颈层（bottle 层，连接编码器和解码器的底层）的残差模块之后。例如，如果编码器和解码器有四层，那么第二层和第三层就会有一个注意力模块。

使用注意力模块对我们的模型有诸多好处。其一是它有助于捕捉图像不同部分之间的空间关系信息。其二是它有助于衡量模型特征的重要性。最后，注意力模块还可以用来调节图像生成过程。

有多种方法可以为注意力模块实现注意力机制。其中一些选项包括自注意力和交叉注意力。我们将使用的方法是 GLIDE 模型中用到的两种方法的混合。对于这种方法，我们将从输入和条件信息中获取 K 和 V，并将它们连接在一起以获得最终的 K 和 V。

```

1  Q, K, V = self.qkv(x).chunk(3, dim=-1)
2
3  Q = Q.view(B, L, self.n_heads, self.head_size).transpose(1, 2)
4
5  K = K.view(B, L, self.n_heads, self.head_size).transpose(1, 2)
6
7  V = V.view(B, L, self.n_heads, self.head_size).transpose(1, 2)
8
9  # Concatenating keys and values of input and condition
10 if cond is not None:

```

Python

```

11     k_c, v_c = self.cond_kv(cond).chunk(2, dim=-1)
12     k_c = k_c.view(B, cond.shape[1], self.n_heads, self.head_size).transpose(1, 2)
13     v_c = v_c.view(B, cond.shape[1], self.n_heads, self.head_size).transpose(1, 2)
14     K = torch.cat((K, k_c), dim=-2)
15     V = torch.cat((V, v_c), dim=-2)

```

需要注意的是，输入图像的形状为 (B, C, H, W) ，但对于我们的注意力机制，我们希望其形状为 (B, L, C) 。因此，我们需要将 H 和 W 维度合并，并将其与 C 维度转置。执行注意力机制后，我们需要将输出转换回其原始形状。

```

1  b, c, h, w = x_0.shape
2
3  # Changing shape to perform attention
4  x = x.permute(0, 2, 3, 1).view(b, h * w, c) # (B, C, H, W) → (B, H * W, C)
5
6  # Attention
7  x = self.attention(x, cond)
8
9  # Changing back to original shape
10 x = x.view(b, h, w, c).permute(0, 3, 1, 2)

```

最终的代码看起来应该是这样的：

```

1  class AttentionBlock(nn.Module):
2      def __init__(self, n_channels, cond_channels, n_groups=8, n_heads=1,
3          dropout=0.0):
4      super().__init__()
5
6      assert n_channels % n_heads == 0, "n_channels must be divisible by n_heads"
7
8      self.n_heads = n_heads
9      self.head_size = n_channels // n_heads
10     self.scale = self.head_size ** -0.5
11
12     self.group_norm = nn.GroupNorm(n_groups, n_channels)
13
14     self.qkv = nn.Linear(n_channels, n_channels * 3)
15
16     self.cond_kv = nn.Linear(cond_channels, n_channels * 2)
17
18     self.out_proj = nn.Linear(n_channels, n_channels)
19
20     self.dropout = nn.Dropout(dropout)
21
22     def attention(self, x, cond=None):
23         B, L, _ = x.shape
24
25         # Getting queries, keys, and values for input
26         Q, K, V = self.qkv(x).chunk(3, dim=-1)
27
28         Q = Q.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

```

```

29         K = K.view(B, L, self.n_heads, self.head_size).transpose(1, 2)
30
31         V = V.view(B, L, self.n_heads, self.head_size).transpose(1, 2)
32
33         # Concatenating keys and values of condition to keys and values of input
34         if cond is not None:
35             k_c, v_c = self.cond_kv(cond).chunk(2, dim=-1)
36             k_c = k_c.view(B, cond.shape[1], self.n_heads,
37                             self.head_size).transpose(1, 2)
37             v_c = v_c.view(B, cond.shape[1], self.n_heads,
38                             self.head_size).transpose(1, 2)
38             K = torch.cat((K, k_c), dim=-2)
39             V = torch.cat((V, v_c), dim=-2)
40
41         # Get dot product between queries and keys
42         attention = torch.matmul(Q, K.transpose(-2, -1))
43
44         # Scale
45         attention = attention * self.scale
46
47         # Applying softmax
48         attention = torch.softmax(attention, dim=-1)
49
50         # Get dot product with values
51         attention = torch.matmul(attention, V)
52
53         # Combine heads
54         attention = attention.transpose(1, 2)
55         attention = attention.contiguous().view(x.shape)
56
57         # Output projection
58         attention = self.out_proj(attention)
59
60         # Dropout
61         attention = self.dropout(attention)
62
63         return attention
64
65     def forward(self, x_0, cond=None):
66         b, c, h, w = x_0.shape
67
68         # Group normalization
69         x = self.group_norm(x_0)
70
71         # Changing shape to perform attention
72         x = x.permute(0, 2, 3, 1).view(b, h * w, c) # (B, C, H, W) → (B, H * W, C)
73
74         # Attention
75         x = self.attention(x, cond)
76
77         # Changing back to original shape

```



```

78         x = x.view(b, h, w, c).permute(0, 3, 1, 2)
79
80         # Residual connection
81         x = x + x_0
82
83         return x

```

7.3.4 下采样

在每个编码器层之间，我们需要降低输入的分辨率。主要的下采样方法有两种：池化和使用卷积层。池化无需参数，这使得它的计算效率更高，并且有助于防止过拟合。卷积层的优势在于它本身带有参数，这使得它能够学习并保留重要的特征。我们将在模型中使用步长 2，这将使输入的分辨率降低 2 倍。

我们将对模型进行编码，以便能够使用任一方法，但是在训练模型时，我们将使用卷积方法。

```

1  class Downsample(nn.Module): Python
2      def __init__(self, n_channels, kernel_size=(3,3), stride=2, down_pool=False):
3          super().__init__()
4
5          if down_pool:
6              self.down = nn.AvgPool2d(stride)
7          else:
8              self.down = nn.Conv2d(n_channels, n_channels, kernel_size,
9                                   stride=stride, padding=1)
10
11     def forward(self, x):
12         x = self.down(x)
13         return x

```

7.3.5 上采样

在每个解码器层之间，我们需要提高输入的分辨率。为此，我们首先进行插值，将输入的高度和宽度乘以 2。之后，我们将它传入一个卷积层，该层将学习并保留重要特征，同时确保通道数正确。

```

1  class Upsample(nn.Module): Python
2      def __init__(self, d_in, d_out, kernel_size=(3,3)):
3          super().__init__()
4
5          self.conv = nn.Conv2d(d_in, d_out, kernel_size, padding=1)
6
7      def forward(self, x):
8          # interpolate: 插值
9          x = nn.functional.interpolate(x, scale_factor=2)
10         x = self.conv(x)
11         return x

```

7.3.6 模型的训练

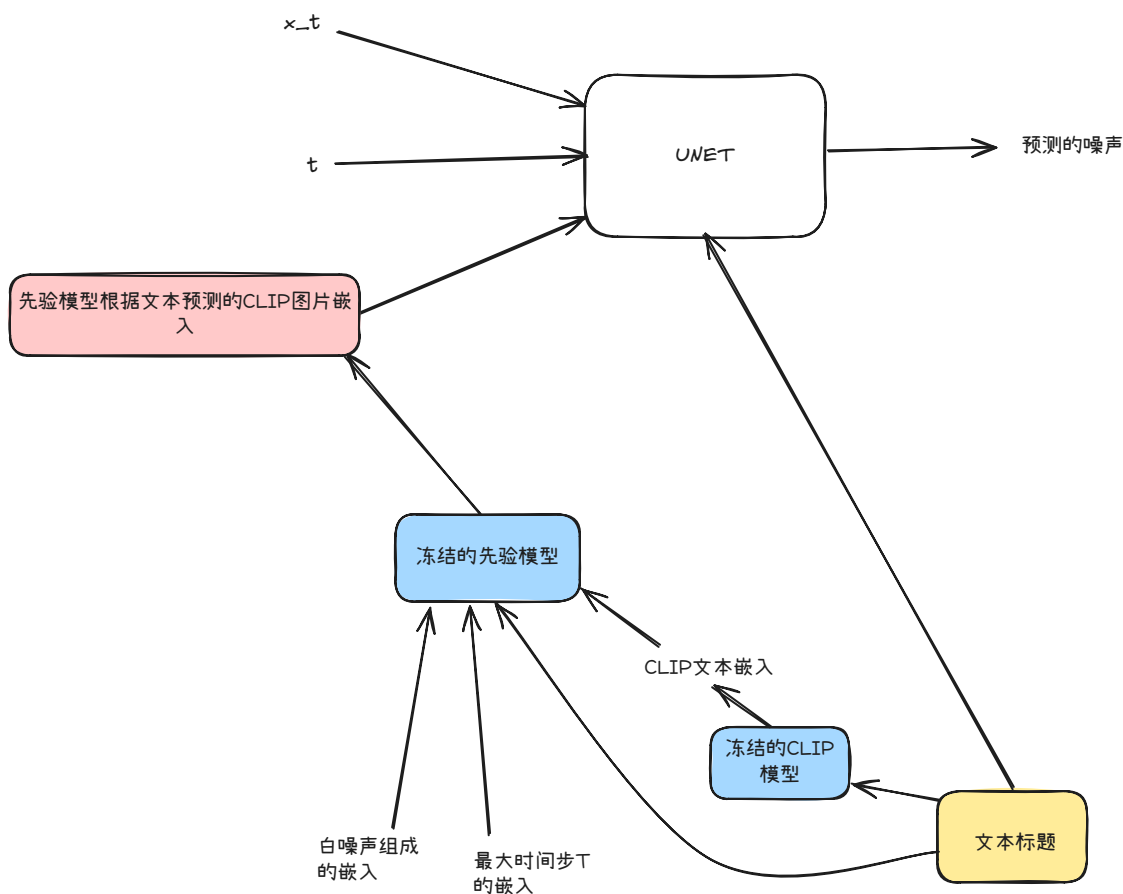


图 7.7 解码器训练流程

在训练解码器时，模型需要做的第一件事就是设置条件信息。为此，我们首先需要从先验模型中采样以获取 CLIP 图像嵌入。与 CLIP 模型一样，在加载先验模型时，应冻结层并将模式设置为 `eval`。

```

1 # 初始化先验网络
2 self.prior = DiffusionPrior(config).to(config.device)
3 self.prior.load_state_dict(torch.load(
4     config.prior.model_location,
5     map_location=config.device
6 ))
7 # 冻结模型
8 freeze_model(self.prior)
9
10 # Forward
11 # 得到预测的图片clip嵌入
12 img_embeddings = self.prior.sample(caption, mask).to(x.device)
  
```

之后，需要使用输入的时间步来获取时间步嵌入。然后将先验模型生成的 CLIP 图像嵌入投影并添加到时间步嵌入中。这些嵌入将用于调节模型的残差块。

```

1 # 投影层，将clip图片嵌入转换成和时间步嵌入相同的形状，方便相加
2 self.img_projection = nn.Sequential(
  
```

```

3     nn.Linear(config.latent_dim, config.decoder.cond_channels),
4     nn.SiLU(),
5     nn.Linear(config.decoder.cond_channels, config.decoder.cond_channels)
6 )
7
8 #Forward
9 # 条件嵌入 = 时间步嵌入 + clip图片嵌入
10 c_emb = self.time_mlp(time) + self.img_projection(img_embeddings)
11
12 for module in self.encoder:
13     if isinstance(module, ResidualBlock):
14         x = module(x, c_emb)

```

对于注意力条件信息，我们首先需要将文本标题传入文本 Transformer 编码器。在 unCLIP 论文中，之所以使用这些文本编码，是因为 Ramesh 等人认为这有助于学习 CLIP 无法学习的自然语言知识。但在测试过程中，他们发现这种方法效果不佳，因此这部分是可选的。

```

1 # Constructor
2 self.text_embedding = nn.Embedding(config.vocab_size, config.latent_dim)
  self.positional_encodings =
3 nn.Parameter(torch.randn(config.text_seq_length, config.latent_dim) *
  (config.latent_dim ** -0.5))
4
5 self.text_encoder = nn.ModuleList(
6     [TransformerBlock(
7         config.latent_dim,
8         cond_width=config.latent_dim,
9         n_heads=config.decoder.n_heads,
10        dropout=config.decoder.dropout,
11        r_mlp=config.decoder.r_mlp,
12        bias=config.decoder.bias
13     ) for _ in range(config.decoder.text_layers)]
14 )
15
16 self.final_ln = nn.LayerNorm(config.latent_dim)
17
18 # Function
19 def encode_text(self, text, mask=None):
20     x = self.text_embedding(text)
21
22     x = x + self.positional_encodings
23
24     for block in self.text_encoder:
25         x = block(x, mask=mask)
26
27     x = self.final_ln(x)
28
29     return x
30
31 # Forward
32 text_encodings = self.encode_text(text, mask)

```

 Python

对文本进行编码后，CLIP 图像嵌入被投影到四个额外的 token 中，并连接到编码文本的末尾。

```
1 # Constructor
2 self.get_img_tokens = nn.Linear(1, config.decoder.n_img_tokens)
3
4 # Forward
5 img_tokens = self.get_img_tokens(img_embeddings[ ..., None]).permute(0, 2, 1)
6
7 c_attn = torch.cat([text_encodings, img_tokens], dim=1)
```

设置条件信息后，噪声图像将通过初始卷积层，以获得初始模型通道的通道数。

```
1 # Constructor
2 ch = config.decoder.model_channels
3
4 self.in_conv = nn.Conv2d(
5     config.img_channels,
6     ch,
7     config.decoder.kernel_size,
8     padding=1
9 )
10
11 # Forward
12 x = self.in_conv(x)
```

现在，含噪图像已达到所需的通道数，它们可以与条件信息一起通过 UNet 层。UNet 将包含四个编码器层和解码器层，中间有一个瓶颈层。对于每层的通道数，我们将使用 [1, 2, 4, 8] 作为层通道数与模型通道数的比率。UNet 的编码器、解码器和瓶颈层都将包含两个残差块。在瓶颈残差块之间以及编码器和解码器内层的每个残差块之后，放置了注意力块。跳跃连接位于编码器和解码器的残差块之间。

```
1 # Config
2 model_channels:int = 32
3 channel_ratios:list[int] = field(default_factory=lambda: [1, 2, 4, 8])
4 n_layer_blocks:int = 2
5
6 # Constructor
7 # UNet Encoder Layers
8 self.encoder = nn.ModuleList([])
9 for r in config.decoder.channel_ratios:
10     for _ in range(config.decoder.n_layer_blocks):
11         self.encoder.append(ResidualBlock(ch, config.decoder.model_channels * r,
12                                           config.decoder.cond_channels, config.decoder.n_groups,
13                                           config.decoder.kernel_size, config.decoder.dropout,
14                                           config.decoder.use_scale_shift))
15
16         ch = config.decoder.model_channels * r
17         if r != config.decoder.channel_ratios[0] and r !=
18             config.decoder.channel_ratios[-1]:
19             self.encoder.append(AttentionBlock(ch, config.latent_dim,
20                                                config.decoder.n_groups, config.decoder.n_heads,
21                                                config.decoder.dropout))
22
23         if r != config.decoder.channel_ratios[-1]:
```

```

18         self.encoder.append(Downsample(ch, config.decoder.kernel_size,
19                                         config.decoder.stride, config.decoder.down_pool))
20     # UNet Bottleneck Layers
21     self.bottleneck = nn.ModuleList([])
22     for block in range(config.decoder.n_layer_blocks):
23         self.bottleneck.append(ResidualBlock(ch, ch, config.decoder.cond_channels,
24                                             config.decoder.n_groups, config.decoder.kernel_size, config.decoder.dropout,
25                                             config.decoder.use_scale_shift))
26
27     if block != config.decoder.n_layer_blocks - 1:
28         self.bottleneck.append(AttentionBlock(ch, config.latent_dim,
29                                             config.decoder.n_groups, config.decoder.n_heads, config.decoder.dropout))
30
31     # UNet Decoder Layers
32     self.decoder = nn.ModuleList([])
33     for r in range(len(config.decoder.channel_ratios))[::-1]:
34         for _ in range(config.decoder.n_layer_blocks):
35             self.decoder.append(ResidualBlock(ch * 2, ch, config.decoder.cond_channels,
36                                             config.decoder.n_groups, config.decoder.kernel_size,
37                                             config.decoder.dropout, config.decoder.use_scale_shift))
38
39             if r != 0 and r != len(config.decoder.channel_ratios) - 1:
40                 self.decoder.append(AttentionBlock(ch, config.latent_dim,
41                                             config.decoder.n_groups, config.decoder.n_heads,
42                                             config.decoder.dropout))
43
44             if r != 0:
45                 ch = config.decoder.model_channels * config.decoder.channel_ratios[r-1]
46                 self.decoder.append(Upsample(config.decoder.model_channels *
47                                             config.decoder.channel_ratios[r], ch, config.decoder.kernel_size))
48
49     # Forward
50     for module in self.encoder:
51         if isinstance(module, ResidualBlock):
52             x = module(x, c_emb)
53             self.connections.append(x)
54         elif isinstance(module, AttentionBlock):
55             x = module(x, cond=c_attn)
56         else:
57             x = module(x)
58
59     for module in self.bottleneck:
60         if isinstance(module, ResidualBlock):
61             x = module(x, c_emb)
62         elif isinstance(module, AttentionBlock):
63             x = module(x, cond=c_attn)
64         else:
65             x = module(x)
66
67     for module in self.decoder:
68         if isinstance(module, ResidualBlock):
69             x = torch.cat([x, self.connections.pop()], dim=1)

```

```

62         x = module(x, c_emb)
63     elif isinstance(module, AttentionBlock):
64         x = module(x, cond=c_attn)
65     else:
66         x = module(x)

```

然后，UNet 解码器层的输出会经过 GroupNorm 和 SiLU 激活层，之后使用 Conv2d 让输出恢复到原始通道数。

```

1  # Constructor
2  self.output = nn.Sequential(
3      nn.GroupNorm(config.decoder.n_groups, config.decoder.model_channels),
4      nn.SiLU(),
5      nn.Conv2d(config.decoder.model_channels, config.img_channels,
6                config.decoder.kernel_size, padding=1)
7  )
8  # Forward
9  x = self.output(x)

```

将所有内容放在一起，解码器模型的最终代码将如下所示：

```

1  class Decoder(nn.Module):
2      def __init__(self, config):
3          super().__init__()
4          # Loading Prior Model
5          self.prior = DiffusionPrior(config).to(config.device)
6          self.prior.load_state_dict(torch.load(
7              config.prior.model_location,
8              map_location=config.device
9          ))
10         freeze_model(self.prior)
11
12         # MLP to get time embeddings
13         self.time_mlp = nn.Sequential(
14             SinusoidalPositionalEmbedding(
15                 config.decoder.max_time,
16                 config.decoder.model_channels
17             ),
18             nn.Linear(
19                 config.decoder.model_channels,
20                 config.decoder.cond_channels
21             ),
22             nn.SiLU(),
23             nn.Linear(
24                 config.decoder.cond_channels,
25                 config.decoder.cond_channels
26             )
27         )
28
29         # MLP to project CLIP image embeddings
30         self.img_projection = nn.Sequential(

```

```

31         nn.Linear(config.latent_dim, config.decoder.cond_channels),
32         nn.SiLU(),
33         nn.Linear(config.decoder.cond_channels, config.decoder.cond_channels)
34     )
35
36     # Projection to get image tokens
37     self.get_img_tokens = nn.Linear(1, config.decoder.n_img_tokens)
38
39     # Embedding layer for text captions
40     self.text_embedding = nn.Embedding(config.vocab_size, config.latent_dim)
41
42     # Learned positional encodings for text captions
43     self.positional_encodings = nn.Parameter(torch.randn(
44         config.text_seq_length,
45         config.latent_dim
46     ) * (config.latent_dim ** -0.5))
47
48     # Transformer encoder blocks to encode text captions
49     self.text_encoder = nn.ModuleList(
50         [TransformerBlock(
51             config.latent_dim,
52             cond_width=config.latent_dim,
53             n_heads=config.decoder.n_heads,
54             dropout=config.decoder.dropout,
55             r_mlp=config.decoder.r_mlp,
56             bias=config.decoder.bias
57         ) for _ in range(config.decoder.text_layers)]
58     )
59
60     self.final_ln = nn.LayerNorm(config.latent_dim)
61
62     ch = config.decoder.model_channels
63
64     # Initial convolution
65     self.in_conv = nn.Conv2d(
66         config.img_channels,
67         ch,
68         config.decoder.kernel_size,
69         padding=1
70     )
71
72     # UNet Encoder Layers
73     self.encoder = nn.ModuleList([])
74     for r in config.decoder.channel_ratios:
75         for _ in range(config.decoder.n_layer_blocks):
76             self.encoder.append(ResidualBlock(
77                 ch,
78                 config.decoder.model_channels * r,
79                 config.decoder.cond_channels,
80                 config.decoder.n_groups,

```

```

81         config.decoder.kernel_size,
82         config.decoder.dropout,
83         config.decoder.use_scale_shift
84     ))
85
86     ch = config.decoder.model_channels * r
87     if r != config.decoder.channel_ratios[0] \
88         and r != config.decoder.channel_ratios[-1]:
89         self.encoder.append(AttentionBlock(
90             ch,
91             config.latent_dim,
92             config.decoder.n_groups,
93             config.decoder.n_heads,
94             config.decoder.dropout
95         ))
96
97     if r != config.decoder.channel_ratios[-1]:
98         self.encoder.append(Downsample(
99             ch,
100            config.decoder.kernel_size,
101            config.decoder.stride,
102            config.decoder.down_pool
103        ))
104
105     # UNet Bottleneck Layers
106     self.bottleneck = nn.ModuleList([])
107     for block in range(config.decoder.n_layer_blocks):
108         self.bottleneck.append(ResidualBlock(
109             ch,
110             ch,
111             config.decoder.cond_channels,
112             config.decoder.n_groups,
113             config.decoder.kernel_size,
114             config.decoder.dropout,
115             config.decoder.use_scale_shift
116         ))
117
118     if block != config.decoder.n_layer_blocks - 1:
119         self.bottleneck.append(AttentionBlock(
120             ch,
121             config.latent_dim,
122             config.decoder.n_groups,
123             config.decoder.n_heads,
124             config.decoder.dropout
125         ))
126
127     # UNet Decoder Layers
128     self.decoder = nn.ModuleList([])
129     for r in range(len(config.decoder.channel_ratios))[::-1]:
130         for _ in range(config.decoder.n_layer_blocks):

```



```

131         self.decoder.append(ResidualBlock(
132             ch * 2,
133             ch,
134             config.decoder.cond_channels,
135             config.decoder.n_groups,
136             config.decoder.kernel_size,
137             config.decoder.dropout,
138             config.decoder.use_scale_shift
139         ))
140
141         if r != 0 and r != len(config.decoder.channel_ratios) - 1:
142             self.decoder.append(AttentionBlock(
143                 ch,
144                 config.latent_dim,
145                 config.decoder.n_groups,
146                 config.decoder.n_heads,
147                 config.decoder.dropout
148             ))
149
150         if r != 0:
151             ch = config.decoder.model_channels \
152                 * config.decoder.channel_ratios[r-1]
153             self.decoder.append(Upsample(
154                 config.decoder.model_channels \
155                     * config.decoder.channel_ratios[r],
156                 ch,
157                 config.decoder.kernel_size
158             ))
159
160         # Output projection
161         self.output = nn.Sequential(
162             nn.GroupNorm(
163                 config.decoder.n_groups,
164                 config.decoder.model_channels
165             ),
166             nn.SiLU(),
167             nn.Conv2d(
168                 config.decoder.model_channels,
169                 config.img_channels,
170                 config.decoder.kernel_size,
171                 padding=1
172             )
173         )
174
175         # Skip connections
176         self.connections = []
177
178         def encode_text(self, text, mask=None):
179             x = self.text_embedding(text)
180

```

```

181     x = x + self.positional_encodings
182
183     for block in self.text_encoder:
184         x = block(x, mask=mask)
185
186     x = self.final_ln(x)
187
188     return x
189
190     def forward(self, x, time, caption=None, mask=None):
191         # Sample prior model to get CLIP image embeddings
192         # 使用文本标题输入给先验模型prior model, 获得预测的clip图片嵌入
193         img_embeddings = self.prior.sample(caption, mask).to(x.device)
194
195         # 条件嵌入 = 时间步嵌入 + clip图片嵌入
196         c_emb = self.time_mlp(time) + self.img_projection(img_embeddings)
197
198         # Get conditioning information for attention blocks
199         c_attn = self.get_img_tokens(img_embeddings[... , None]).permute(0, 2, 1)
200         # 图片标题为空, 那么unet是一个无条件的扩散模型
201         if caption is not None:
202             c_attn = torch.cat([self.encode_text(caption, mask), c_attn], dim=1)
203
204         # Initial convolution
205         x = self.in_conv(x)
206
207         # UNet encoder layers
208         for module in self.encoder:
209             if isinstance(module, ResidualBlock):
210                 x = module(x, c_emb)
211                 self.connections.append(x)
212             elif isinstance(module, AttentionBlock):
213                 x = module(x, cond=c_attn)
214             else:
215                 x = module(x)
216
217         # UNet bottleneck layers
218         for module in self.bottleneck:
219             if isinstance(module, ResidualBlock):
220                 x = module(x, c_emb)
221             elif isinstance(module, AttentionBlock):
222                 x = module(x, cond=c_attn)
223             else:
224                 x = module(x)
225
226         # UNet decoder layers
227         for module in self.decoder:
228             if isinstance(module, ResidualBlock):
229                 x = torch.cat([x, self.connections.pop()], dim=1)
230                 x = module(x, c_emb)

```

```

231         elif isinstance(module, AttentionBlock):
232             x = module(x, cond=c_attn)
233         else:
234             x = module(x)
235
236         # Output projection
237         x = self.output(x)
238
239         return x

```

7.3.7 损失的计算

训练解码器模型的一个重要部分是损失函数。我们的损失函数首先要做的就是对批次中的每个图像进行随机时间步长的采样。

```

1 timesteps = torch.randint(
2     0,
3     config.decoder.max_time,
4     (image.shape[0],),
5     device=config.device,
6     dtype=torch.long
7 )

```

然后使用这些时间步长和计划值来获取通过前向扩散添加的噪声图像和噪声。

```

1 # 方差计划
2 schedule_values = get_schedule_values(config)
3 # x_t, 真实的噪声noise
4 noisy_image, noise = forward_diffusion(
5     image,
6     schedule_values,
7     timesteps
8 )

```

然后将噪声图像、时间步长、文本标题和文本掩码传递到模型中以获得预测噪声。

```

1 # x_t, 时间步, 图片标题 (最终被处理为条件信息, clip图片嵌入)
2 pred_noise = decoder(noisy_image, timesteps, caption, mask)

```

最后, 我们可以使用模型的预测噪声和前向扩散函数的实际噪声来计算损失。我们将用于解码器的损失是预测噪声和实际噪声之间的均方误差损失。

```

1 # 预测的噪声和真实噪声之间计算均方误差
2 loss = nn.functional.mse_loss(pred_noise, noise)

```

综合起来, 我们得到:

```

1 # Calculating Loss
2 get_schedule_values(config)
3 timesteps = torch.randint(0, config.decoder.max_time, (image.shape[0],),
4     device=config.device, dtype=torch.long)
5 noisy_image, noise = forward_diffusion(image, schedule_values, timesteps)
6 pred_noise = decoder(noisy_image, timesteps, caption, mask)
7 loss = nn.functional.mse_loss(pred_noise, noise)

```

7.4 训练配置

```

1  @dataclass
2  class CLIPConfig:
3      # Vision Transformer
4      patch_size:tuple[int,int] = (4,4)
5      vit_width:int = 256
6      vit_layers:int = 6
7      vit_heads:int = 8
8      # Text Transformer
9      text_width:int = 256
10     text_layers:int = 6
11     text_heads:int = 8
12     # Attention
13     dropout:float = 0.2
14     r_mlp:int = 4
15     bias:bool = False
16     # Training
17     augment_data:bool = True
18     num_workers:int = 0
19     batch_size:int = 128
20     lr:float = 5e-4
21     lr_min:float = 1e-5
22     weight_decay:float = 1e-4
23     epochs:int = 200
24     warmup_epochs:int = 5
25     grad_max_norm:float = 1.0
26     get_val_accuracy:bool = False
27     model_location:str = "../clip_fmnet.pt"
28
29 @dataclass
30 class PriorConfig:
31     # Diffusion
32     max_time:int = 1000
33     schedule:str = "cosine"
34     schedule_offset:float = 0.008
35     # Transformer Decoder
36     width:int = 256
37     n_layers:int = 6
38     n_heads:int = 8
39     # Attention
40     dropout:float = 0.2
41     r_mlp:int = 4
42     bias:bool = False
43     # Training
44     augment_data:bool = False
45     num_workers:int = 0
46     batch_size:int = 128
47     lr:float = 5e-4
48     lr_min:float = 1e-5

```

```

49     weight_decay:float = 1e-4
50     epochs:int = 150
51     warmup_epochs:int = 5
52     grad_max_norm:float = 1.0
53     model_location:str = "../prior_fmnnist.pt"
54
55     @dataclass
56     class DecoderConfig:
57         # Diffusion
58         max_time:int = 1000
59         schedule:str = "cosine"
60         # UNet
61         n_groups:int = 8
62         kernel_size:tuple[int, int] = (3,3)
63         model_channels:int = 32
64         cond_channels:int = 128
65         channel_ratios:list[int] = field(default_factory=lambda: [1, 2, 4, 8])
66         n_layer_blocks:int = 2
67         dropout:float = 0.1
68         use_scale_shift:bool = True
69         n_heads:int = 8
70         stride:int = 2
71         down_pool:bool = False
72         r_mlp:int = 4
73         bias:bool = False
74         text_layers:int = 4
75         n_img_tokens:int = 4
76         # Training
77         augment_data:bool = False
78         num_workers:int = 0
79         batch_size:int = 32
80         lr:float = 5e-4
81         lr_min:float = 1e-5
82         weight_decay:float = 1e-4
83         epochs:int = 100
84         warmup_epochs:int = 5
85         grad_max_norm:float = 1.0
86         sample_after_epoch:bool = True
87         model_location:str = "../decoder_fmnnist.pt"
88
89     @dataclass
90     class FMNNISTConfig:
91         latent_dim:int = 256
92         # Dataset Info
93         dataset:str = "fashion_mnnist"
94         data_location:str = "../datasets"
95         img_size:tuple[int,int] = (32,32)
96         img_channels:int = 1
97         vocab_size:int = 256
98         text_seq_length:int = 64

```

```

99     # Data Augmentation / Normalization
100     prob_hflip:float = 0.5
101     crop_padding:int = 4
102     train_mean:list[float] = field(default_factory=lambda: [0.285552])
103     train_std:list[float] = field(default_factory=lambda: [0.33848408])
104     # Training
105     train_val_split:tuple[int,int] = (50000, 10000)
106     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
107     # Model Configs
108     clip = CLIPConfig()
109     prior = PriorConfig()
110     decoder = DecoderConfig()

```

7.5 训练

对于这个文本到图像的扩散模型，我们实际上需要训练三个独立的模型：CLIP、扩散先验模型和扩散解码器 UNET。包含这三个训练脚本的所有代码会非常多。

对于数据集，所有图像的尺寸都从 (28,28) 调整为 (32,32)。图像也进行了归一化，平均值和标准差设置为训练样本的平均值和标准差。

```

1 transform = T.Compose([
2     T.Resize(config.img_size),
3     T.ToTensor(),
4     T.Normalize(config.train_mean, config.train_std)
5 ])

```

 Python

对于 CLIP 模型的训练集，我还通过随机水平翻转和随机裁剪实现了训练分割的数据增强。

```

1 transform = T.Compose([
2     T.Resize(config.img_size),
3     T.RandomHorizontalFlip(p=config.prob_hflip)
4     T.RandomCrop(config.img_size[0], padding=config.crop_padding)
5     T.ToTensor(),
6     T.Normalize(config.train_mean, config.train_std)
7 ])

```

 Python

在训练这三个模型时，当使用权重衰减时，我使用 AdamW 作为优化器，否则使用 Adam。

```

1 if config.clip.weight_decay == 0:
2     optimizer = Adam(clip.parameters(), lr=config.clip.lr)
3 else:
4     optimizer = AdamW(clip.parameters(), lr=config.clip.lr,
5                       weight_decay=config.clip.weight_decay)

```

 Python

对于所有模型，我还使用了带有线性预热的余弦退火学习率调度器。该调度器在每个 epoch 结束时更新。

```

1 if config.clip.warmup_epochs > 0:
2     warmup = lr_scheduler.LinearLR(optimizer=optimizer, start_factor=(1 /
3     config.clip.warmup_epochs), end_factor=1.0,
4     total_iters=(config.clip.warmup_epochs - 1), last_epoch=-1)
3
4 scheduler = lr_scheduler.CosineAnnealingLR(optimizer, T_max=(config.clip.epochs -
5 config.clip.warmup_epochs), eta_min=config.clip.lr_min)

```

 Python

所有模型还使用了最大范数为 1.0 的梯度剪裁，以帮助防止梯度爆炸。

```
1 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=config.  
[model].grad_max_norm)
```

 Python

在训练 CLIP 模型时，训练数据被随机分成训练集和验证集。当某个 epoch 的验证损失小于或等于前一个最低验证损失时，模型权重会被保存。

采样过程的第一步是随机生成仅包含噪声的图像。

```
1 B = prompts.shape[0]  
2 # Get completely noisy image  
3 img = torch.randn((B, config.img_channels, config.img_size[0], config.img_size[1]),  
device=config.device)
```

 Python

得到噪声图像后，我们将计算一些需要计算 x_{t-1} 的时间表值。

```
1 def get_schedule_values(config):  
2     schedule_values = {}  
3     schedule_values["betas"] = get_beta_schedule(config.decoder.schedule,  
config.decoder.max_time).to(config.device)  
4     schedule_values["alphas"] = 1.0 - schedule_values["betas"]  
5     schedule_values["alpha_bars"] = torch.cumprod(schedule_values["alphas"],  
axis = 0)  
6     schedule_values["sqrt_recip_alphas"] = torch.sqrt(1.0 /  
schedule_values["alphas"])  
7     schedule_values["sqrt_alpha_bars"] =  
torch.sqrt(schedule_values["alpha_bars"])  
8     schedule_values["sqrt_one_minus_alpha_bars"] = torch.sqrt(1.0 -  
schedule_values["alpha_bars"])  
9     schedule_values["alpha_bars_prev"] = torch.cat((torch.ones(1,  
device=config.device), schedule_values["alpha_bars"][:-1]))  
10    schedule_values["sigma"] = schedule_values["betas"] * (1.0 -  
schedule_values["alpha_bars_prev"]) / (1.0 - schedule_values["alpha_bars"])  
11    return schedule_values  
12  
13 schedule_values = get_schedule_values(config)
```

 Python

然后，我们需要以相反的顺序迭代遍历所有时间步，从 0 到 max_time-1。我们还需要扩展时间步，以便批次中的每条数据都有一个时间步。

```
1 for t in range(0, config.max_time)[::-1]:  
2     timesteps = torch.full((B,), t, device=config.device, dtype=torch.long)
```

 Python

对于每个时间步的采样，第一步是获取该时间步的调度值。获取该时间步的调度值后，需要对其进行扩展，使其维度数等于图像的维度数。

```
1 # Getting schedule values for timestep  
2 sqrt_recip_alphas_t = extract_and_expand(schedule_values["sqrt_recip_alphas"],  
timesteps, img.shape)  
3 betas_t = extract_and_expand(schedule_values["betas"], timesteps, img.shape)  
sqrt_one_minus_alpha_bars_t =  
4 extract_and_expand(schedule_values["sqrt_one_minus_alpha_bars"], timesteps,  
img.shape)  
5 sigma_t = extract_and_expand(schedule_values["sigma"], timesteps, img.shape)
```

 Python

之后，我们使用解码器模型来预测时间步长 t 的噪声。

```
1 # Predicting noise at timestep t with decoder  
2 pred_noise = decoder(img, timesteps, caption=prompt, mask=mask)
```

 Python

如果不是最终时间步，我们还需要生成随机噪声 z 。我们需要这个值和 σ_t ，因为我们的模型直接预测从 x_t 到 x_0 的噪声，而不是从 x_t 到 x_{t-1} 的噪声。因此，模型会预测从 x_t 到 x_0 的噪声，并将从 x_{t-1} 到 x_0 的噪声加回去。这样一来，只有从 x_t 到 x_{t-1} 的噪声被移除。最后一步不需要 σ_t 和 z 值，因为从 x_t 到 x_0 的预测噪声与从 x_t 到 x_{t-1} 的预测噪声相同。

```
1 # Generating random noise
2 z = torch.randn_like(img) if t > 0 else 0
```

Python

利用生成的噪声、方差计划值和预测噪声，我们可以计算时间步 $t-1$ 的图像。

```
1 # Calculating image at timestep t-1
2 img = sqrt_recip_alphas_t * (img - (betas_t / sqrt_one_minus_alpha_bars_t) *
  pred_noise) + (sigma_t * z)
3 img = torch.clamp(img, -1.0, 1.0)
```

Python

综合起来，采样和图像的代码将如下所示：

```
1 @torch.no_grad()
2 def sample_image(config, prompt, mask, schedule_values=None):
3     # Load decoder model
4     decoder = Decoder(config).to(config.device)
5     decoder.load_state_dict(torch.load(config.decoder.model_location,
6     map_location=config.device))
7     decoder.eval()
8     B = prompt.shape[0]
9     # Get completely noisy image
10    img = torch.randn((B, config.img_channels, config.img_size[0],
11    config.img_size[1]), device=config.device)
12    # Calculate schedule values
13    if schedule_values is None:
14        schedule_values = get_schedule_values(config)
15
16    for t in range(0, config.decoder.max_time)[::-1]:
17        # Setting the timesteps for all the items in the batch
18        timesteps = torch.full((B,), t, device=config.device, dtype=torch.long)
19
20        # Getting schedule values for timestep
21        sqrt_recip_alphas_t =
22        extract_and_expand(schedule_values["sqrt_recip_alphas"], timesteps,
23        img.shape)
24        betas_t = extract_and_expand(schedule_values["betas"], timesteps,
25        img.shape)
26        sqrt_one_minus_alpha_bars_t =
27        extract_and_expand(schedule_values["sqrt_one_minus_alpha_bars"], timesteps,
28        img.shape)
29        sigma_t = extract_and_expand(schedule_values["sigma"], timesteps,
30        img.shape)
31
32        # Predicting noise at timestep t with decoder
33        pred_noise = decoder(img, timesteps, caption=prompt, mask=mask)
34
35        # Generating random noise
36        z = torch.randn_like(img) if t > 0 else 0
```

Python


```
31
32     # Calculating image at timestep t-1
33     img = sqrt_recip_alphas_t * (img - (betas_t / sqrt_one_minus_alpha_bars_t)
34     * pred_noise) + (sigma_t * z)
35
36     img = torch.clamp(img, -1.0, 1.0)
37
38     return img
```